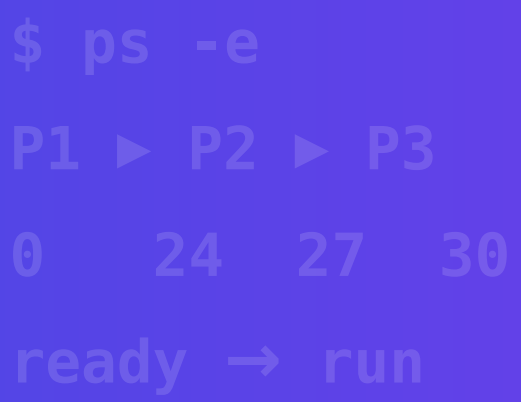



CMP103 · 3. Hafta · Vize ve genel sınav hazırlığı

# İşletim Sistemleri

## Kaynak yönetimi, süreçler, bellek ve CPU zamanlama

13 Ekim 2026 | Kaynak materyale dayalı öğretici çalışma notu

# İçindekiler


**Başlangıç** 4

Bu not nasıl kullanılmalı? . . . 4
Kaynak dosya ve sayfa etiketleri. . . . 4
Gereken ön bilgiler . . . 5
Dersin genel haritası . . . 5
Kutular ve etiketler ne anlama geliyor? . . . 6
Ders bilgileri ve varsayımlarım . . . 6
Çalışma planı önerisi . . . 7

**1 İşletim sistemi neden var?** 9

1.1 Bir bilgisayarda kaç çeşit program yaşar? . . . 9
1.2 İşletim sistemi tam olarak ne iş yapar? . . . 10
1.3 Soyutlama neden bu kadar önemli? . . . 11
1.4 İşletim sistemi katmanların neresinde? . . . 12
1.5 Bunu neden öğreniyoruz? Mühendislikte ödünleşim . . . 12
1.6 Kullanıcı kipi ve çekirdek kipi. . . . 13
1.7 Sistem çağrısı: çekirdeğe kontrollü giriş . . . 14
1.8 Bu bölümün bağlantıları . . . 15
1.9 Bölüm cevapları. . . . 16

**2 İşletim sistemi aileleri ve arayüzler** 18

2.1 Her cihaz aynı şeyi mi önemser? . . . 18
2.2 Gerçek zamanlı = çok hızlı mı? . . . 19
2.3 Yaygın işletim sistemi aileleri . . . 20
2.4 Grafik arayüz mü, komut satırı mı? . . . 21
2.5 Bir ürün için işletim sistemi seçmek . . . 21
2.6 Bu bölümün bağlantıları . . . 22
2.7 Bölüm cevapları . . . 23

**3 Tarihçe ve bilgisayarın kaynakları** 24

3.1 Dört dönemde işletim sistemleri . . . 24
3.2 Toplu işlem sistemleri: işleri grup grup çalıştırmak . . . 25
3.3 Çoklu programlama neden ortaya çıktı? . . . 25
3.4 Çoklu programlama ile zaman paylaşımı farkı. . . . 26
3.5 Modern bir işletim sisteminden beklentiler . . . 27
3.6 Basit bir bilgisayardaki kaynaklar . . . 27
3.7 Hangi iş hangi kaynağı zorlar? . . . 28
3.8 İşlemci çekirdeği, iş parçacığı ve süreç . . . 28
3.9 Bu bölümün bağlantıları . . . 29

3.10 Bölüm cevapları . . . 31

**4 Temel görevler ve çalışma kipleri 32**
4.1 İşletim sisteminin beş görevi . . . 32
4.2 Görev 1: Kullanıcıya bir arayüz sunmak . . . 33
4.3 Görev 2: Programları çalıştırmak . . . 34
4.4 Görev 3: Aygıtları yönetmek ve kesmeler . . . 34
4.5 Görev 4: Dosya ve klasörleri düzenlemek . . . 35
4.6 Görev 5: Koruma ve güvenlik . . . 35
4.7 "Aynı anda birden fazla iş"in dört farklı anlamı . . . 36
4.8 Slayt örneği: I/O beklerken ne olur? . . . 38
4.9 Ara durak: şimdiye kadar . . . 39
4.10 Bu bölümün bağlantıları . . . 39
4.11 Bölüm cevapları . . . 41

**5 Bellek yönetimi 42**
5.1 Bellek neden yönetilmek zorunda? . . . 42
5.2 Dinamik bölümleme ve zamanla oluşan delikler . . . 43
5.3 İç ve dış parçalanma . . . 43
5.4 Yerleştirme algoritmaları . . . 44
5.5 Sıkıştırma: delikleri birleştirmek . . . 46
5.6 Takas: belleği diske ödünç vermek . . . 46
5.7 Sanal bellek fikri . . . 47
5.8 Sayfalama: sanal belleğin nasıl çalıştığı . . . 47
5.9 Dosya yöneticisinin rolü . . . 49
5.10 Bu bölümün bağlantıları . . . 49
5.11 Bölüm cevapları . . . 50

**6 Süreç yönetimi 52**
6.1 Program ile süreç aynı şey değil . . . 52
6.2 Süreç Denetim Bloğu (PCB) . . . 53
6.3 Bir sürecin hayatı: beş durum . . . 53
6.4 Zamanlayıcı ve kuyruklar . . . 55
6.5 Bağlam değiştirme . . . 56
6.6 Kesintisiz ve kesintili zamanlama . . . 56
6.7 CPU patlamaları ve I/O patlamaları . . . 57
6.8 Zamanlama ölçütleri . . . 57
6.9 "En iyi" algoritma diye bir şey yok . . . 58
6.10 Bu bölümün bağlantıları . . . 59
6.11 Bölüm cevapları . . . 60

**7 CPU zamanlama algoritmaları 61**
7.1 Dört temel yaklaşım . . . 61
7.2 FCFS: ilk gelen ilk hizmet alır . . . 62
7.3 SJF: en kısa iş önce . . . 64
7.4 SRTF: en kısa kalan süre önce . . . 65
7.5 Öncelikli zamanlama . . . 66
7.6 Round Robin: sırayla, dilim dilim . . . 67
7.7 Aynı süreçler, dört algoritma . . . 69

7.8 Hangi algoritma ne zaman? . . . 70
7.9 Slayt alıştırması: algoritmayı seç. . . . 71
7.10 Bu bölümün bağlantıları . . . 72
7.11 Bölüm cevapları. . . . 73

**8 Bellek hiyerarşisi 75**
8.1 CPU ile bellek arasındaki hız uçurumu . . . 75
8.2 Bellek hiyerarşisi piramidi . . . 76
8.3 Önbellek nasıl yardım eder? . . . 77
8.4 Yerellik ilkesi: önbellek neden işe yarıyor? . . . 77
8.5 Haftanın özeti ve gelecek hafta . . . 79
8.6 Bu bölümün bağlantıları . . . 79
8.7 Bölüm cevapları. . . . 80

**9 Konular arası bağlantılar 81**
9.1 Kavramlar birbirine nasıl bağlanıyor? . . . 81
9.2 Karşılaştırma tabloları. . . . 81
9.3 Bölümler arası “uyum kontrolü” . . . 84

**10 Sınav öncesi hızlı tekrar 85**
10.1 Temel tanımlar . . . 85
10.2 Formüller . . . 86
10.3 Sayısal sonuçlar (slayt örnekleri) . . . 86
10.4 Kritik ayrımlar . . . 86
10.5 En sık yapılan hatalar . . . 86
10.6 Sınav öncesi kontrol listesi . . . 87

**11 Deneme sınavı 88**
Bölüm A: Çoktan seçmeli. . . . 88
Bölüm B: Açık uçlu ve hesap soruları . . . 90

**12 Deneme sınavı cevap anahtarı 92**
Bölüm A cevapları. . . . 92
Bölüm B cevapları. . . . 93
Cevapların bağımsız kontrolü . . . 96

**13 Terim sözlüğü 97**

**14 Hocaya sorulacak noktalar 100**

**15 Kaynak kapsamı kontrol tablosu 102**

# Başlangıç

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Bu not, CMP103 dersinin 3. hafta slaytlarını (**İşletim Sistemleri**, 95 sayfa) sıfırdan öğrenmen ve vize ile genel sınava hazırlanman için yazıldı. Slaytlardaki her ana başlığı işledi, slaytlardaki bütün sayısal örnekleri adım adım çözdü, sınavda sorulabilecek biçimleri gösterdi. Önce nasıl kullanacağını, sonra dersin genel haritasını görelim.

## Bu not nasıl kullanılmalı?

1. **İlk geçiş (öğrenme):** 1. bölümden 8. bölüme sırayla oku. Sıra, slayt sırasıyla aynıdır; her bölüm bir öncekine dayanır. Her "Durup düşün" kutusunda dur, cevabı kafanda kur, sonra bölüm sonundaki cevaplarla karşılaştır.
2. **Alıştırma:** "Kendin çöz" sorularını kâğıtta çöz. Zamanlama ve bellek yerleştirme (5. ve 7. bölüm) ancak elle çizerek oturur; okumak yetmez.
3. **Tekrar:** Sınavdan önce *Sınav öncesi hızlı tekrar* bölümünü oku (tek oturuşluk).
4. **Deneme:** Deneme sınavını cevap anahtarına bakmadan, süre tutarak çöz. Sonra anahtarla karşılaştır; yanlışlarını ilgili bölümün "Sık yapılan hatalar" kutusuyla eşleştir.

### Önerilen sıra ve süre

| Bölüm | Konu | Tahmini süre | Zorluk |
|---|---|---|---|
| 1 | İşletim sistemi neden var? | 40 dk | ●○○ |
| 2 | Aileler ve arayüzler | 30 dk | ●○○ |
| 3 | Tarihçe ve kaynaklar | 35 dk | ●●○ |
| 4 | Temel görevler ve kipler | 50 dk | ●●○ |
| 5 | Bellek yönetimi | 55 dk | ●●○ |
| 6 | Süreç yönetimi | 50 dk | ●●○ |
| 7 | CPU zamanlama | 75 dk | ●●● |
| 8 | Bellek hiyerarşisi | 30 dk | ●●○ |

Süreler okuma + örnekleri çözme için kaba tahminlerdir; senin hızına göre değişir.

## Kaynak dosya ve sayfa etiketleri

Bu notun tek kaynağı vardır:

- **D1** = `CMP103_03.pdf` — "CMP103 · Week 03 — Operating Systems" (Biruni Üniversitesi, 95 sayfa).

Her başlığın yanındaki gri rozet kaynak konumunu gösterir. Okumak için bir kural bilmen yeter:

- **Fiziksel sayfa 1–70:** PDF sayfa numarası ile slayt üzerindeki numara aynıdır. Örnek: `D1 s. 52`.
- **Fiziksel sayfa 72'den sonra:** Slayt numaraları PDF sayfasından **1 fazladır**, çünkü slaytlarda 72 numaralı slayt yoktur (muhtemelen gizli bir slayt). Bu yüzden iki numara birlikte yazılır: `D1 s. 74 / slayt 75`.

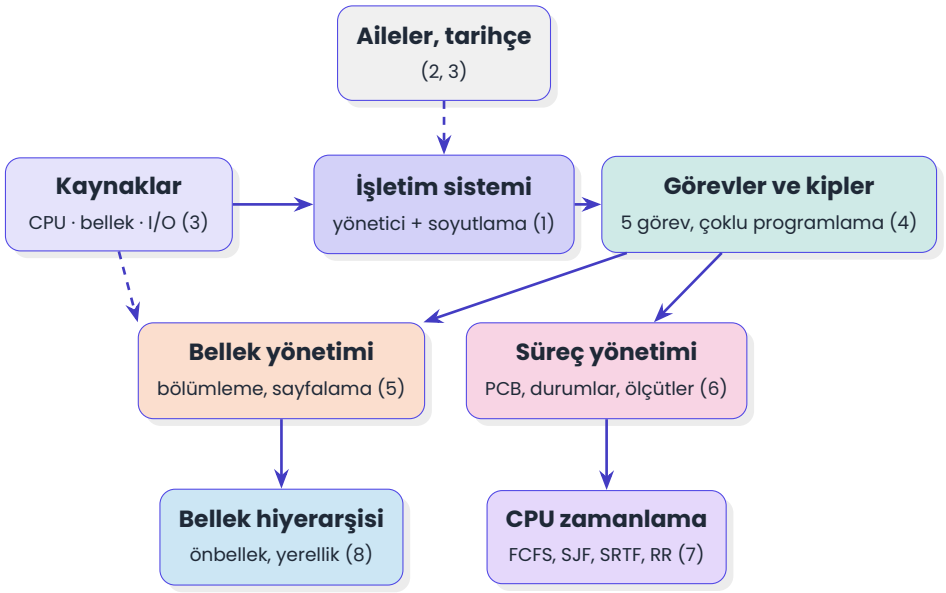
Slaytta olmayıp bu nota benim eklediğim bilgiler **kaynak dışı**, **ek bilgi** veya **slaytta yok** diye işaretlidir; sınavda hocanın slayttaki halini esas al.

## Gereken ön bilgiler

Ders giriş seviyesindedir; özel bir ön bilgi gerekmez. Şunları biliyor olman yeterli:

- Bilgisayarın ana parçaları: işlemci (CPU), bellek (RAM), depolama (disk), ağ.
- Bir programı açıp kapatmanın ne demek olduğu (kullanıcı gözüyle).
- Temel aritmetik: toplama, çıkarma, ortalama, yüzde. Kodlama bilgisi **gerekmez**; slaytlarda kod yoktur.

## Dersin genel haritası




Şekil 1: Dersin haritası. Parantez içindeki sayılar bölüm numaralarıdır. Okun yönü "bunun üzerine kurulur" demektir.

Haritayı şöyle oku: bilgisayarın **kaynakları** (CPU, bellek, I/O) vardır; **işletim sistemi** bunları yöneten ve soyutlayan yazılımdır. Görevlerinden ikisi bu hafta ayrıntılı işlenir: **belleği** paylaştırmak (5. bölüm) ve **CPU'yu** süreçler arasında paylaştırmak (6. ve 7. bölüm). Bellek hiyerarşisi (8. bölüm), belleğin neden katmanlı olduğunu açıklayarak haftayı kapatır.

## Kutular ve etiketler ne anlama geliyor?

**∑ TANIM / FORMÜL**

**TANIM**: Sınavda aynen bilmen gereken tanım ve formüller. Kenarında sarı vurgu vardır.

**⚠ SINAVDA DİKKAT**

**SINAVDA DİKKAT**: Yüksek öncelikli kavram, formül ve karıştırılan ayrımlar (kırmızı).

**▶ ÖRNEK 0.1 — Çözümlü örnek**

**ÖRNEK**: Adım adım çözülmüş soru (mavi). Önce kendin dene, sonra çözüme bak.

**✘ SIK YAPILAN HATA**

**SIK YAPILAN HATA**: Yanlış düşünce → doğrusu → neden yanlış (turuncu).

**✎ DURUP DÜŞÜN**

**DURUP DÜŞÜN / KENDİN ÇÖZ**: Cevabı bölüm sonunda (yeşil).

**+ EK BİLGİ (KAYNAK DIŞI) — Ek bilgi**

**EK BİLGİ**: Slaytta olmayan, anlamayı kolaylaştıran ek (mor, "kaynak dışı").

**★ MERAKLISINA (SINAV DIŞI)**

**MERAKLISINA**: Sınavda çıkmaz; merakını gidermek için (pembe).

**⚡ HIZLI TEKRAR**

**HIZLI TEKRAR**: Her bölümün özü, neyi ezberleyeceğin ve neyi türetebileceğin (indigo).

Rozetler: SINAV ÖNCELİĞİ: YÜKSEK sınav önceliğini, ❖ Kaynak: D1 s. 6 kaynak konumunu, ✔ hesapla doğrulandı ise sayısal sonucun Python benzetimiyle kontrol edildiğini gösterir.

## Ders bilgileri ve varsayımlarım

| Bilgi | Değer | Nereden çıkardım? |
|---|---|---|
| Ders | CMP103 (Bilgisayar Mühendisliğine Giriş ve Kariyer Planlama) | Slayt kapağı |
| Kapsam | 3. hafta: İşletim Sistemleri (D1, 95 sayfa) | Dosya adı ve içerik |
| Üniversite | Biruni Üniversitesi, 2026 güz | Slayt kapağı |
| Sınav | Vize ve genel sınav; tarih bilinmiyor | Senin isteğinden |
| Sınav formatı | Bilinmiyor; **karışık** varsaydım (çoktan seçmeli + klasik/hesap) | Geçmiş soru verilmediği için varsayım |
| Dil | Türkçe anlatım, terimlerin İngilizcesi parantez içinde | Tercihin |
| Kod | Kaynakta programlama kodu yok; yalnızca birkaç komut satırı örneği var | Slayt incelemesi |

⚠ **SINAVDA DİKKAT**

**Önemli:** Hocanın gerçek sınavını görmedim. Bu nottaki "sınav önceliği" etiketleri **tahmindir**; slayttaki vurgulara, tekrarlara, formül ve tanım olmalarına ve soru üretmeye elverişli olmalarına dayanır. "Kesin çıkar" anlamına gelmez. Hocanın derste özellikle vurguladığı konular varsa onları öne al.

## Çalışma planı önerisi

Sınav tarihi belli olmadığı için "sınava kaç gün kala" biçiminde yazdım.

| Zaman | Ne yap? |
|---|---|
| **7 gün kala** | 1.–4. bölümleri oku; "Durup düşün" sorularını çöz. Her gün bir iki bölüm yeter. |
| **5–4 gün kala** | 5. ve 6. bölümler. Bellek yerleştirme alıştırmalarını (ilk/en iyi/en kötü/sonraki uyan) kâğıtta kendin yap. |
| **3 gün kala** | 7. bölüm (zamanlama). Her algoritmayla en az iki Gantt şeması çiz ve ortalamaları hesapla. 8. bölümü oku. |
| **2 gün kala** | Deneme sınavını süre tutarak çöz, anahtarla karşılaştır, yanlışlarını bölümlere dön. |
| **1 gün kala** | Yalnızca *Sınav öncesi hızlı tekrar* ve her bölümün "Sık yapılan hatalar" kutuları. Yeni konu açma; erken uyu. |

| Zaman | Ne yap? |
| --- | --- |
| **Sınav sabahı** | Hızlı tekrar bölümünü bir kez oku, formülleri (TAT = C − A, W = TAT − B, R = S − A) kâğıda yaz. |

BÖLÜM 1

# İşletim sistemi neden var?

◆ **BU BÖLÜMDE** Okuma süresi: 40 dk Zorluk: ●○○

- Bir bilgisayarda çalışan üç program türünü (uygulama, sistem programı, aygıt sürücüsü) ayırt etmek
- İşletim sistemini iki cümleyle tanımlamak: **kaynak yöneticisi** ve **soyutlama katmanı**
- Soyutlamanın uygulama yazan birine ne kazandırdığını örnekle açıklamak
- Kullanıcı kipi ile çekirdek kipi arasındaki farkı ve bunun güvenlik için neden şart olduğunu anlatmak
- Bir sistem çağrısının (system call) baştan sona nasıl ilerlediğini adım adım çizebilmek

**SINAV ÖNCELİĞİ: YÜKSEK** ❖ Kaynak: D1 s. 5–12

*Neden yüksek?* Haftanın geri kalanı (bellek, süreç, zamanlama) bu bölümdeki "kaynak yöneticisi + soyutlama" fikrinin üzerine kurulu ve tanım soruları en kolay buradan gelir.

→ **BU KONUYA NEDEN GEÇİYORUZ?**

Bu ders bilgisayar mühendisliğine bir giriş; bu hafta ilk kez "bilgisayarın içinde kim neyi yönetiyor?" sorusunu soruyoruz. İşlemciyi, RAM'i, diski tek tek biliyor olabilirsin ama bunlar kendi başlarına birbirleriyle anlaşamaz. Telefonunda aynı anda müzik çalarken mesaj yazabiliyor, oyun oynarken bildirim alabiliyorsan bunu sahnede görmediğin bir yönetici sağlıyor: işletim sistemi. Bu bölümde o yöneticinin kim olduğunu, nerede oturduğunu ve programlarla nasıl konuştuğunu öğreneceğiz.

## 1.1 Bir bilgisayarda kaç çeşit program yaşar?

❖ Kaynak: D1 s. 6

Bilgisayarındaki her şey "program", ama hepsi aynı işi yapmıyor. Slayt bunları üç gruba ayırıyor ve bu ayrım sınavda çoktan seçmeliye çok uygun.

| Tür | Kimin işine yarar? | Örnek | Slayttaki ifade |
|---|---|---|---|
| Uygulama programı (**application program**) | Doğrudan kullanıcının, onun bir işini yapar | Tarayıcı, ofis yazılımı, IDE, oyun | *Help the user perform a task* |
| Sistem programı (**system program**) | Sistemin kendisini destekler, işleri kolaylaştırır | Dosya araçları, derleyici (compiler), kabuk (shell), servisler | *Support the system* |

| Tür | Kimin işine yarar? | Örnek | Slayttaki ifade |
|---|---|---|---|
| Aygıt sürücüsü (**device driver**) | İşletim sistemiyle belirli bir donanım arasında tercümanlık yapar | Ekran kartı sürücüsü, yazıcı sürücüsü | *Enable communication between the OS and a specific hardware device* |

Ayrımı yaparken kendine tek bir soru sor: **"Bu program kimin için çalışıyor?"** Kullanıcının bir hedefi (yazı yazmak, oyun oynamak) için çalışıyorsa uygulama; sistemi kullanılabilir ve bakımlı tutmak için çalışıyorsa sistem programı; belirli bir donanım parçasının "dilini" işletim sistemine çeviriyorsa sürücü.

**▶ ÖRNEK 1.1 — Programı doğru gruba koy**

**Soru.** Şunları sınıflandır: (a) Spotify, (b) C derleyicisi `gcc`, (c) yeni aldığın oyuncu faresinin sürücüsü, (d) Windows'taki Görev Yöneticisi.
**Çözüm.** Her biri için "kimin için çalışıyor?" sorusunu soruyoruz.

- (a) Spotify senin müzik dinleme isteğine hizmet ediyor → **uygulama programı**.
- (b) `gcc` kaynak kodu makine koduna çeviriyor; slayt derleyiciyi açıkça sistem programı örneği olarak veriyor → **sistem programı**. (Programcı için bir "araç" olsa da sınavda slaytın sınıflandırmasını esas al.)
- (c) Farenin hangi sinyali gönderdiğini işletim sistemine çeviren yazılım → **aygıt sürücüsü**.
- (d) CPU ve bellek kullanımını izleyen araç → **sistem programı** (slayt 36'daki "monitoring tools" grubu).

**Sık hata:** IDE'yi sistem programı sanmak. Slayt IDE'yi uygulama programı örnekleri arasında sayıyor.

## 1.2 İşletim sistemi tam olarak ne iş yapar?

❖ Kaynak: D1 s. 7

Slayt dört madde veriyor, sonra hepsini tek cümlede topluyor. Önce maddeleri tek tek açalım:

1. **Donanım ayrıntılarını uygulamalardan gizleyen bir yazılım katmanıdır.** Uygulama "şu dosyayı aç" der; diskin hangi kafasının hangi izde duracağını bilmek zorunda değildir.
2. **CPU zamanını, belleği, depolamayı, ağı ve giriş/çıkış (I/O) aygıtlarını yönetir.** Bunlar bilgisayarın sınırlı kaynaklarıdır; birden fazla program aynı anda bunları ister.
3. **Programlara tutarlı ve güvenli hizmetler sunar.** Her program dosya açmak için aynı yolu kullanır; kimse başkasının dosyasına izinsiz erişemez.
4. **Aynı kaynakları paylaşan işlerin birbirine karışmasını engeller.** İki program aynı bellek bölgesine yazmaya kalkarsa felaket olur; işletim sistemi bunu önler.

**∑ TANIM / FORMÜL — İşletim sistemi (operating system, OS)**

Donanım ile uygulamalar arasında duran; donanımın karmaşıklığını gizleyip programlara basit ve güvenli bir arayüz sunan (**soyutlama katmanı**) ve CPU, bellek, depolama, ağ ve I/O gibi paylaşılan kaynakları programlar arasında dağıtıp denetleyen (**kaynak yöneticisi**) yazılım. *Slayttaki özet:* "In short: an operating system is a **resource manager** and an **abstraction layer**."

**Sınavda tanım istenirse mutlaka geçmesi gereken anahtar kelimeler:** donanım ile uygulama arasında, kaynak yönetimi (CPU, bellek, I/O), soyutlama, güvenli/koruma.

Bu iki rol birbirinden farklı sorulara cevap verir. *Kaynak yöneticisi* rolü "kim, neyi, ne zaman, ne kadar kullanacak?" sorusunu; *soyutlama* rolü ise "programlar donanımla nasıl konuşacak?" sorusunu çözer.

### 1.2.1 Bir benzetme: restoranın salon müdürü

Kalabalık bir restoranı düşün. Mutfak (donanım) tek; ocak sayısı, şef sayısı sınırlı. Masalardaki müşteriler (programlar) aynı anda yemek istiyor.

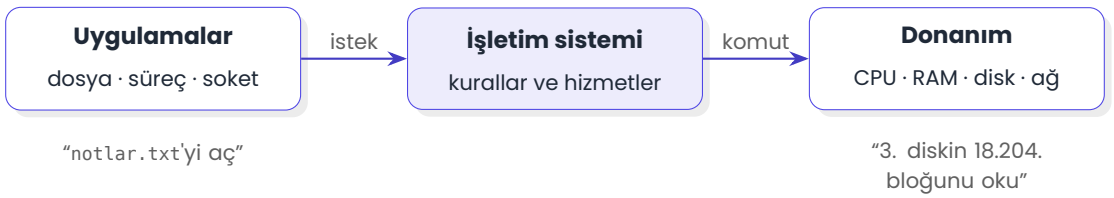
- **Soyutlama:** Müşteri mutfağa girip ocağı yakmaz. Menüden sipariş verir; garson siparişi mutfağa iletir. Menü, mutfağın karmaşıklığını gizleyen arayüzdür.
- **Kaynak yönetimi:** Salon müdürü hangi masanın siparişinin önce hazırlanacağına karar verir, bir masanın bütün ocakları kapatmasına izin vermez.
- **Koruma:** Bir müşteri başka masanın tabağına uzanamaz.

*Benzetmenin bozulduğu yer:* Restoranda müdür, yemeğini yarısına kadar yemiş bir müşterinin tabağını alıp başka masaya vermez. İşletim sistemi ise CPU'yu çalışan bir programın elinden alıp başkasına verebilir (buna **preemption**, kesme diyeceğiz; 4. ve 6. bölümde gelecek). Ayrıca işletim sisteminin "müdür"ü de aynı mutfakta, aynı CPU'da çalışan bir programdır; restoranda müdür yemek pişirmek için ocak işgal etmez.

## 1.3 Soyutlama neden bu kadar önemli?

❖ Kaynak: D1 s. 8

Slaytın şeması üç kutudan oluşuyor: uygulamalar *dosya, süreç, soket* gibi kavramlarla konuşur; işletim sistemi *kurallar ve hizmetler* sunar; donanım ise *CPU, RAM, disk, ağ*tır.




**Şekil 1.1:** Soyutlama: uygulama yüksek seviyeli bir istek verir, ayrıntıyı işletim sistemi çevirir. Alttaki iki satır aynı işin iki farklı seviyedeki karşılığıdır.

Soyutlama olmasaydı ne olurdu? Bir metin editörü yazdığını düşün. Kaydet butonuna basıldığında kodun şunları bilmek zorunda kalırdı: bilgisayarda SSD mi HDD mi var, hangi marka, hangi komut setiyle konuşuluyor, dosyanın parçaları diskin neresinde, boş yer nerede... Üstelik bunu piyasadaki her disk modeli için ayrı ayrı yazman gerekirdi. Her uygulama bu işi yeniden yapsaydı hem inanılmaz zaman kaybı olurdu hem de bir uygulamanın hatası başka bir uygulamanın dosyasını bozabilirdi.

Soyutlamayla editörün kodu yalnızca "bu dosyayı aç, şu baytları yaz" der. Slaytın vurgusu tam olarak bu: *"An application does not need to manage disk sectors or device registers directly."* (Uygulamanın disk sektörlerini ya da aygıt yazmaçlarını doğrudan yönetmesi gerekmez.)

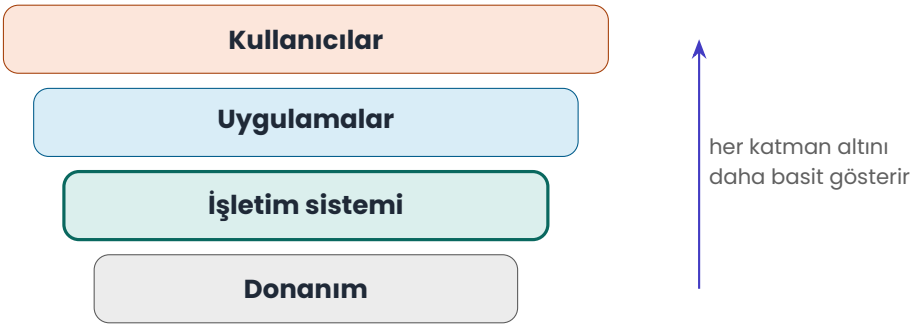
**✎ DURUP DÜŞÜN**

1. Soyutlamanın uygulama geliştiricisine sağladığı iki kazancı say.
2. Sence soyutlamanın bir *bedeli* olabilir mi? (İpucu: aradaki her katman bir iş yapar.)

## 1.4 İşletim sistemi katmanların neresinde?

❖ Kaynak: D1 s. 9

Slayt 9 bilgisayarı üst üste dört katman olarak çiziyor. Her katman, altındaki ayrıntıları üstündekine daha basit bir arayüzle sunar.




**Şekil 1.2:** Katmanlı görünüm (D1 s. 9). İşletim sistemi uygulamalar ile donanım arasındadır; kullanıcı donanıma çoğunlukla uygulamalar üzerinden ulaşır.

Bu şekilden çıkarman gereken iki şey var. Birincisi, kullanıcı normalde donanıma doğrudan dokunmaz; araya en az iki katman girer. İkincisi, bu bir *arayüzler zinciri*dir: kullanıcı uygulamanın arayüzünü, uygulama işletim sisteminin arayüzünü (sistem çağrılarını), işletim sistemi de donanımın arayüzünü (sürücüler aracılığıyla) kullanır.

## 1.5 Bunu neden öğreniyoruz? Mühendislikte ödünleşim

❖ Kaynak: D1 s. 10

Slayt 10 altı kavram sıralıyor: **maliyet (cost), performans (performance), güvenlik (security), işlevsellik (functionality), karmaşıklık (complexity), adalet (fairness)**. Alt kutudaki cümle asıl mesaj: işletim sistemi tasarımı, mühendislikteki *ödünleşimlerin* (**trade-off**) güçlü bir örneğidir.

Ödünleşim, birini iyileştirirken diğerinden vazgeçmek demektir. Hepsini aynı anda en iyiye çıkaramazsın:

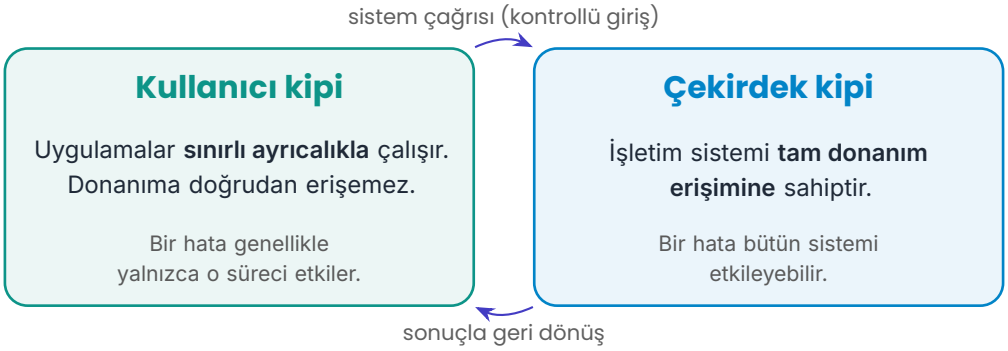
| Kazanmak istediğin | Neyi feda edebilirsin? | Somut örnek |
|---|---|---|
| Güvenlik | Performans | Her dosya erişiminde izin kontrolü yapmak zaman alır |
| İşlevsellik | Karmaşıklık, maliyet | Her yeni özellik daha çok kod, daha çok hata riski demektir |
| Adalet | Performans | Herkese sırayla CPU vermek sık geçiş yapmayı, yani ek yükü gerektirir (7. bölümdeki Round Robin) |
| Düşük maliyet | Performans, işlevsellik | Ucuz bir akıllı saatin işletim sistemi çok sade olmak zorundadır |

Bu tablodaki "adalet ↔ performans" çatışmasını aklında tut; CPU zamanlama bölümünde bunu sayılarla göreceksin.

## 1.6 Kullanıcı kipi ve çekirdek kipi

❖ Kaynak: D1 s. 11

Şimdi işin püf noktası geliyor: işletim sistemi programları birbirinden nasıl koruyor? Program "ben kötü niyetli değilim" dedi diye ona güvenemez. Cevap, işlemcinin iki farklı **ayrıcalık seviyesi**nde (**privilege level**) çalışabilmesi.




Şekil 1.3: İki kip ve aralarındaki tek meşru kapı: sistem çağrısı (D1 s. 11–12).

- **Kullanıcı kipi (user mode):** Uygulamalar burada çalışır. Bazı tehlikeli komutları (örneğin bir aygıtı doğrudan programlamak, başka bir sürecin belleğine yazmak) çalıştırmaları yasaktır. Bir uygulama çökerse genellikle yalnızca o süreç kapanır.
- **Çekirdek kipi (kernel mode):** İşletim sisteminin çekirdeği (**kernel**) burada çalışır; her şeye erişebilir. Bu yüzden burada yapılan bir hata bütün sistemi çökertebilir.

Slaytın vurguladığı cümle: *"Privilege separation is fundamental to protection and security."* Yani **ayrıcalık ayrımı**, koruma ve güvenliğin temelidir.

**+ EK BİLGİ (KAYNAK DIŞI) — Bu ayrımı kim zorluyor?**

Ayrım yazılımla "rica edilerek" değil, **donanımla** sağlanır: işlemci o anda hangi kipte çalıştığını tutan bir bit (mod biti) taşır ve kullanıcı kipindeyken ayrıcalıklı bir komut çalıştırılmaya kalkılırsa komutu yapmaz, kontrolü işletim sistemine devreder. Slayt bu mekanizmanın ayrıntısına girmiyor; sınavda "neden program bu kontrolü atlayamaz?" diye sorulursa "çünkü ayrım işlemci donanımında uygulanır" demen yeterli.

**▶ ÖRNEK 1.2 — Hangi hata neyi çökertir?**

**Soru.** (a) Bir tarayıcı sekmesi hatalı bir işlem yüzünden çöküyor. (b) Hatalı bir ekran kartı sürücüsü geçersiz bir bellek adresine yazıyor. Her durumda ne kadar büyük bir zarar beklersin? Neden?

**Çözüm.**

- *Verilen:* (a) kullanıcı kipindeki bir uygulama, (b) çekirdek kipinde çalışan bir sürücü.
- *Yöntem:* Hatanın hangi kipte oluştuğuna bakıyoruz; çünkü slayt 11'e göre zararın büyüklüğünü kip belirliyor.
- (a) Kullanıcı kipi → hata genellikle **yalnızca o süreci** etkiler. İşletim sistemi süreci sonlandırır, diğer programlar çalışmaya devam eder.

- (b) Sürücüler genellikle çekirdeğin parçası olarak, çekirdek kipinde çalışır → hata **bütün sistemi** etkileyebilir (Windows'taki "mavi ekran" bunun bilinen örneğidir).

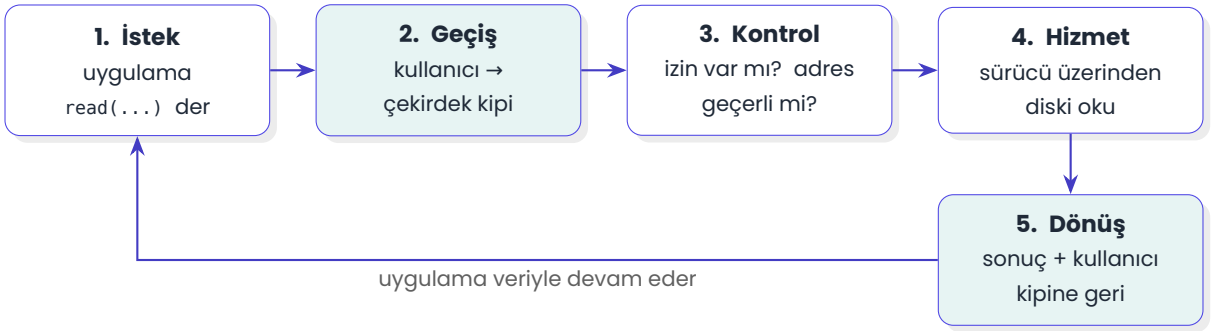
**Sonuç:** Hata ne kadar ayrıcalıklı bir katmanda olursa zarar o kadar büyük olabilir.
**Aynı mantık nerede işe yarar?** "Neden her programa tam yetki verilmez?" tipindeki bütün sorularda.

## 1.7 Sistem çağrısı: çekirdeğe kontrollü giriş

❖ Kaynak: D1 s. 12

Uygulama donanıma doğrudan dokunamıyorsa, bir dosya okumak istediğinde ne yapacak? İşletim sisteminden **rica edecek**. Bu ricanın resmî adı **sistem çağrısı (system call)**.

Slayt 12'deki akış: *Uygulamanın isteği* (open, read, write) → *Sistem çağrısı* (çekirdeğe geçiş) → *İşletim sistemi* (kontrol, hizmet, sonuç).




**Şekil 1.4:** Bir sistem çağrısının yaşam döngüsü. Slayttaki "check · service · result" üçlüsü 3., 4. ve 5. adımlardır.

Neden bu kadar tören var? Çünkü "kontrol" adımı olmadan güvenlik olmaz. İşletim sistemi her istekte şunu sorar: *Bu süreç bu dosyayı okumaya yetkili mi? Verdiği bellek adresi gerçekten kendisine mi ait?* Slaytın vurgusu: *"The application accesses hardware through a defined interface, not directly."* (Uygulama donanıma doğrudan değil, tanımlı bir arayüz üzerinden erişir.)

### ▶ ÖRNEK 1.3 — Python'da bir dosya okumak

**Soru.** Bir Python programında `open("notlar.txt").read()` satırı çalıştığında işletim sistemi tarafında neler olur? Slayt 11–12'deki kavramlarla anlat.
**Çözüm.**

1. Program kullanıcı kipinde çalışıyor; diske doğrudan erişme yetkisi yok.
2. Python, dosyayı açmak için işletim sistemine bir **open** sistem çağrısı yapar → işlemci **çekirdek kipine** geçer.
3. İşletim sistemi **kontrol eder:** dosya var mı, bu kullanıcının okuma izni var mı?
4. İzin varsa dosyayı açar ve programa açık dosyayı temsil eden bir numara döndürür (buna 5. bölümde **dosya tanımlayıcısı**, file descriptor diyeceğiz). Ardından **kullanıcı kipine** dönülür.
5. `.read()` için bu kez **read** sistem çağrısı yapılır; işletim sistemi sürücü aracılığıyla veriyi diskten okur ve programın belleğine kopyalar.

**Sonuç:** Tek satırlık Python kodu en az iki sistem çağrısına (open, read) ve her birinde iki kip geçişine yol açar.
**Bu tip soruda sık hata:** "Python dosyayı diskten kendisi okur" demek. Okuyan her zaman işletim sistemidir; program yalnızca ister.

**★ MERAKLISINA (SINAV DIŞI)**

Linux çekirdeği x86-64 işlemcilerde birkaç yüz farklı sistem çağrısı sunar ve her birinin bir numarası vardır. Yazdığın en basit "Merhaba dünya" programı bile ekrana yazmak için bir **write** sistem çağrısı yapar. Linux'ta `strace ./program` komutuyla bir programın yaptığı bütün sistem çağrılarını canlı izleyebilirsin; ileride İşletim Sistemleri dersinde bu araçla bol bol karşılaşacaksın.

### 1.7.1 Yarı çözümlü örnek

**▶ ÖRNEK 1.4 — Yarı çözümlü: kayıt düğmesi**

Bir metin editöründe "Kaydet"e bastın. Aşağıdaki adımları tamamla:

1. Editör kullanıcı kipindedir ve dosyaya yazmak için **write** sistem çağrısı yapar.
2. İşlemci ..... kipine geçer.
3. İşletim sistemi ..... kontrol eder.
4. ..... aracılığıyla veri diske yazılır ve sonuç editöre döner.

*Cevap bölüm sonunda.*

**✎ KENDİN ÇÖZ**

Bir öğrenci şöyle diyor: "Oyun programları çok hızlı olmak zorunda; o yüzden onları doğrudan çekirdek kipinde çalıştırmak mantıklı olur." Bu öneriye slayt 10 ve 11'deki kavramları (ödünleşim, ayrıcalık ayrımı) kullanarak 3–4 cümlelik bir cevap yaz.

## 1.8 Bu bölümün bağlantıları

- *Kaynak yöneticisi* rolü → 5. bölüm (belleğin paylaştırılması), 6–7. bölüm (CPU'nun paylaştırılması).
- *Soyutlama* rolü → 5. bölümdeki **sanal bellek** (her süreç büyük ve bitişik bir bellek görür) ve dosyalar (disk bloklarını isimli dosyalar olarak görmek).
- *Kullanıcı/çekirdek kipi* → 4. bölümdeki koruma ve güvenlik görevi; 6. bölümdeki bağlam değiştirmenin neden çekirdekte yapıldığı.

**⚠ SINAVDA DİKKAT**

- Tanımın iki yüzü: **kaynak yöneticisi + soyutlama katmanı.** Biri eksik kalırsa tanım eksik sayılır.
- Kullanıcı kipindeki hata → genellikle **bir süreç**; çekirdek kipindeki hata → **bütün sistem.**
- Sistem çağrısı = uygulamanın donanıma ulaşmasının **tek meşru yolu**; içinde *kip geçişi + kontrol* vardır.

Kolay karıştırılan ikili:

| Kavram | Ne? | Kim kullanır? |
|---|---|---|
| Sistem programı | Derleyici, kabuk gibi destek yazılımı (çoğu kullanıcı kipinde çalışır) | Kullanıcı veya sistem |
| Sistem çağrısı | Çekirdeğe yapılan hizmet isteği (open, read, write) | Programlar |

**❖ BU KONUDAN NASIL SORU GELİR?**

1. **Kavramsal:** "Explain why an operating system is described as both a resource manager and an abstraction layer." *İpucu:* iki rolü ayrı ayrı tanımla, her birine bir örnek ver.
2. **Senaryo:** "A buggy application crashes. Why does the rest of the system usually keep running?" *İpucu:* soruda "why" ve "rest of the system" geçiyorsa cevap kullanıcı/çekirdek kipi ayrımıdır.
3. **Sıralama/akış:** "Describe the steps of a system call." *İpucu:* istek → kip geçişi → kontrol → hizmet → sonuç.
4. **Çoktan seçmeli sınıflandırma:** "Which of the following is a device driver?" *İpucu:* "belirli bir donanım" ifadesini ara.

**✗ SIK YAPILAN HATA**

- **"İşletim sistemi = masaüstü/pencereler."** → Doğrusu: Gördüğün arayüz işletim sisteminin görünen yüzüdür; çekirdeğin kendisi değildir (slayt 33 bunu açıkça söyler). → Neden yanlış: Arayüzü olmayan sunucu işletim sistemleri de vardır.
- **"Sistem çağrısı ile sistem programı aynı şey."** → Doğrusu: Biri bir *istek* (fonksiyon benzeri bir giriş noktası), diğeri bir *program*. → Neden yanlış: Derleyici bir sistem programıdır ama dosya okumak için kendisi de sistem çağrısı yapar.
- **"Kullanıcı kipindeki bir hata hiçbir zaman sistemi etkilemez."** → Doğrusu: Slayt "usually" (genellikle) diyor. → Neden yanlış: Mutlak ifadeler ("hiçbir zaman", "her zaman") çoktan seçmelide çoğu zaman tuzaktır.

**⚡ HIZLI TEKRAR**

- İşletim sistemi = **kaynak yöneticisi** (CPU, bellek, depolama, ağ, I/O) + **soyutlama katmanı**.
- Program türleri: **uygulama** (kullanıcı için), **sistem programı** (sistem için), **sürücü** (belirli donanım için).
- Katmanlar: kullanıcı → uygulama → işletim sistemi → donanım.
- Tasarım = **ödünleşim**: maliyet, performans, güvenlik, işlevsellik, karmaşıklık, adalet.
- **Kullanıcı kipi** sınırlı, **çekirdek kipi** tam yetkili; ayrıcalık ayrımı güvenliğin temeli.
- **Sistem çağrısı:** istek → çekirdeğe geçiş → kontrol → hizmet → sonuç.

*Ezberle:* tanımdaki iki anahtar ifade ve üç program türü. *Türetebil:* sistem çağrısı adımları (mantıkla kurulur).
**Bu bölümden sonra şunları yapabiliyor olmalısın:** bir programı doğru türe koymak; işletim sistemini iki rolüyle tanımlamak; bir hatanın neden tek süreci ya da tüm sistemi etkilediğini açıklamak; sistem çağrısının adımlarını sırayla yazmak.

## 1.9 Bölüm cevapları

**Durup düşün (soyutlama).** (1) Geliştirici her donanım modeli için ayrı kod yazmaz, kodu taşınabilir olur; ayrıca dosya, bellek gibi kavramları güvenli ve tutarlı biçimde kullanır. (2) Evet: aradaki her katman çalışmak için zaman harcar; doğrudan donanıma erişmek kuramsal olarak daha hızlı olabilir. Bu da yine bir ödünleşimdir (kolaylık ve güvenlik ↔ ham hız).

**Yarı çözümlü örnek (kayıt düğmesi).** 2. **çekirdek** kipine; 3. editörün o dosyaya **yazma izni** olup

olmadığını (ve verdiği bellek adresinin geçerli olduğunu); 4. **aygıt sürücüsü** aracılığıyla.

**Kendin çöz (oyunlar çekirdek kipinde mi?).** Örnek cevap: Bu öneri performans uğruna güvenliği feda eden kötü bir ödünleşimdir. Çekirdek kipinde çalışan bir programın hatası bütün sistemi çökertebilir; kötü niyetli bir oyun bütün dosyalara ve diğer programların belleğine erişebilir. Ayrıcalık ayrımı korumanın temeli olduğu için uygulamalar kullanıcı kipinde kalır ve donanıma sistem çağrılarıyla erişir; kazanılacak küçük hız farkı bu riske değmez.

BÖLÜM 2

# İşletim sistemi aileleri ve arayüzler

◆ **BU BÖLÜMDE** Okuma süresi: 30 dk Zorluk: ●○○

- İşletim sistemlerini kullanım bağlamına göre (masaüstü, sunucu, mobil, gömülü) sınıflandırmak ve her birinin öncelikli ölçütünü söylemek
- Gerçek zamanlı işletim sisteminin neden "çok hızlı" değil "zamanında" demek olduğunu açıklamak
- Windows, macOS/iOS ve Linux/Android ailelerini ve bir Linux dağıtımının ne olduğunu tanımlamak
- GUI ile CLI'yi karşılaştırmak ve bir ürün için doğru işletim sistemi ölçütünü gerekçelendirmek

**SINAV ÖNCELİĞİ: ORTA** ❖ Kaynak: D1 s. 13–21

*Neden orta?* Kavramlar basit ve hesap gerektirmiyor, ama "hangi sistem için hangi ölçüt kritik?" sorusu senaryo sorusu olarak çok kolay sorulur; gerçek zamanlı sistem tanımı ise klasik bir tuzaktır.

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Önceki bölümde her işletim sisteminin iki işi olduğunu gördük: kaynakları yönetmek ve donanımı soyutlamak. Ama bir akıllı saatin kaynakları ile bir veri merkezindeki sunucunun kaynakları çok farklı; birinde pil, diğerinde binlerce eşzamanlı kullanıcı kritik. Aynı iki görevi yaparken *neyi önemsedikleri* farklı olduğu için işletim sistemleri de ailelere ayrılır. Bu bölümde bu farkı ve en önemli tuzağı (gerçek zamanlı = hızlı mı?) çözeceğiz.

## 2.1 Her cihaz aynı şeyi mi önemser?

❖ Kaynak: D1 s. 14–16

Slayt 14 işletim sistemlerini kullanım bağlamına göre dörde ayırıyor. Her birinin yanında "neyin önemli olduğu" yazıyor; sınav için asıl ezberlemen gereken de bu öncelikler.




**Şekil 2.1:** Dört kullanım bağlamı ve her birinin öncelikleri (D1 s. 14–16).

Bu dört kutuyu ezberlemek yerine "cihazın hayatı nasıl geçiyor?" diye düşünürsen kendin türetebilirsin:

- **Masaüstü (desktop):** Önünde bir kişi oturuyor ve tıkladığında hemen tepki bekliyor. Binlerce farklı programı (oyun, ofis, IDE) çalıştırabilmeli. → *Bir kullanıcının etkileşimi*, *grafik arayüz (GUI)* ve *geniş uygulama desteği* önemli (slayt 15).
- **Sunucu (server):** Önünde kimse oturmuyor; ağ üzerinden aynı anda yüzlerce, binlerce istemciye (client) hizmet veriyor. Bir dakika çökmesi binlerce kişiyi etkiler. → *Erişilebilirlik (availability)*, *güvenlik* ve *uzaktan yönetim* önemli (slayt 15).
- **Mobil (mobile):** Genel amaçlı; kullanıcı istediği uygulamayı yükleyebiliyor. Ama cebinde, pille çalışıyor ve içinde konum, kamera gibi hassas veriler var. → *Pil ömrü* ve *gizlilik* kritik tasarım ölçütleri (slayt 16). "Uygulama yalıtımı" (app isolation) da bu yüzden önemli: bir oyun, banka uygulamanın verisini okuyamamalı.
- **Gömülü (embedded):** Bir çamaşır makinesinin, bir arabanın fren sisteminin ya da bir modemin içindeki bilgisayar. Genellikle *tek bir ürün veya işlev* için tasarlanır; kullanıcı ona yeni uygulama yüklemez. → *Sıkı bellek, enerji ve gerçek zaman kısıtları* olabilir (slayt 16).

**✎ DURUP DÜŞÜN**

Mobil ve gömülü sistemlerin ikisi de küçük cihazlarda ve pille çalışabiliyor. Slayt 16'ya göre aralarındaki en temel fark ne? (İpucu: kullanıcı uygulama yükleyebiliyor mu?)

## 2.2 Gerçek zamanlı = çok hızlı mı?

❖ Kaynak: D1 s. 17

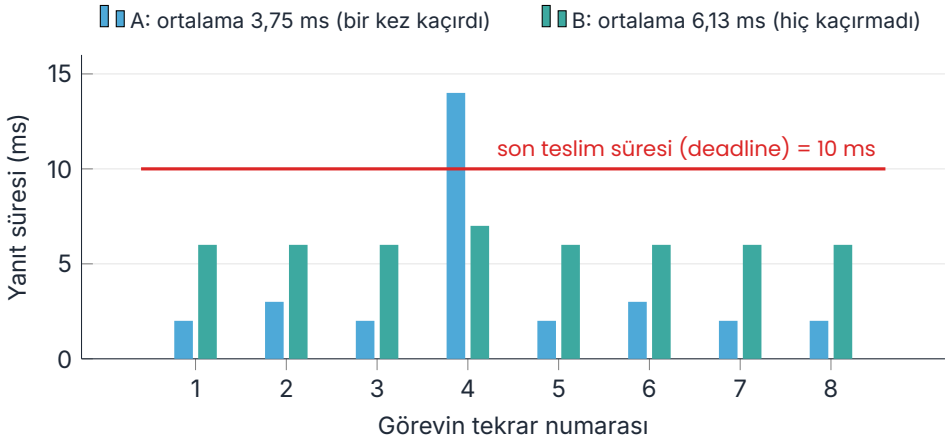
Bu bölümün en önemli tuzağı burada. Gündelik dilde "gerçek zamanlı" deyince "anında, çok hızlı" anlarız. İşletim sistemlerinde öyle değil.

**∑ TANIM / FORMÜL — Gerçek zamanlı işletim sistemi (real-time operating system, RTOS)**

Doğruluğun yalnızca sonuca değil, sonucun **zamanında** (belirlenen son teslim süresi, **deadline**, dolmadan) üretilmesine de bağlı olduğu sistem.
*Slayttaki ifade:* "Correctness depends not only on the result but also on producing it on time." ve "Real-time does not simply mean 'very fast'; it means **meeting a deadline reliably**."
**Anahtar kelimeler:** deadline (son teslim süresi), öngörülebilir gecikme (predictable latency), güvenilir biçimde.

Bir arabanın hava yastığını düşün. Sensör çarpmayı algıladıktan sonra hava yastığı belirli bir süre içinde açılmazsa, iki saniye sonra açılması "doğru sonuç" sayılmaz; işe yaramaz, hatta tehlikelidir. Burada önemli olan ortalamada hızlı olmak değil, **her seferinde** süreyi tutturmaktır. Slayt bunu şöyle söylüyor: *öngörülebilir gecikme (predictable latency), ortalama hızdan daha önemli olabilir.*

Aşağıdaki şekil bu fikri sayılarla gösteriyor. A sistemi ortalamada daha hızlı ama bir kez son teslim süresini kaçırıyor; B sistemi biraz daha yavaş ama hiç kaçırmıyor. Gerçek zamanlı bir iş için doğru seçim **B**'dir.




**Şekil 2.2:** Uydurma sayılarla bir düşünce deneyi: ortalamada hızlı olan A, 4. tekrarda deadline'ı kaçırıyor. Gerçek zamanlı sistem için B daha doğrudur, çünkü gecikmesi öngörülebilir.

Slaytın verdiği örnek alanlar: **tıbbi cihazlar, otomotiv sistemleri, robotlar, endüstriyel kontrol**. Hepsinde ortak nokta, katı zaman kısıtları olan *kontrol* görevleri olmaları.

**+ EK BİLGİ (KAYNAK DIŞI) — Katı ve esnek gerçek zaman**

Ders kitaplarında gerçek zamanlı sistemler genellikle ikiye ayrılır. **Katı (hard) gerçek zaman:** deadline kaçarsa sonuç tamamen başarısızdır (hava yastığı, kalp pili). **Esnek (soft) gerçek zaman:** deadline kaçarsa kalite düşer ama sistem çalışmaya devam eder (görüntülü görüşmede bir karenin geç gelmesi). Slayt bu ayrımı yapmıyor; sınavda sorulursa slaytın genel tanımını kullanman yeterli.

## 2.3 Yaygın işletim sistemi aileleri

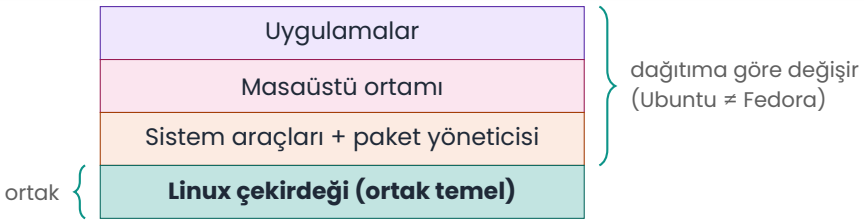
❖ Kaynak: D1 s. 18–19

| Aile | Slayta göre nerede kullanılır? | Bilmen gereken |
|---|---|---|
| **Windows** | Masaüstü, kurumsal istemci (şirket bilgisayarları) ve sunucu ekosistemleri | Masaüstünde çok yaygın |
| **macOS / iOS** | Apple donanımıyla bütünleşik masaüstü ve mobil platformlar | Donanım ve yazılım aynı şirketten |
| **Linux / Android** | Sunucular, gömülü sistemler, masaüstleri ve mobil cihazlar | Açık kaynaklı (**open-source**) bir *çekirdek ailesi*; Android, Linux çekirdeği üzerine kuruludur |

Linux satırındaki "çekirdek ailesi" ifadesine dikkat: Linux aslında bir *çekirdek*tir (1. bölümdeki kernel). İnsanların "Linux kullanıyorum" derken kastettiği şey ise genellikle bir **dağıtım**dır.

**∑ TANIM / FORMÜL — Linux dağıtımı (Linux distribution)**

Linux çekirdeğinin; sistem araçları, bir **paket yöneticisi** (program kurup güncelleyen araç), bir masaüstü ortamı ve uygulamalarla bir araya getirilip tek bir paket hâline getirilmiş hâli. Örnekler (slayt 19): **Ubuntu, Debian, Fedora, Linux Mint, openSUSE**. Bu dağıtımlar aynı çekirdek temelini paylaşabilir ama varsayılan araçları ve destek modelleri farklıdır.




**Şekil 2.3:** Bir Linux dağıtımı çekirdeğin üstüne eklenen katmanlardan oluşur (D1 s. 19).

**★ MERAKLISINA (SINAV DIŞI)**

Android telefonun da altta bir Linux çekirdeği çalıştırır; ama üstündeki katmanlar (uygulama çatısı, arayüz) bir masaüstü Linux dağıtımından çok farklıdır. Yani "Linux" kelimesi akıllı telefonlardan dünyadaki sunucuların büyük kısmına kadar çok farklı cihazlarda karşına çıkar. Bu, çekirdek ile üstündeki katmanları ayrı düşünmenin ne kadar işe yaradığını gösteren güzel bir örnek.

## 2.4 Grafik arayüz mü, komut satırı mı?

❖ Kaynak: D1 s. 20

| | GUI (Graphical User Interface, grafik kullanıcı arayüzü) | CLI (Command-Line Interface, komut satırı arayüzü) |
|---|---|---|
| Nasıl kullanılır? | Pencere, simge, menü, fare/dokunma | Yazılan komutlar |
| Güçlü yanı | Görsel keşif ve doğrudan etkileşim | Kesin ve **tekrarlanabilir** işlemler |
| Kimin için kolay? | Yeni kullanıcılar için öğrenmesi genellikle daha kolay | Otomasyon, uzaktan yönetim ve yazılım geliştirme için güçlü |

Slaytın alt notu önemli: *Modern işletim sistemleri genellikle her iki arayüzü de sunar.* Yani "Windows GUI'dir, Linux CLI'dir" gibi bir ayrım yanlıştır; Windows'ta da PowerShell, Linux'ta da masaüstü ortamları vardır. CLI'nin neden hâlâ önemli olduğunu 4. bölümde (slayt 34) tekrar göreceğiz.

## 2.5 Bir ürün için işletim sistemi seçmek

❖ Kaynak: D1 s. 21

Slayt 21 bir sınıf etkinliği: dört ürün için hangi gereksinimin en kritik olduğunu *son teslim süresi, enerji, uyumluluk, güvenlik ve ölçeklenebilirlik* kavramlarıyla gerekçelendirmen isteniyor. Slaytta cevap yok; aşağıdaki çözüm bu bölümdeki bilgilere dayanan önerimdir.

**▶ ÖRNEK 2.1 — Slayt örneği [D1 s. 21]: doğru sistemi eşleştir**

**Soru.** Yoğun bakım (ICU) monitörü, oyun bilgisayarı, web sunucusu ve akıllı saat için en kritik gereksinim hangisidir?
**Yöntem.** Her ürün için "Bu cihaz başarısız olursa en kötü ne olur?" ve "cihazın en kıt kaynağı ne?" sorularını soruyoruz.

- **ICU monitörü → son teslim süresi (deadline).** Hastanın kalp ritmi bozulduğunda alarm *zamanında* çalmalı; geç gelen alarm yanlış sonuçtur. Uygun tür: gerçek zamanlı / gömülü.
- **Oyun bilgisayarı → uyumluluk (ve performans).** Binlerce oyunu ve donanımı (ekran kartı, oyun kolu) desteklemeli. Uygun tür: masaüstü.
- **Web sunucusu → ölçeklenebilirlik ve güvenlik.** Aynı anda çok sayıda istemciye hizmet verir; internete açık olduğu için saldırı hedefidir. Uygun tür: sunucu.
- **Akıllı saat → enerji.** Pili küçük, günlerce dayanmalı. Uygun tür: mobil / gömülü.

**Sık hata:** Oyun bilgisayarı için "gerçek zaman" demek. Oyunda kare hızı önemlidir ama bir karenin geç gelmesi kimsenin hayatını tehlikeye atmaz; bu tür sistemler genel amaçlı masaüstü işletim sistemi kullanır.

**✎ KENDİN ÇÖZ**

Bir akıllı ev termostatı için işletim sistemi seçiyorsun: tek işi var, sıcaklığı ölçüp kombiyi açıp kapatmak; duvara takılı, yıllarca yeniden başlatılmadan çalışmalı. Slayt 14'teki dört bağlamdan hangisine girer? Hangi iki ölçüt en kritik? Gerekçelendir.

## 2.6 Bu bölümün bağlantıları

- Gerçek zamanlı sistemlerin "deadline" ölçütü, 4. bölümdeki işletim sistemi türleri karşılaştırma tablosunda (slayt 45) ve 6. bölümdeki "tek en iyi algoritma yoktur" fikrinde (slayt 70) tekrar karşına çıkacak.
- Sunucunun "çok kullanıcılı" yapısı → 4. bölümdeki **çok kullanıcılı sistem** (slayt 43).
- Mobilde "uygulama yalıtımı" → 1. bölümdeki kullanıcı/çekirdek kipi ve 4. bölümdeki koruma görevi.

**⚠ SINAVDA DİKKAT**

- **Gerçek zamanlı ≠ çok hızlı.** Gerçek zamanlı = son teslim süresini *güvenilir biçimde* tutturmak. Öngörülebilir gecikme ortalama hızdan önemli olabilir.
- Her bağlamın anahtar kelimesi: masaüstü → **etkileşim/uyumluluk**; sunucu → **erişilebilirlik/çok kullanıcı**; mobil → **pil/gizlilik**; gömülü → **tek işlev/sınırlı kaynak/güvenilirlik**.
- **Mobil genel amaçlıdır** (uygulama yüklenir); **gömülü genellikle tek işlevlidir.**
- Linux bir **çekirdek**; Ubuntu, Fedora vb. **dağıtım**.

**❖ BU KONUDAN NASIL SORU GELİR?**

1. **Kavramsal (tuzaklı):** "Real-time operating systems are mainly characterized by: (a) the highest average speed (b) meeting deadlines reliably ..." *İpucu:* "very fast", "highest speed" geçen şık tuzaktır.

2. **Senaryo:** "You are designing the OS for a pacemaker / a web server / a smartwatch. Which requirement is most critical? Justify." *İpucu:* cihaz başarısız olunca en kötü sonucu düşün; slayt 21'deki beş kelimeden birini seç.
3. **Tanım:** "What is a Linux distribution?" *İpucu:* çekirdek + araçlar + paket yöneticisi + masaüstü + uygulamalar.
4. **Karşılaştırma:** "Compare GUI and CLI." *İpucu:* her biri için bir güçlü yan ve bir kullanım alanı yaz.

**✗ SIK YAPILAN HATA**

- **"Gerçek zamanlı sistem en hızlı sistemdir."** → Doğrusu: Zamanında sonuç üreten, gecikmesi öngörülebilir sistemdir. → Neden yanlış: Ortalaması hızlı ama arada bir çok geciken sistem deadline'ı kaçırır.
- **"Linux bir işletim sistemi markasıdır, Ubuntu onun bir sürümüdür."** → Doğrusu: Linux bir çekirdektir; Ubuntu, çekirdeğin üstüne araçlar ve uygulamalar eklenmiş bir dağıtımdır. → Neden yanlış: Aynı çekirdeği kullanan farklı dağıtımlar farklı hedeflere sahiptir.
- **"Mobil işletim sistemi = gömülü işletim sistemi."** → Doğrusu: Mobil genel amaçlıdır ve kullanıcı uygulama yükler; gömülü genellikle tek ürün/işlev içindir.

**⚡ HIZLI TEKRAR**

- Dört bağlam: **masaüstü, sunucu, mobil, gömülü**; her birinin önceliği farklı.
- **RTOS:** doğru sonuç + doğru zaman; *deadline'ı güvenilir biçimde karşılamak*; örnekler: tıbbi, otomotiv, robot, endüstriyel kontrol.
- Aileler: **Windows**, **macOS/iOS**, **Linux/Android** (açık kaynak çekirdek).
- **Dağıtım** = Linux çekirdeği + araçlar + paket yöneticisi + masaüstü + uygulamalar.
- **GUI** öğrenmesi kolay, görsel; **CLI** tekrarlanabilir, otomasyona uygun. Modern sistemler ikisini de sunar.

*Ezberle:* RTOS tanımı ve slayt 14'teki dört bağlamın anahtar kelimeleri. *Türetebil:* bir ürün için kritik ölçüt ("başarısız olursa ne olur?" sorusuyla).
**Bu bölümden sonra şunları yapabiliyor olmalısın:** verilen bir ürünü doğru bağlama koymak ve kritik ölçütünü gerekçelendirmek; RTOS'u doğru tanımlamak; çekirdek ile dağıtımı ayırmak.

## 2.7 Bölüm cevapları

**Durup düşün (mobil–gömülü).** Mobil sistemler genel amaçlıdır ve kullanıcı uygulama yükleyebilir; gömülü sistemler genellikle tek bir ürün ya da işlev için tasarlanır ve kullanıcı ona uygulama yüklemez.

**Kendin çöz (termostat).** Tek bir işlevi var ve bir ürünün içine yerleşik → **gömülü** sistem. En kritik iki ölçüt: **güvenilirlik** (yıllarca yeniden başlatılmadan çalışmalı) ve **sınırlı kaynaklarla çalışabilmek** (ucuz, düşük bellekli donanım; pilliyse enerji). Çok sıkı bir deadline yoktur (sıcaklık birkaç saniye geç ölçülse sorun olmaz), bu yüzden katı gerçek zaman gerekmez; ama kombiyi zamanında kapatmak gibi basit zaman kısıtları olabilir.

BÖLÜM 3

# Tarihçe ve bilgisayarın kaynakları

◆ **BU BÖLÜMDE** Okuma süresi: 35 dk Zorluk: ●●○

- İşletim sistemlerinin dört dönemini ve her dönemi tetikleyen problemi sıralamak
- Toplu işlem (batch) sistemlerinin nasıl çalıştığını ve zayıf yanını açıklamak
- Çoklu programlamanın (multiprogramming) neden ortaya çıktığını ve hangi yeni ihtiyaçları doğurduğunu anlatmak
- Çoklu programlama ile zaman paylaşımını (time sharing) hedefleri üzerinden ayırt etmek
- Bir işin CPU'ya mı, belleğe mi, diske mi, ağa mı bağlı olduğunu (bound) belirlemek
- İşlemci çekirdeği (core), iş parçacığı (thread) ve süreç (process) kavramlarını ayırmak

**SINAV ÖNCELİĞİ: ORTA** ❖ Kaynak: D1 s. 22–30

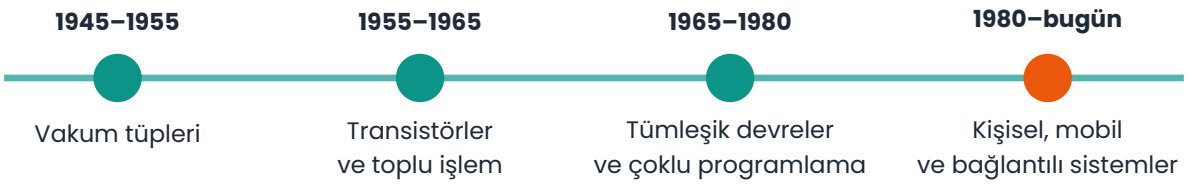
*Neden orta?* Tarih tarihleriyle sorulmaktan çok "neden" sorusuyla sorulur: *çoklu programlama neden doğdu?* Bu soru ve "bound" sınıflandırması sınavda yüksek olasılıkla karşına çıkar; yılları ezberlemen gerekmez ama sırasını bil.

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Şimdiye kadar işletim sisteminin bugün ne yaptığını konuştuk. Ama ilk bilgisayarlarda işletim sistemi neredeyse yoktu. Her yeni özellik (çoklu programlama, zaman paylaşımı, bellek koruması) bir önceki dönemin canını yakan bir problemi çözmek için doğdu. Tarihi bu gözle okursan, 5–7. bölümdeki bellek yönetimi ve zamanlamanın *neden* var olduğu kendiliğinden ortaya çıkar. Bölümün sonunda da bu yönetimin konusu olan kaynaklara (CPU, bellek, I/O) ve süreç kavramına geçiş yapacağız.

## 3.1 Dört dönemde işletim sistemleri

❖ Kaynak: D1 s. 23




**Şekil 3.1:** İşletim sistemlerinin evrimi (D1 s. 23). Her dönemi bir donanım teknolojisi ve bir işletim fikri temsil ediyor.

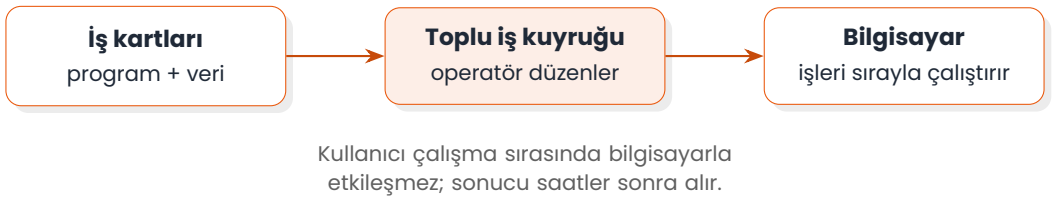
Slayt her dönem için yalnızca bir donanım ve bir fikir yazıyor. Aradaki mantığı birleştirelim:

1. **1945–1955, vakum tüpleri:** Bilgisayarlar oda büyüklüğünde, çok pahalı ve elle programlanıyor. Bugünkü anlamda bir işletim sistemi yok.
2. **1955–1965, transistörler ve toplu işlem (batch):** Bilgisayarlar daha güvenilir hâle geliyor ama hâlâ çok pahalı. Pahalı makinenin boş beklememesi için işler gruplar hâlinde sırayla çalıştırılıyor.
3. **1965–1980, tümleşik devreler (IC) ve çoklu programlama:** Bellek artıyor; aynı anda birden fazla iş bellekte tutulabiliyor. CPU, bir iş beklerken diğerini çalıştırıyor.
4. **1980–bugün, kişisel, mobil ve bağlantılı sistemler:** Herkesin kendi bilgisayarı ve telefonu var; etkileşim, ağ ve güvenlik ön plana çıkıyor.

Bu sırayı akılda tutmanın kolay yolu: **pahalı makineyi boş bırakmama** isteği toplu işlemi, sonra çoklu programlamayı doğurdu; **insanların beklemek istememesi** ise zaman paylaşımını ve etkileşimli sistemleri doğurdu.

## 3.2 Toplu işlem sistemleri: işleri grup grup çalıştırmak

❖ Kaynak: D1 s. 24




**Şekil 3.2:** Toplu işlem (batch) akışı (D1 s. 24).

Toplu işlem sisteminde programcı programını ve verisini delikli kartlara yazar, kart destesini bilgisayar operatörüne teslim eder. Operatör benzer işleri bir araya toplayıp (batch) bilgisayara sırayla yükler. Sonuçlar yazıcıdan çıkar ve programcıya saatler, hatta bir gün sonra ulaşır.

Bunu kuru temizlemeciye çamaşır bırakmaya benzetebilirsin: kıyafeti bırakırsın, ertesi gün alırsın; arada temizlemecinin ne yaptığına karışamazsın. Programında küçük bir yazım hatası varsa bunu ancak ertesi gün çıktıyı görünce öğrenirsin.

Slaytın vurgusu: *Kullanıcı çalışma sırasında bilgisayarla etkileşmez; tamamlanma süresi (**turnaround time**) uzundur.* Turnaround'u 6. bölümde kesin tanımlayacağız: bir işin sisteme girişinden bitişine kadar geçen toplam süre.

## 3.3 Çoklu programlama neden ortaya çıktı?

❖ Kaynak: D1 s. 25

Şimdi bu bölümün en önemli fikri geliyor. Bir program çalışırken iki tür şey yapar: **hesap** (CPU'yu kullanır) ve **giriş/çıkış, yani I/O** (diskten okuma, yazıcıya yazma gibi). I/O, CPU'ya göre son derece yavaştır. Bellekte tek bir program varsa, o program diskten veri beklerken CPU'nun yapacak hiçbir işi kalmaz; **boşta (idle) bekler.**

**▶ ÖRNEK 3.1 — Ek örnek: tek programla CPU ne kadar boşta kalır?**

**Soru.** Bir iş sürekli şu döngüyü tekrarlıyor: 2 ms CPU'da hesap, ardından 8 ms disk I/O'su bekleme. Bellekte yalnızca bu iş varsa CPU zamanın yüzde kaçında çalışır?
**Verilenler:** CPU süresi = 2 ms, I/O süresi = 8 ms (her döngüde). **İstenen:** CPU kullanımı (**CPU utilization**) = CPU'nun meşgul olduğu sürenin toplam süreye oranı.
**Çözüm.** Bir döngü 2 + 8 = 10 ms sürer. Bunun yalnızca 2 ms'sinde CPU çalışıyor.
CPU kullanımı = 2 / 10 = 0,20 → **%20**. Yani pahalı CPU zamanın %80'inde hiçbir şey yapmıyor!
**Aynı mantık:** 4. bölümdeki slayt 46 örneği (6 ms CPU + 10 ms I/O) birebir bu yöntemle çözülür.

Çözüm fikri basit: **bellekte birden fazla iş tutalım.** Bir iş I/O beklerken CPU hazırda bekleyen başka bir işe geçsin.




**Şekil 3.3:** Ek örnek: 2 ms CPU + 8 ms I/O yapan işlerden bellekte beş tane (A–E) varsa, biri I/O beklerken diğeri CPU'yu kullanır ve boşluklar dolar (idealleştirilmiş çizim; I/O aygıtının yetişebildiği varsayıldı).

Slayt 25 bunu dört maddede özetliyor ve sonuncusu çok önemli:

1. Tek program I/O beklerken CPU boşta kalıyordu.
2. Bellekte birden fazla iş tutulursa CPU hazır olan başka bir işe geçebilir.
3. Amaç **CPU kullanımını (utilization)** ve **sistem üretilen işini (throughput)** artırmaktır. Throughput: birim zamanda tamamlanan iş sayısı.
4. **Bu yaklaşım bellek korumasına ve zamanlamaya ihtiyaç doğurur.**

Neden dördüncü madde? Bellekte birden fazla iş varsa iki yeni soru çıkar: *(a) Bir iş başkasının bellek bölgesini bozmasın diye ne yapacağız?* → **bellek koruması** (5. bölüm). *(b) CPU boşaldığında bekleyen birkaç işten hangisini seçeceğiz?* → **zamanlama** (6–7. bölüm). Yani bu haftanın geri kalanı aslında bu tek maddenin açılımı.

**✎ DURUP DÜŞÜN**

Çoklu programlama tek bir işin daha hızlı bitmesini sağlar mı? Yoksa başka bir şeyi mi iyileştirir? (İpucu: slayt 46'daki "Elapsed time may not be identical" cümlesini hatırla.)

## 3.4 Çoklu programlama ile zaman paylaşımı farkı

❖ Kaynak: D1 s. 26

İkisi de "CPU birden fazla iş arasında geçiş yapıyor" gibi görünür; fark **neyi hedeflediklerin**dedir.

| | Çoklu programlama (multiprogramming) | Zaman paylaşımı (time sharing) |
|---|---|---|
| Hedef | CPU'yu **meşgul tutmak** | Birçok kullanıcıya **hızlı tepki veren etkileşim** sağlamak |

| | Çoklu programlama (multiprogramming) | Zaman paylaşımı (time sharing) |
|---|---|---|
| Tipik iş | Uzun süren toplu (batch) işler | Terminal başında oturan kullanıcıların komutları |
| Ne zaman geçiş yapılır? | Çalışan iş I/O beklemeye başlayınca | **Kısa zaman dilimleri** dolunca, sık sık |
| Ölçütü | CPU kullanımı, throughput | Yanıt süresi (**response time**) |



Benzetme: Çoklu programlama, bir aşçının fırına kek koyduktan sonra pişmesini beklerken salata hazırlamasıdır; amaç aşçıyı boş bırakmamak. Zaman paylaşımı ise bir öğretmenin sınıfta her öğrencinin sorusuna sırayla 30 saniye ayırmasıdır; amaç kimsenin uzun süre cevapsız kalmaması.

Zaman paylaşımındaki "kısa zaman dilimi" fikri 7. bölümdeki **Round Robin** algoritmasının ta kendisidir.

## 3.5 Modern bir işletim sisteminden beklentiler

❖ Kaynak: D1 s. 27

Slayt 27 bugünkü işletim sistemlerinin karşılaması gereken beş beklenti sayıyor. Her birinin yanına neden önemli olduğunu ekledim:

- **Eşzamanlı uygulamalar ve çok çekirdekli işlemciler:** Bugünkü işlemcilerde birden fazla çekirdek var; işletim sistemi işleri bunlara dağıtmalı.
- **Ağ, bulut hizmetleri ve dağıtık kaynaklar:** Dosyaların bir kısmı artık başka bir bilgisayarda (bulutta).
- **Güvenlik, gizlilik ve uygulama yalıtımı:** Her uygulama kendi kutusunda çalışmalı (2. bölümdeki mobil sistemler).
- **Enerji verimliliği ve farklı cihaz biçimleri:** Aynı aile saatte de, sunucuda da çalışabiliyor.
- **Sanal makineler (virtual machines), konteynerler (containers) ve ölçeklenebilirlik:** Tek bir fiziksel makinede birden fazla yalıtılmış sistem çalıştırmak.

**★ MERAKLISINA (SINAV DIŞI)**

Bir **sanal makine**, bir bilgisayarın içinde yazılımla taklit edilen başka bir bilgisayardır; Windows'un içinde Linux çalıştırmak gibi. Bulutta kiraladığın "sunucu" da çoğu zaman büyük bir fiziksel makinenin içindeki sanal makinelerden biridir. Bu, 1. bölümdeki soyutlama fikrinin bir seviye daha yukarı taşınmasıdır: artık donanımın kendisi de soyutlanıyor.

## 3.6 Basit bir bilgisayardaki kaynaklar

❖ Kaynak: D1 s. 28

İşletim sisteminin yönettiği üç temel kaynak:




**Şekil 3.4:** Bir bilgisayarın üç temel kaynağı; işletim sistemi bunların paylaşımını koordine eder (D1 s. 28).

- **CPU (Central Processing Unit, merkezi işlem birimi):** Komutları çalıştırır.

- **Bellek (RAM):** O anda çalışan programların kodunu ve verisini tutar. CPU yalnızca bellekteki veriyle doğrudan çalışabilir; bu yüzden program diskten önce belleğe yüklenir.
- **I/O (Input/Output):** Disk, ağ kartı, klavye gibi dış dünyayla veri alışverişi yapan aygıtlar.

## 3.7 Hangi iş hangi kaynağı zorlar?

❖ Kaynak: D1 s. 29

Bir iş genellikle birden fazla kaynağı kullanır, ama çoğu zaman bir tanesi **darboğaz (bottleneck)** olur: işin ne kadar hızlı biteceğini o belirler. Buna göre işler sınıflandırılır:

- **CPU'ya bağlı (CPU-bound):** Zamanının çoğunu hesap yaparak geçirir. Daha hızlı işlemci → daha hızlı biter.
- **Belleğe bağlı (memory-bound):** Bellekten çok sayıda veri okuyup yazar; bellek erişiminin hızı belirleyicidir.
- **Diske bağlı (disk-bound):** Zamanının çoğunu diske okuma/yazma yaparak geçirir.
- **Ağa bağlı (network-bound):** Ağ bağlantısının hızı belirleyicidir.

**▶ ÖRNEK 3.2 — Slayt örneği [D1 s. 29]: dört işi sınıflandır**

Slayt cevap vermiyor ve "baskın aşamayı tartışın" diyor; aşağıdaki cevaplar tipik durum içindir.

- **Video indirmek → ağa bağlı.** CPU ve disk, internetten gelen veriyi rahatça yetiştirir; hızı belirleyen bağlantı hızıdır.
- **Dosya kopyalamak → diske bağlı.** Hesap neredeyse yok; iş, veriyi diskten okuyup diske yazmaktır.
- **Asal sayı aramak → CPU'ya bağlı.** Çok az veri, çok fazla bölme işlemi.
- **Virüs taraması → genellikle diske bağlı, ama CPU'ya da bağlı olabilir.** Tarama her dosyayı diskten okur (disk) ve içeriğini bilinen zararlı yazılım imzalarıyla karşılaştırır (CPU). Yavaş bir sabit diskte (HDD) baskın aşama okumadır; çok hızlı bir SSD'de ise karşılaştırma işi öne geçebilir.

**Ders:** "Bound" bir etiket değil, bir **baskın aşama** tespitidir; donanım değişince darboğaz da değişebilir.

**✎ KENDİN ÇÖZ**

Bir öğrencinin bilgisayarında iki iş var: (a) 4K bir videoyu farklı bir formata dönüştürmek (yeniden kodlamak), (b) bir oyunun 80 GB'lık güncellemesini indirmek. Her biri hangi kaynağa bağlıdır? Neden?

## 3.8 İşlemci çekirdeği, iş parçacığı ve süreç

❖ Kaynak: D1 s. 30

Bu üç kavram sık karıştırılır; slayt 30 her birini bir cümleyle tanımlıyor.

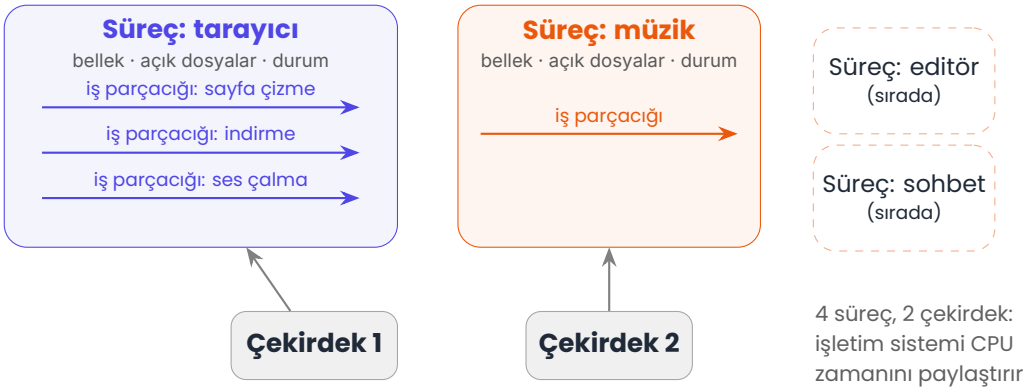
**∑ TANIM / FORMÜL — Çekirdek, iş parçacığı, süreç**

- **İşlemci çekirdeği (core):** Komutları çalıştıran **fiziksel** yürütme birimi. *"A processor core is a physical execution unit that runs instructions."*

- **İş parçacığı (thread):** Bir süreç **içindeki** bir yürütme akışı. *"A flow of execution within a process."*
- **Süreç (process):** Kaynakları ve yürütme durumuyla birlikte **çalışan bir program**. *"A running program together with its resources and execution state."*

Süreç sayısı çekirdek sayısından fazla olabilir; bu durumda işletim sistemi **CPU zamanını süreçler arasında paylaştırır.**

*Not:* Türkçede "çekirdek" kelimesi hem işletim sistemi çekirdeği (**kernel**) hem de işlemci çekirdeği (**core**) için kullanılıyor. Bu kitapta karışmasın diye core için **işlemci çekirdeği** diyeceğim.




Şekil 3.5: Süreçler kaynakları (bellek, dosyalar) sahiplenir; iş parçacıkları bir sürecin içinde akan yürütme yollarıdır; çekirdekler bunları fiziksel olarak çalıştırır.

Bir mutfak benzetmesi: **işlemci çekirdeği** bir aşçıdır. **Süreç** bir yemek siparişidir: tarifi (program kodu), malzemeleri (bellekteki veri) ve "şu an hangi adımdayız?" bilgisi (yürütme durumu) ile birlikte. **İş parçacıkları** aynı siparişin içinde paralel yürüyebilen alt işlerdir: biri sosu karıştırırken diğeri makarnayı haşlar, ikisi aynı malzemeleri paylaşır. Siparişler aşçılardan fazlaysa aşçılar siparişler arasında gidip gelir.

*Benzetmenin bozulduğu yer:* Gerçek bir aşçı bir siparişten diğerine geçerken nerede kaldığını kafasında tutar; işlemci çekirdeği tutamaz. Bu bilginin bir yere kaydedilmesi gerekir. O yer 6. bölümde göreceğimiz **Süreç Denetim Bloğu (PCB)**dur.

## 3.9 Bu bölümün bağlantıları

- "Çoklu programlama bellek koruması ve zamanlama gerektirir" → 5. bölüm (bellek) ve 6–7. bölüm (zamanlama).
- Zaman paylaşımının kısa zaman dilimleri → 4. bölümde **çoklu görev ve kesme (preemption)**, 7. bölümde **Round Robin**.
- CPU'ya bağlı / I/O'ya bağlı işler → 6. bölümdeki **CPU patlaması / I/O patlaması** ve 7. bölümdeki **konvoy etkisi**.
- Süreç kavramı → 6. bölümün ana konusu.

**⚠ SINAVDA DİKKAT**

- Çoklu programlamanın **sebebi:** CPU'nun I/O beklerken boşta kalması. **Amacı:** CPU kullanımı ve throughput. **Sonucu:** bellek koruması ve zamanlama ihtiyacı. Üçünü birlikte yaz, tam puan al.
- **Çoklu programlama → CPU meşgul; zaman paylaşımı → kullanıcıya hızlı tepki.**

Karıştırma.

- CPU kullanımı = CPU'nun meşgul olduğu süre / toplam süre.
- Çekirdek = **fiziksel**; iş parçacığı = süreç **içinde** akış; süreç = **çalışan program + kaynakları + durumu.**

### ❖ BU KONUDAN NASIL SORU GELİR?

1. **Neden sorusu:** "Why did multiprogramming emerge? What new requirements did it create?" *İpucu:* idle CPU → çok iş bellekte → koruma + zamanlama.
2. **Karşılaştırma:** "Distinguish multiprogramming from time sharing." *İpucu:* hedef (CPU meşgul / kullanıcıya hızlı yanıt) ve geçiş anı (I/O bekleme / zaman dilimi).
3. **Hesap:** "A job uses the CPU for x ms and then waits y ms for I/O. What is the CPU utilization with uniprogramming?" *İpucu:* x / (x + y).
4. **Sınıflandırma:** "Classify the following jobs as CPU-, memory-, disk-, or network-bound." *İpucu:* işin zamanının çoğunu nerede geçirdiğini (baskın aşamayı) bul.
5. **Tanım:** "Define core, thread, and process." *İpucu:* fiziksel / süreç içinde akış / kaynaklarıyla çalışan program.

### ✘ SIK YAPILAN HATA

- **"Çoklu programlama işleri daha hızlı bitirir."** → Doğrusu: Tek bir işin süresi kısalmayabilir; CPU kullanımı ve toplam throughput artar. → Neden yanlış: CPU artık başka işlerle paylaşıldığı için tek bir iş daha geç bile bitebilir.
- **"Zaman paylaşımı ile çoklu programlama aynı şeydir."** → Doğrusu: Zaman paylaşımı çoklu programlamanın üzerine kurulur ama hedefi etkileşimdir ve kısa zaman dilimleriyle *düzenli* geçiş yapar. → Neden yanlış: Çoklu programlamada geçiş için işin I/O'ya girmesi beklenir.
- **"Thread ile process aynı şey."** → Doğrusu: Süreç kaynakların sahibidir; iş parçacığı onun içinde çalışan bir akıştır ve kaynakları paylaşır.
- **"Çekirdek sayısı kadar program çalışabilir."** → Doğrusu: Süreç sayısı çekirdekten fazla olabilir; işletim sistemi CPU zamanını paylaştırır.

### ⚡ HIZLI TEKRAR

- Dönemler: vakum tüpü → transistör + **toplu işlem** → IC + **çoklu programlama** → kişisel/mobil/bağlantılı.
- **Toplu işlem:** kullanıcı etkileşmez, turnaround uzun.
- **Çoklu programlama:** I/O beklerken CPU boşta kalmasın diye bellekte çok iş; hedef **utilization + throughput**; doğurduğu ihtiyaç **bellek koruması + zamanlama**.
- **Zaman paylaşımı:** kısa zaman dilimleriyle çok kullanıcıya **hızlı yanıt.**
- Kaynaklar: **CPU, bellek, I/O.** İşler: CPU-, bellek-, disk-, ağa bağlı (baskın aşama).
- **Core** fiziksel birim; **thread** süreç içi akış; **process** kaynaklı çalışan program.

*Ezberle:* üç tanım (core/thread/process). *Türetebil:* CPU kullanımı hesabı, bound sınıflandırması, çoklu programlamanın gerekçesi.

**Bu bölümden sonra şunları yapabiliyor olmalısın:** çoklu programlamanın nedenini ve sonuçlarını açıklamak; tek programlı bir sistemde CPU kullanımını hesaplamak; bir işi baskın kaynağına göre sınıflandırmak.

## 3.10 Bölüm cevapları

**Durup düşün (çoklu programlama hızlandırır mı?).** Tek bir işi genellikle hızlandırmaz; hatta CPU'yu başkalarıyla paylaştığı için o iş biraz daha geç bitebilir. İyileşen şey CPU kullanımı ve birim zamanda biten iş sayısıdır (throughput). Slayt 46 da bunu söyler: geçen süre aynı olmayabilir ama CPU kullanımı artar.

**Kendin çöz (video dönüştürme, güncelleme indirme).** (a) Video yeniden kodlama **CPU'ya bağlıdır**: her kare için yoğun sıkıştırma hesapları yapılır, okunan/yazılan veri miktarına göre hesap çok daha ağırdır. (b) 80 GB indirme **ağa bağlıdır**: hızı internet bağlantısı belirler (bağlantı çok hızlı, disk çok yavaşsa darboğaz diske kayabilir).

BÖLÜM 4

# Temel görevler ve çalışma kipleri

◆ **BU BÖLÜMDE** Okuma süresi: 50 dk Zorluk: ●●○

- İşletim sisteminin beş temel görevini sayıp her birine örnek vermek
- Bir programın çalıştırılmasının baştan sona hangi adımlardan geçtiğini anlatmak
- Kesmenin (interrupt) ne olduğunu ve aygıt sürücüsünün rolünü açıklamak
- Kimlik doğrulama, yetkilendirme, yalıtım ve denetimi senaryolarda tanımak
- Tekli/çoklu programlama, tekli/çoklu görev, çok kullanıcılı ve gerçek zamanlı sistemleri ayırt etmek
- Tek ve çok programlı bir sistemde CPU kullanımını hesaplamak

**SINAV ÖNCELİĞİ: YÜKSEK** ❖ Kaynak: D1 s. 31–47

*Neden yüksek?* Öğrenme çıktılarından biri doğrudan “single-tasking, multitasking, multiprogramming ve real-time sistemleri ayırt etmek”; slayt 45’teki karşılaştırma tablosu ve slayt 46’daki sayısal örnek sınav için birebir malzeme.

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Önceki bölümde çoklu programlamanın neden doğduğunu ve bir bilgisayarın hangi kaynaklardan oluştuğunu gördük. Şimdi işletim sisteminin bu kaynaklarla *somut olarak* hangi işleri yaptığını sıralıyoruz: kullanıcıyla konuşmak, program çalıştırmak, aygıtları yönetmek, dosyaları düzenlemek ve her şeyi korumak. Bölümün ikinci yarısında ise “aynı anda birden fazla şey” yapmanın farklı biçimlerini (çoklu programlama, çoklu görev, çok kullanıcı) birbirinden ayıracağız. Bu ayrım yapılmadan 6–7. bölümdeki zamanlama anlaşılmaz.

## 4.1 İşletim sisteminin beş görevi

❖ Kaynak: D1 s. 32




**Şekil 4.1:** Slayt 32’deki beş temel sorumluluk. Slaytta 3. kutu (kaynak verimliliği) vurgulanmış.

Bu beş görev, 1. bölümdeki iki rolün (kaynak yöneticisi + soyutlama) somut hâlidir. Arayüz ve dosyalar daha çok *soyutlama*ya; program çalıştırma, kaynak verimliliği ve aygıt yönetimi daha çok *kaynak yönetimi*ne; koruma ise ikisine birden hizmet eder.

**⚠ SINAVDA DİKKAT — Slayt numaralarına dikkat**

Slayt 32'deki sıra *1 Arayüz, 2 Program çalıştırma, 3 Kaynak verimliliği, 4 Aygıt yönetimi, 5 Dosyalar ve koruma* şeklinde. Ama ayrıntı slaytlarında numaralar kayıyor: **3 — Manage devices** (s. 37), **4 — Organize files** (s. 38), **5 — Protection and security** (s. 39). "Kaynak verimliliği"nin ayrı bir slaytı yok; içeriği çoklu programlama ve zamanlama slaytlarına dağılmış. Sınavda "beş görevi sayın" denirse slayt 32'deki listeyi yaz. (Bu tutarsızlık "Hocaya sorulacaklar" bölümünde de var.)



## 4.2 Görev 1: Kullanıcıya bir arayüz sunmak

❖ Kaynak: D1 s. 33–34

Slayt 33 dört madde veriyor:

- **Grafik arayüz (GUI)** pencereler, simgeler, menüler ve fare gibi işaretleme aygıtlarıyla etkileşim sağlar.
- **Komut satırı (CLI)** komutların doğrudan çalıştırılmasını ve otomasyonu mümkün kılar.
- **Erişilebilirlik hizmetleri** (ekran okuyucu, büyüteç, sesle komut gibi) farklı kullanıcı ihtiyaçlarını destekler.
- **Arayüz, işletim sistemi hizmetlerinin görünen kısmıdır; çekirdeğin kendisi değildir.** Bu cümle önemli: Windows'un masaüstünü değiştirsen bile altta aynı çekirdek çalışır.

### 4.2.1 Komut satırı hâlâ neden önemli?

Slayt 34 beş örnek komut veriyor ve CLI'nin üç gücünü sayıyor: **tekrarlanabilir, otomasyona uygun, uzaktan yönetimde etkili.** Komutların ne yaptığına bakalım:

Terminal

```
$ pwd               # (print working directory) şu an hangi klasördeyim?
$ ls -la            # bu klasördeki her şeyi (gizliler dahil, -a) ayrıntılı (-l) listele
$ mkdir project     # "project" adında yeni bir klasör oluştur
$ ping example.com  # example.com'a ağ üzerinden ulaşabiliyor muyum, ne kadar sürede?
$ python main.py    # main.py adlı Python programını çalıştır
```

İlk üç komutu boş bir klasörde gerçekten çalıştırdım; çıktı şöyle (klasör adı, tarih ve kullanıcı adı senin bilgisayarında farklı olur):

▶ Çıktı

```
$ pwd
/home/claude/kitap/dogrulama/cli
$ mkdir project
$ ls -la
total 12
drwxr-xr-x 3 root root 4096 Oct 13 17:45 .
drwxr-xr-x 3 root root 4096 Oct 13 17:45 ..
drwxr-xr-x 2 root root 4096 Oct 13 17:45 project
```

✔ Linux kabuğunda çalıştırıldı (ping ve python main.py hariç)

Çıktıdaki `.` bulunduğun klasörü, `..` bir üst klasörü gösterir; satır başındaki `d` bunun bir dizin (directory) olduğunu söyler. Burada önemli olan şey komutların kendisi değil, fikir: aynı beş satırı bir dosyaya yazıp yüz sunucuda aynı anda çalıştırabilirsin. GUI'de yüz sunucuya tek tek bağlanıp tıklaman gerekirdi. "Tekrarlanabilir" ve "otomasyon" tam olarak bu demek.

**+ EK BİLGİ (KAYNAK DIŞI) — Windows'ta aynı komutlar**

Windows PowerShell'de `pwd`, `ls`, `mkdir`, `ping` ve `python main.py` da çalışır (PowerShell bunları kendi komutlarına eşler); yalnızca `ls -la` yerine `ls -Force` (gizli dosyalar dahil) yazman gerekir. Slayttaki komutlar Linux/macOS kabuğuna (shell) göre yazılmış.



## 4.3 Görev 2: Programları çalıştırmak

❖ Kaynak: D1 s. 35–36

Bir uygulamanın simgesine çift tıkladığında işletim sistemi slayt 35'teki dört adımı uygular:



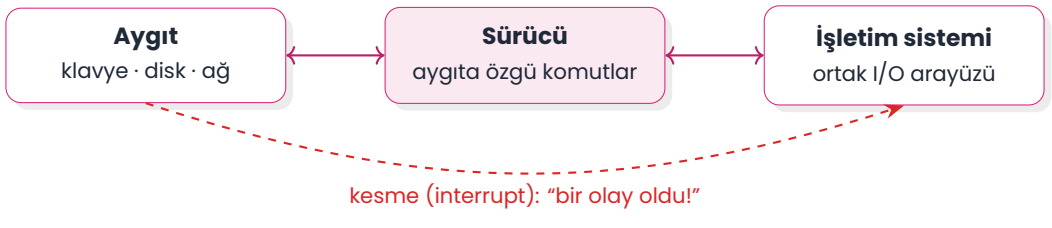

Şekil 4.2: İşletim sisteminin bir programı çalıştırma döngüsü (D1 s. 35).

1. **Programı depolamadan belleğe yükler ve bir süreç oluşturur.** Diskteki program dosyası tek başına hiçbir şey yapmaz (6. bölümde "program ≠ süreç" diyeceğiz).
2. **CPU zamanı, bellek alanı ve gerekli I/O kaynaklarını ayırır.**
3. **Dosya, ağ ve aygıt hizmetlerini sistem çağrılarıyla sağlar** (1. bölümdeki akış).
4. **Program bitince kaynakları geri alır ve çıkış durumunu (exit status) bildirir.** Çıkış durumu, programın başarıyla mı yoksa bir hatayla mı bittiğini söyleyen küçük bir sayıdır (genellikle 0 = başarılı).
5. adım neden önemli? Kaynaklar geri alınmazsa, kapanan her program bir miktar belleği "işgal edilmiş" olarak bırakır ve bilgisayar zamanla belleksiz kalır.

Slayt 36 işletim sistemiyle gelen **sistem yardımcı programlarını (system utilities)** üç gruba ayırıyor: **dosya araçları** (kopyalama, arama, sıkıştırma, yedekleme), **izleme araçları** (CPU, bellek, disk, ağ ve süreçleri gözlemlemek; ör. Görev Yöneticisi) ve **yapılandırma** (kullanıcı, ağ, aygıt ve güvenlik ayarları). Bunlar 1. bölümdeki "sistem programları"nın örnekleridir.

## 4.4 Görev 3: Aygıtları yönetmek ve kesmeler

❖ Kaynak: D1 s. 37



**Şekil 4.3:** Aygıt yönetimi zinciri (D1 s. 37). Kesik ok: aygıtın işlemciye gönderdiği kesme sinyali.

Her aygıtın kendine özgü bir "dili" vardır; bir yazıcı ile bir ağ kartı aynı komutları anlamaz. **Sürücü (driver)** bu aygıta özgü komutları bilir ve işletim sistemine **ortak bir I/O arayüzü** sunar. Böylece işletim sistemi "yaz" dediğinde aygıtın markası önemli olmaz (yine soyutlama).

**∑ TANIM / FORMÜL — Kesme (interrupt)**

Bir aygıtın işlemciye "bir olay gerçekleşti" diye haber veren sinyali. *"An interrupt is a device signal telling the processor that an event has occurred."*
Örnek olaylar: bir tuşa basıldı, diskten istenen veri okundu, ağdan paket geldi.

Kesme olmasaydı ne olurdu? İşletim sistemi her aygıta sürekli "bir şey oldu mu?" diye sormak zorunda kalırdı (buna **yoklama, polling** denir). Bu, kapıya her 10 saniyede bir gidip "gelen var mı?" diye bakmaya benzer; kesme ise **kapı zilidir**: sen işine bakarsın, biri gelince zil seni çağırır. Çoklu programlamanın işleyebilmesi de buna bağlıdır: süreç diskten veri beklerken CPU başka işe geçer; disk işini bitirince bir kesme gönderir ve işletim sistemi bekleyen süreci yeniden "hazır" yapar (6. bölümdeki *Waiting → Ready* geçişi).

## 4.5 Görev 4: Dosya ve klasörleri düzenlemek

❖ Kaynak: D1 s. 38

Disk aslında numaralı bloklardan oluşan dev bir dizidir. İnsanlar ise "Ödevler/CMP103/rapor.pdf" gibi isimlerle düşünür. İşletim sistemi bu ikisi arasında köprü kurar:

- **Dosya adlandırma, dizin (klasör) yapısı ve erişim yollarını (path) tanımlar.**
- **Depolamayı bloklar hâlinde ayırır ve boş alanı takip eder.**
- **Paylaşımı ve erişimi izinlerle (permissions) denetler.** Örneğin bir dosyayı yalnızca sahibi değiştirebilir, diğerleri okuyabilir.
- **Farklı depolama aygıtlarında tutarlı bir dosya arayüzü sunar.** USB bellekteki dosyayı da SSD'deki dosyayı da aynı şekilde açarsın.

Dosya sistemine 5. bölümün sonunda (slayt 58) tekrar döneceğiz.

## 4.6 Görev 5: Koruma ve güvenlik

❖ Kaynak: D1 s. 39

Slayt 39 dört kavramı birer soruyla tanımlıyor; bu soru biçimi ezberlemek için harika:

| Kavram | Cevapladığı soru | Gündelik örnek |
|---|---|---|
| Kimlik doğrulama (**authentication**) | Kullanıcı ya da servis **kim?** | Şifre, parmak izi ile giriş |
| Yetkilendirme (**authorization**) | Hangi kaynak üzerinde hangi işlemlere **izin var?** | Öğrenci notunu görebilir ama değiştiremez |
| Yalıtım (**isolation**) | Bir süreç başkasının belleğini veya dosyalarını **bozamasın** | Oyunun çökmesi tarayıcıyı etkilemez |
| Kayıt ve denetim (**logging and auditing**) | Olaylar **izlenebilir** olsun | "Dün 03.12'de kim giriş yaptı?" sorusunun cevabı |

Slaytın vurgusu: *Güvenlik tek bir özellik değil, bütün kaynak yönetimi kararlarına yayılan bir tasarım ilkesidir.* Yani güvenliği sonradan eklenen bir "antivirüs" gibi düşünme; bellek ayırırken, dosya açarken, CPU dağıtırken her adımda vardır.

**▶ ÖRNEK 4.1 — Ek örnek: hangi güvenlik kavramı?**

**Soru.** Aşağıdaki her durum slayt 39'daki hangi kavrama örnektir? (a) Okulun sistemine öğrenci numarası ve şifrenle girmek. (b) Sisteme girdikten sonra notları görüp değiştirememek. (c) Bir hastane sisteminin, hangi doktorun hangi hasta kaydını ne zaman açtığını kaydetmesi. (d) Bir tarayıcı sekmesindeki zararlı sayfanın, banka sekmesinin belleğini okuyamaması.

**Çözüm.** Her durumda cevaplanan *soruyu* buluyoruz:

- (a) "Kimsin?" → **kimlik doğrulama**.
- (b) "Ne yapmaya iznin var?" → **yetkilendirme**. Kimliğin doğrulandı ama yetkin sınırlı.
- (c) "Ne oldu, kim yaptı?" → **kayıt ve denetim**.
- (d) "Bir süreç diğerini bozabilir mi?" → **yalıtım**.

**Sık hata:** (a) ile (b)'yi karıştırmak. Önce kimlik doğrulanır, *sonra* yetki kontrol edilir; kimliği doğrulanmış biri her şeyi yapabilir demek değildir.

## 4.7 ``Aynı anda birden fazla iş"in dört farklı anlamı

❖ Kaynak: D1 s. 40–45

Bu kısım haftanın en çok karıştırılan yeri. Çoklu programlama, çoklu görev, zaman paylaşımı, çok kullanıcılı... Hepsi "aynı anda birden fazla şey" gibi duruyor. Her birini **hangi soruya cevap verdiği** üzerinden ayıralım.

### 4.7.1 Tekli programlama ve çoklu programlama

❖ Kaynak: D1 s. 40–41

| | Tekli programlama (uniprogramming) | Çoklu programlama (multiprogramming) |
|---|---|---|
| Bellekte kaç iş var? | **Yalnızca bir** | **Birkaç** |
| İş I/O beklerken CPU ne yapar? | **Boşta kalabilir** | **Başka bir işi çalıştırır** |

Slayt 41'in mesajı: *Amaç yalnızca eşzamanlı çalışma değil, bekleme süresini başka işlerle doldurmaktır.* (A işi hesap yapar → A I/O bekler → B o boşluğu kullanır.)

### 4.7.2 Çoklu görev ve kesme (preemption)

❖ Kaynak: D1 s. 42, 44

Çoklu programlamada CPU, çalışan iş *kendisi* I/O beklemeye başladığında el değiştirir. Peki hiç I/O yapmadan saatlerce hesap yapan bir iş varsa? Diğer herkes onu bekler. Çoklu görev bunu çözer:

- **İşletim sistemi CPU'yu çalışan bir süreçten zorla alabilir** (buna **preemption**, kesme denir).
- **Zamanlayıcı (scheduler) hazır süreçlerden birini seçer.**
- **Kısa zaman dilimleri kullanıcıya eşzamanlılık hissi verir.** Tek çekirdekli bir bilgisayarda bile müzik çalarken yazı yazabiliyormuşsun gibi hissedersin; aslında CPU saniyede yüzlerce kez

aralarında gidip geliyordur.
- **Görev değiştirmenin kendisi zaman alır; bu bir ek yüktür (overhead).**

Slayt 44 tekli görev ile çoklu görevi karşılaştırıyor:

| | Tekli görev (single-tasking) | Çoklu görev (multitasking) |
|---|---|---|
| Ne olur? | Bir anda **bir kullanıcı görevi** çalışır | **Birden fazla uygulama ilerler** |
| Bedeli | **Daha basit**, ama kaynak kullanımı sınırlı olabilir | **Zamanlama, koruma ve bağlam değiştirme** gerektirir |

### 4.7.3 Çok kullanıcılı sistem

❖ Kaynak: D1 s. 43

- **Birçok kullanıcı tek bir bilgisayarın kaynaklarını paylaşır.**
- **Her kullanıcı için ayrı oturum, kimlik ve izinler tutulur.**
- **CPU, bellek, depolama ve ağ kaynaklarında adalet (fairness) gerekir.**
- **Sunucular ve yüksek performanslı sistemler tipik örneklerdir.**

Üniversitenin öğrenci bilgi sistemini çalıştıran sunucuyu düşün: binlerce öğrenci aynı anda giriş yapıyor; herkesin kendi oturumu var ve kimse başkasının notunu göremiyor.

### 4.7.4 Hepsi bir arada: karşılaştırma tablosu

❖ Kaynak: D1 s. 45

Slayt 45'teki tablo sınav için altın değerinde; satır satır ezberlemek yerine her satırın mantığını kur:

| Tür | Birincil hedef | Tipik ortam | Kritik ölçüt |
|---|---|---|---|
| Tekli görev (**single-tasking**) | Basitlik | Sınırlı cihaz | Düşük kaynak kullanımı |
| Çoklu görev (**multitasking**) | Etkileşim | Masaüstü / mobil | Yanıt süresi |
| Çok kullanıcılı (**multi-user**) | Paylaşım | Sunucu | Adalet ve güvenlik |
| Gerçek zamanlı (**real-time**) | Öngörülebilirlik | Kontrol sistemi | Son teslim süresi |

Ve bu kavramları ayırmak için kendi özet tablomuz (kaynakta tek tablo olarak yok; slayt 25, 26, 40–45'in birleşimi):

| Kavram | Cevapladığı soru | Geçişi ne tetikler? |
|---|---|---|
| Çoklu programlama | CPU boşta kalmasın diye bellekte birden çok iş tutalım mı? | Çalışan işin I/O beklemeye başlaması |
| Zaman paylaşımı / çoklu görev | Kullanıcı(lar)a hızlı tepki verebilmek için CPU'yu dilimlere bölelim mi? | Zaman diliminin dolması (preemption) |
| Çok kullanıcılı | Birden fazla *insan* aynı makineyi güvenle paylaşabilir mi? | (Bir geçiş türü değil; kimlik, izin ve adalet meselesi) |
| Gerçek zamanlı | İş, son teslim süresinden önce garanti bitecek mi? | Zaman kısıtları ve öncelikler |

**✎ DURUP DÜŞÜN**

Telefonun aynı anda müzik çalıp mesajlaşmana izin veriyor ama telefonu yalnızca sen kullanıyorsun. Bu telefon çoklu görevli midir? Çok kullanıcılı mıdır?

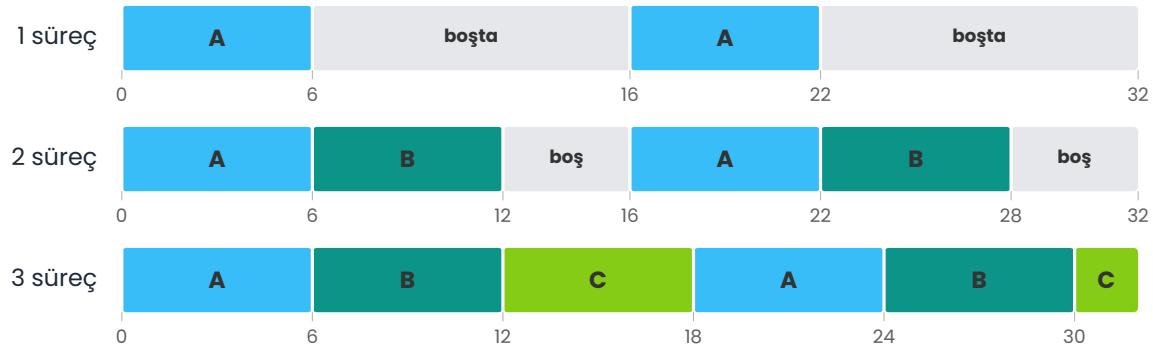
## 4.8 Slayt örneği: I/O beklerken ne olur?

❖ Kaynak: D1 s. 46

**▶ ÖRNEK 4.2 — Slayt örneği [D1 s. 46]: 6 ms CPU, 10 ms I/O**

**Soru (slayt).** Bir süreç CPU'yu 6 ms kullanıyor, sonra disk I/O'su için 10 ms bekliyor. Tekli ve çoklu programlamada ne olur?
**Slaytın üç cevabı.** (1) Tekli programlamada CPU 10 ms boşta kalabilir. (2) Çoklu programlamada bu aralıkta hazır başka bir süreç çalışabilir. (3) Geçen süre aynı olmayabilir ama CPU kullanımı iyileşir.
**Sayılarla tamamlayalım** (bu kısım slaytta yok). Sürecin bu döngüyü sürekli tekrarladığını varsayalım.
*Tekli programlama.* Bir döngü 6 + 10 = 16 ms; CPU'nun meşgul olduğu süre 6 ms. CPU kullanımı = 6 / 16 = 0,375 → **%37,5**.
*Çoklu programlama, aynı davranışta iki süreç (A ve B).* A 0–6 arası çalışır, sonra 6–16 arası I/O bekler. A beklerken B 6–12 arası çalışır. 12–16 arası ikisi de I/O beklediği için CPU yine boştadır. 16'da A geri gelir ve döngü tekrar eder.
Her 16 ms'de CPU 12 ms çalışır: 12 / 16 = **%75**.
*Üç süreç (A, B, C).* 0–6 A, 6–12 B, 12–18 C. A 16'da hazır olur ama CPU 18'e kadar C'dedir, bu yüzden A **2 ms bekler.** CPU artık hiç boş kalmaz: **%100**. Ama A'nın bir döngüsü 16 ms yerine 18 ms sürer.
**Sonuç:** Süreç sayısı arttıkça CPU kullanımı artar; ama tek tek süreçlerin geçen süresi uzayabilir. Slayttaki "elapsed time may not be identical" cümlesinin anlamı tam olarak bu.

✔ Python simülasyonuyla hesaplandı: 37,5 / 75 / 100 yüzde




**Şekil 4.4:** CPU'nun ilk 32 ms'si (zaman ms cinsinden). Gri bölgeler CPU'nun boşta kaldığı süreler. Süreç sayısı arttıkça boşluklar kayboluyor.

**▶ ÖRNEK 4.3 — Yarı çözümlü: farklı oranlar**

Bir süreç 4 ms CPU kullanıp 12 ms I/O bekliyor ve bunu sürekli tekrarlıyor.

1. Tekli programlamada bir döngü 4 + 12 = 16 ms sürer; CPU 4 ms meşguldür. CPU kullanımı = 4/16 = %25.
2. Aynı davranışta iki süreç olursa her 16 ms'de CPU ..... ms meşgul olur, kullanım = ..... .
3. CPU'yu tamamen doldurmak için en az ..... süreç gerekir. (İpucu: 16 ms'lik döngüyü 4'er ms'lik CPU dilimleriyle doldurmak istiyorsun.)

*Cevap bölüm sonunda.*

**✎ KENDİN ÇÖZ**

Bir süreç 5 ms CPU, 15 ms I/O döngüsüyle çalışıyor. (a) Tekli programlamada CPU kullanımı nedir? (b) Bellekte aynı davranışta 3 süreç varsa kullanım nedir? (c) Bu sonuç, CPU'nun "her zaman %100'e yakın" çalışması için ne gerekiğini gösteriyor?



## 4.9 Ara durak: şimdiye kadar

❖ Kaynak: D1 s. 47

Slayt 47 ilk yarıyı dört cümlede özetliyor: (1) İşletim sistemi donanımı soyutlar ve kaynakları yönetir. (2) Sistemin türü kullanıcılara, görev yapısına ve zaman gereksinimlerine göre değişir. (3) **Çoklu programlama CPU'nun boş kalma süresini azaltır; çoklu görev etkileşimi destekler.** (4) **Bu hedefler bellek yönetimi ve süreç zamanlaması gerektirir.** Son cümle, haftanın ikinci yarısının (5–7. bölüm) giriş kapısı.

**★ MERAKLISINA (SINAV DIŞI)**

Günümüz bilgisayarlarında işletim sistemi, CPU'yu süreçler arasında genellikle birkaç milisaniyelik dilimlerle paylaştırır; yani saniyede yüzlerce geçiş olur. İnsan gözü ve kulağı bu kadar kısa kesintileri fark edemediği için müzik kesilmeden çalıyor gibi gelir. "Eşzamanlılık hissi" (sense of concurrency) ifadesindeki "his" kelimesi tam olarak bunu anlatıyor: tek çekirdekte gerçekte aynı anda tek bir iş çalışır.

## 4.10 Bu bölümün bağlantıları

- Program çalıştırmanın 1. adımı (süreç oluşturmak) ve ayrılan kaynaklar → 6. bölümdeki **PCB**.
- Kesmeler → 6. bölümde *Waiting* → *Ready* geçişini tetikleyen olay; ayrıca zaman diliminin dolması da bir zamanlayıcı kesmesiyle anlaşılır.
- Preemption ve ek yük → 6. bölümdeki **kesintili/kesintisiz zamanlama** ve **bağlam değiştirme**; 7. bölümde Round Robin'in **kuantum seçimi**.
- CPU kullanımı hesabı → 6. bölümdeki zamanlama ölçütleri tablosu.

**⚠ SINAVDA DİKKAT**

- Beş görev: **arayüz, program çalıştırma, kaynak verimliliği, aygıt yönetimi, dosyalar ve koruma** (slayt 32 sırası).

- **Arayüz ≠ çekirdek.**
- **Kesme** = aygıtın işlemciye "olay oldu" sinyali.
- **Authentication = kimsin?** / **Authorization = neye iznin var?** Bu ikisi en sık karıştırılan çift.
- **Çoklu programlama:** I/O beklerken başka iş (hedef: CPU kullanımı). **Çoklu görev:** CPU zorla alınabilir, kısa dilimler (hedef: etkileşim/yanıt süresi). **Çok kullanıcılı:** birçok insan, ayrı kimlik/izin (hedef: adalet ve güvenlik). **Gerçek zamanlı:** deadline (hedef: öngörülebilirlik).
- Tek programlı CPU kullanımı = CPU süresi / (CPU süresi + I/O süresi).



### ❖ BU KONUDAN NASIL SORU GELİR?

1. **Tablo tamamlama:** Slayt 45'teki tablonun bir sütunu boş verilir, doldurman istenir. *İpucu:* her türün "neden var?" sorusunun cevabı birincil hedeftir.
2. **Hesap:** "A process uses the CPU for 6 ms, then waits 10 ms for I/O. Compute the CPU utilization with uniprogramming and with two such processes." *İpucu:* 6/16 ve 12/16.
3. **Senaryo eşleştirme (güvenlik):** "Which security concept is illustrated by ...?" *İpucu:* durumun cevapladığı soruyu bul (kim / ne izinli / bozabilir mi / iz bırakıyor mu).
4. **Kavramsal:** "What is preemption and what is its cost?" *İpucu:* CPU'nun zorla alınması + görev değiştirme ek yükü.
5. **Doğru/yanlış tuzağı:** "Multiprogramming always reduces the completion time of every job." → Yanlış.

### ✘ SIK YAPILAN HATA

- **"Çoklu görev, aynı anda gerçekten birden fazla programın çalışmasıdır."** → Doğrusu: Tek çekirdekte bir anda tek süreç çalışır; kısa dilimler *eşzamanlılık hissi* verir. → Neden yanlış: Slayt 42 bilinçli olarak "sense of concurrency" diyor.
- **"Kimliği doğrulanan kullanıcı her şeyi yapabilir."** → Doğrusu: Kimlik doğrulama ile yetkilendirme ayrı adımlardır.
- **"Görev değiştirme bedavadır."** → Doğrusu: Geçişin kendisi zaman alır ve kullanıcı işi yapmaz; bu bir ek yüktür. → Neden önemli: 7. bölümde kuantumun neden çok küçük seçilmemesi gerektiğinin sebebi budur.
- **Hesapta CPU kullanımını 6/10 yazmak.** → Doğrusu: Payda toplam süredir (6 + 10 = 16), yalnızca I/O süresi değil.

### ⚡ HIZLI TEKRAR

- Beş görev ve her birinin özü: arayüz (GUI/CLI/erişilebilirlik), çalıştırma (yükle → kaynak ver → hizmet et → topla), aygıtlar (sürücü + **kesme**), dosyalar (ad, dizin, blok, izin), koruma (**authentication, authorization, isolation, auditing**).
- **CLI:** tekrarlanabilir, otomasyona uygun, uzaktan yönetime elverişli.
- **Tekli ↔ çoklu programlama:** bellekte bir iş ↔ birkaç iş.
- **Çoklu görev:** preemption + zamanlayıcı + kısa dilim; bedeli **ek yük.**
- **Slayt 45 tablosu:** tekli görev–basitlik, çoklu görev–yanıt süresi, çok kullanıcılı–adalet/güvenlik, gerçek zamanlı–deadline.
- CPU kullanımı: 1 süreç %37,5 → 2 süreç %75 → 3 süreç %100 (6+10 örneği).

*Ezberle:* beş görev, güvenlik dörtlüsü, slayt 45 tablosunun "kritik ölçüt" sütunu. *Türetebil:* CPU kullanımı hesapları.
**Bu bölümden sonra şunları yapabiliyor olmalısın:** bir senaryodaki güvenlik kavramını adlandırmak; dört sistem türünü hedefleri ve ölçütleriyle ayırmak; tek ve çok süreçli CPU kullanımını hesaplamak.

## 4.11 Bölüm cevapları

**Durup düşün (telefon).** Evet, **çoklu görevlidir**: birden fazla uygulama ilerliyor, işletim sistemi CPU'yu aralarında paylaştırıyor. Ama çok kullanıcılı sayılmaz: slayt 43'teki anlamda, birçok kullanıcının ayrı kimlik ve oturumlarla aynı makineyi paylaşması söz konusu değil.

**Yarı çözümlü (4 ms + 12 ms).** 2) İki süreçle her 16 ms'de CPU 8 ms meşgul → 8/16 = **%50**. 3) 16 / 4 = **4** süreç gerekir (dört sürecin 4'er ms'lik dilimleri 16 ms'yi tamamen doldurur).

**Kendin çöz (5 ms + 15 ms).** (a) 5 / 20 = **%25**. (b) Üç süreçle her 20 ms'lik döngüde CPU 15 ms meşgul olur → 15/20 = **%75**. (c) CPU'yu tamamen doldurmak için 20 / 5 = 4 süreç gerekir. Genel olarak, işler ne kadar I/O-ağırlıklıysa (CPU payı ne kadar küçükse) CPU'yu meşgul tutmak için bellekte o kadar çok iş bulunması gerekir. Bu da daha fazla bellek ve iyi bir bellek yönetimi demektir; bir sonraki bölümün konusu.

BÖLÜM 5

# Bellek yönetimi

◆ **BU BÖLÜMDE** Okuma süresi: 55 dk Zorluk: ●●○

- Belleğin neden yönetilmesi gerektiğini dört gerekçeyle açıklamak
- Dinamik bölümlemede belleğin zamanla nasıl parçalandığını çizmek
- İç ve dış parçalanmayı tanımlamak ve hangi yöntemin hangisine yol açtığını söylemek
- İlk uyan, en iyi uyan, en kötü uyan ve sonraki uyan algoritmalarını bir istek dizisine adım adım uygulamak
- Sıkıştırma ve takasın neyi çözdüğünü ve bedelini açıklamak
- Sanal bellek, sayfa, çerçeve, sayfa tablosu ve sayfa hatası kavramlarını ilişkilendirmek
- Dosya, dosya sistemi, yol ve dosya tanımlayıcısını ayırmak

**SINAV ÖNCELİĞİ: YÜKSEK** ❖ Kaynak: D1 s. 48–58

*Neden yüksek?* Öğrenme çıktılarında "basit bellek yerleştirme algoritmalarını uygulamak" açıkça var; slayt 53'teki 70K alıştırması tipik bir sınav sorusu. İç/dış parçalanma ayrımı da klasik bir çoktan seçmeli konusu.

**→ BU KONUYA NEDEN GEÇİYORUZ?**

3. ve 4. bölümde şunu gördük: CPU'yu boş bırakmamak için bellekte *birden fazla iş* tutmamız gerekiyor. Ama bellek sınırlı. O zaman şu sorular kaçınılmaz: Yeni gelen iş belleğin neresine konacak? İşler bitip boşluklar oluşunca ne olacak? Bellek yetmezse ne yapacağız? Bir iş başkasının bölgesine yazmasın diye ne yapacağız? Bu bölüm bu sorulara verilen cevapları sırayla, en basit yöntemden (bölümleme) en akıllıcasına (sanal bellek ve sayfalama) doğru anlatıyor.

## 5.1 Bellek neden yönetilmek zorunda?

❖ Kaynak: D1 s. 49

Slayt 49 dört gerekçe veriyor. Her birinin altında bir problem yatıyor:

1. **RAM hızlı ama sınırlı ve paylaşılan bir kaynaktır.** → Herkese yetmez; kimin ne kadar alacağına biri karar vermeli.
2. **Her süreç kodu, verisi ve çalışma alanı için belleğe ihtiyaç duyar.** → Süreç çalışmadan önce ona yer bulunmalı.
3. **Süreçler birbirinden korunmalı, ama gerektiğinde veri paylaşabilmelidir.** → Hem duvar hem kapı gerekiyor.
4. **İşletim sistemi hangi bölgelerin boş, hangilerinin ayrılmış olduğunu takip eder.** → Bir "boş yer defteri" tutulmalı.

Bunu bir otoparka benzetebilirsin: otopark sınırlı (1), her araç bir yer ister (2), kimse başkasının aracına çarpmamalı ama aynı aileden araçlar yan yana park edilebilmeli (3), görevli hangi yerlerin boş olduğunu bilmeli (4). Benzetmenin bozulduğu yer: otoparkta her araç aşağı yukarı aynı boyuttadır, programlar ise birkaç kilobayttan gigabaytlara kadar çok farklı boyutlardadır. Bütün zorluk da buradan çıkıyor.

## 5.2 Dinamik bölümleme ve zamanla oluşan delikler

❖ Kaynak: D1 s. 50

En basit fikir: Belleği önceden eşit parçalara bölmek yerine, her sürece **tam ihtiyacı kadar bitişik (contiguous) bir bölge** verelim. Buna **dinamik bölümleme (dynamic partitioning)** denir. Slayt 50'deki başlangıç durumu:




Şekil 5.1: Solda slayt 50'deki başlangıç. Sağda P2 bittikten sonraki durum: boş bellek yeterli ama tek parça değil. Slayttaki yan kutu ("total free memory may be sufficient, yet not available as one contiguous block") tam olarak bu ikinci durumu anlatır.

Slayttaki diyagram başlangıçta tek bir boş blok gösteriyor; yan kutudaki uyarı ise süreçler bitip başladıkça ortaya çıkacak durumu anlatıyor. Sağdaki ek örnekte P2 bitince araya 64K'lık bir delik (**hole**) açılıyor. Artık toplam 128K boş yer var ama bitişik değil: iki ayrı 64K'lık delik. 100K isteyen bir süreç yer bulamaz. Bu durumun adı **dış parçalanma**.

## 5.3 İç ve dış parçalanma

❖ Kaynak: D1 s. 51

**∑ TANIM / FORMÜL — Dış ve iç parçalanma**

- **Dış parçalanma (external fragmentation):** Boş alan **vardır**, ama küçük ve dağınık deliklere bölünmüştür; bu yüzden büyük bir istek karşılanamaz. *"Free space exists, but it is split into small scattered holes."* Dinamik bölümlemede belirgindir.
- **İç parçalanma (internal fragmentation):** Bir sürece **ayrılmış** bloğun bir kısmı o süreç tarafından **kullanılmaz**. *"Part of an allocated block is unused by the process."* Sabit boyutlu sayfalama bu tür parçalanmaya yol açabilir.

Ayırt etmenin en kolay yolu "**israf edilen alan kimin?**" sorusudur:

| | Dış parçalanma | İç parçalanma |
|---|---|---|
| Boşa giden alan nerede? | Ayrılmış blokların **arasında**, kimseye ait değil | Ayrılmış bloğun **içinde**, sahibi var ama kullanmıyor |
| Nerede ortaya çıkar? | Dinamik bölümleme (değişken boyutlu bloklar) | Sabit boyutlu bloklar (sayfalama, sabit bölümler) |
| Gündelik benzetme | Sinemada tek tek boş koltuklar var ama 4 kişilik aile yan yana oturamıyor | 4 kişilik masaya oturan 2 kişi: masa onların ama 2 sandalye boş |

**▶ ÖRNEK 5.1 — Ek örnek: sayfalamada iç parçalanma**

**Soru.** Bellek 4 KB'lık sabit parçalarla (sayfalarla) veriliyor. 13 KB'lık bir süreç kaç parça alır? Ne kadar alan iç parçalanma olarak boşa gider?
**Çözüm.** Süreç 13 KB'a ihtiyaç duyar; her parça 4 KB. 3 parça = 12 KB yetmez, 4 parça gerekir. Ayrılan: 4 × 4 = 16 KB. Kullanılan: 13 KB. **Boşa giden: 16 − 13 = 3 KB** (son parçanın içinde).
**Genel kural:** Gereken parça sayısı = boyutun parça boyutuna bölümünün **yukarı yuvarlanmış** hâli; iç parçalanma en fazla "bir parça eksi bir bayt" kadar olabilir, çünkü yalnızca son parça kısmen boştur.

## 5.4 Yerleştirme algoritmaları

❖ Kaynak: D1 s. 52

Yeni bir süreç geldiğinde, ona yetecek kadar büyük birden fazla delik olabilir. Hangisini seçeceğiz? Slayt 52 dört strateji veriyor:

| Algoritma | Hangi deliği seçer? | Güçlü yanı | Riski |
|---|---|---|---|
| İlk uyan (**first fit**) | Baştan taramaya başlar, **yetecek ilk** deliği | Genellikle hızlı | Belleğin başına yakın yerde parçalanma |
| En iyi uyan (**best fit**) | Yetenler arasında **en küçük** delik | En küçük artığı bırakır | Çok sayıda minicik delik |
| En kötü uyan (**worst fit**) | **En büyük** delik | Büyük bir artık bırakır | Büyük blokları tüketir |
| Sonraki uyan (**next fit**) | **Son kalınan yerden** itibaren yetecek ilk delik | Aramayı belleğe yayar | Daha iyi bir yakın seçeneği kaçırabilir |

Her birinin mantığı:

- **İlk uyan** tembel ama hızlı bir arayıcıdır: ilk uygun yeri bulunca durur, bütün listeyi taramaz. Hep baştan başladığı için belleğin başında küçük artıklar birikir.
- **En iyi uyan** "israfı en aza indireyim" der ve en sıkı oturan deliği seçer. Ama bıraktığı artık o kadar küçüktür ki çoğu zaman hiçbir işe yaramaz; zamanla bellek kullanılamaz kırıntılarla dolar. Ayrıca en küçüğü bulmak için (liste sıralı değilse) bütün deliklere bakması gerekir.
- **En kötü uyan** tam tersini düşünür: "En büyük deliğe koyayım ki artakalan parça hâlâ işe yarar büyüklükte olsun." Bedeli: büyük delikler hızla küçülür; sonra gerçekten büyük bir süreç geldiğinde ona yer kalmaz.

- **Sonraki uyan**, ilk uyanın bir varyasyonudur: her seferinde baştan değil, en son yerleştirme yaptığı yerden aramaya devam eder (sona gelince başa döner). Böylece belleğin başı aşırı doldurulmaz.

**▶ ÖRNEK 5.2 — Slayt örneği [D1 s. 53]: 70K'lık süreç nereye?**

**Soru.** Boş bloklar sırasıyla **100K, 500K, 200K ve 300K**. 70K'lık bir süreç gelirse ilk uyan, en iyi uyan ve en kötü uyan hangi bloğu seçer?
**Verilenler:** Delikler (bellekteki sırayla): 100, 500, 200, 300. İstek: 70. **Yöntem:** Önce 70K'ya *yeten* delikleri bulalım: 100 ≥ 70, 500 ≥ 70, 200 ≥ 70, 300 ≥ 70. Hepsi yetiyor. Sonra her algoritmanın kuralını uygulayalım.

- **İlk uyan:** Baştan bak; yeten ilk delik **100K**. → 100K seçilir, 30K artar.
- **En iyi uyan:** Yetenlerin en küçüğü: min(100, 500, 200, 300) = **100K**. → 100K seçilir, 30K artar.
- **En kötü uyan:** En büyüğü: **500K**. → 500K seçilir, 430K artar.
- *(Slaytta sorulmayan)* **Sonraki uyan:** Son kalınan konuma bağlıdır; arama baştan başlıyorsa 100K'yı seçer, en son 500K'lık blokta kalındıysa 500K'yı seçer.




Şekil 5.2: 70K'lık süreç (turuncu) ve kalan artıklar. İlk uyan ile en iyi uyan bu örnekte aynı bloğu seçer; en kötü uyan 500K'yı böler.
**Sonuç:** İlk uyan → 100K, en iyi uyan → 100K, en kötü uyan → 500K. Slaytla aynı.
**Sık hata:** En iyi uyanı "belleğe en iyi oturan = en büyük" sanmak. *Best* burada "en sıkı oturan, en az artık bırakan" demek.
**Slaytın notu:** İlk seçim, sonraki istekler için boş alanın görünümünü değiştirir. Bunu bir sonraki örnekte göreceğiz.

✔ Python yerleştirme simülasyonuyla doğrulandı

**▶ ÖRNEK 5.3 — Ek örnek: art arda gelen istekler**

**Soru.** Aynı delikler (100K, 500K, 200K, 300K). Bu kez sırayla üç süreç geliyor: **70K, 280K, 410K.** Her algoritma için yerleşimi ve hangi isteğin karşılanamadığını bul.
**Çözüm.** Her adımda delik listesini güncelliyoruz (seçilen delikten istek kadar düşülür).
*İlk uyan:* 1. 70K → yeten ilk delik 100K → delikler: 30, 500, 200, 300. 2. 280K → 30 yetmez, 500 yeter → delikler: 30, 220, 200, 300. 3. 410K → 30, 220, 200, 300: hiçbiri yetmez → **karşılanamaz.**
*En iyi uyan:* 1. 70K → yetenler 100, 500, 200, 300; en küçüğü 100 → delikler: 30, 500, 200, 300. 2. 280K → yetenler 500 ve 300; en küçüğü 300 → delikler: 30, 500, 200, 20. 3. 410K → yeten tek delik 500 → delikler: 30, 90, 200, 20. **Üçü de yerleşti.**
*En kötü uyan:* 1. 70K → en büyük 500 → delikler: 100, 430, 200, 300. 2. 280K → en büyük 430 → delikler: 100, 150, 200, 300. 3. 410K → en büyük 300 yetmez → **karşılanamaz.**
*Sonraki uyan* (baştan başlayarak): 70K → 100K bloğu; 280K → oradan devam, 500K bloğu; 410K → oradan devam: 220, 200, 300, (başa dönüp) 30 → hiçbiri yetmez → **karşılanamaz.**
**Sonuç:** Bu dizide yalnızca **en iyi uyan** üç isteği de karşıladı, çünkü büyük 500K'lık deliği en sona sakladı. Ama bu her zaman böyle değildir; başka bir istek dizisinde en iyi uyanın bıraktığı küçük kırıntılar onu yenik düşürebilir. Slayt 52'deki "hangisi en iyi?" sorusunun cevabı

**"isteklerin dizisine bağlı"**dır.

**▶ ÖRNEK 5.4 — Yarı çözümlü**

Delikler sırasıyla **200K, 300K, 150K**; istekler sırasıyla **140K, 290K, 150K**.
*En iyi uyan:* 140K → yetenler 200, 300, 150; en küçüğü 150 → delikler 200, 300, 10. 290K → yeten yalnızca 300 → delikler 200, 10, 10. 150K → ..... .
*En kötü uyan:* 140K → en büyük ..... → delikler ..... . 290K → ..... .
*Cevap bölüm sonunda.*

**✎ KENDİN ÇÖZ**

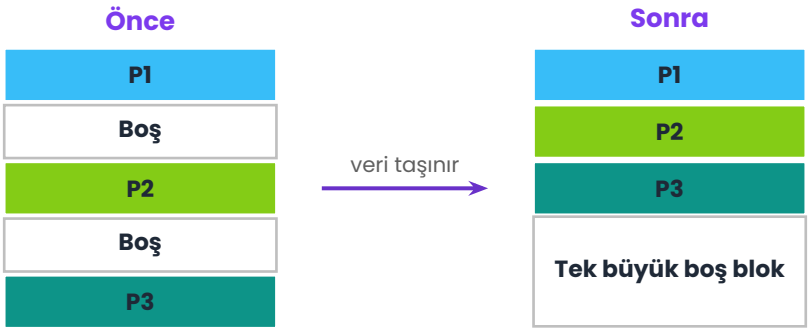
Aynı delikler (200K, 300K, 150K) ve istekler (140K, 290K, 150K) için **ilk uyan** ve **sonraki uyan** algoritmalarını uygula. Hangi algoritmalar üç isteği de karşılar?

5

## 5.5 Sıkıştırma: delikleri birleştirmek

❖ Kaynak: D1 s. 54

Dış parçalanmaya karşı akla gelen ilk çözüm: süreçleri belleğin bir ucuna doğru kaydıralım, bütün delikler diğer uçta tek bir büyük blokta toplansın. Buna **sıkıştırma (compaction)** denir.




Şekil 5.3: Sıkıştırma (D1 s. 54). Renkler süreçleri ayırt etmek için; slaytta da P2 ve P3 farklı renklerde.

Slaytın vurgusu: *Sıkıştırma dış parçalanmayı azaltır, ama verileri bellekte taşımak zaman alır.* Taşıma sırasında süreçler genellikle durdurulmak zorundadır ve megabaytlarca veriyi kopyalamak CPU zamanı harcar.

**+ EK BİLGİ (KAYNAK DIŞI) — Taşınan sürecin adresleri ne olur?**

P2'yi bellekte yukarı kaydırırsan, P2'nin içindeki "şu adresteki veriyi oku" komutları artık yanlış yeri gösterir. Bu yüzden sıkıştırma ancak donanım, sürecin adreslerini çalışma anında yeni yerine göre düzeltebiliyorsa (dinamik yeniden konumlandırma, **relocation**) mümkündür. Slayt bu ayrıntıya girmiyor; sanal bellek bu problemi kökten çözdüğü için birazdan oraya geleceğiz.

## 5.6 Takas: belleği diske ödünç vermek

❖ Kaynak: D1 s. 55

Bellek tamamen dolduysa ne yapacağız? **Takas (swapping)**:

1. **Engellenmiş (blocked) veya düşük öncelikli bir süreç geçici olarak diske taşınabilir.** "Engellenmiş" süreç, örneğin klavyeden giriş bekleyen ve şu an CPU'ya ihtiyacı olmayan süreçtir.
2. **Boşalan RAM alanına başka bir süreç getirilir.**
3. **Takas pahalıdır, çünkü disk erişimi RAM'den çok daha yavaştır.**
4. **Hangi sürecin ne zaman dışarı alınacağına işletim sistemi karar verir.**

Benzetme: Masan küçük ve üzerinde dört dersin kitapları açık. Bir saat dokunmayacağın fizik kitabını kapatıp rafa (diske) koyarsın, açılan yere matematiği açarsın. Fizik gerekince raftan geri almak zaman alır.

## 5.7 Sanal bellek fikri

❖ Kaynak: D1 s. 56

Şimdiye kadarki bütün sorunlar (bitişik yer bulamamak, dış parçalanma, sıkıştırmanın adres problemi, bellek yetmemesi) tek bir varsayımdan çıkıyor: *Sürecin gördüğü adresler, RAM'deki gerçek adreslerle aynı.* Sanal bellek bu varsayımı yıkar.




Şekil 5.4: Sanal belleğin fikri (D1 s. 56): süreç fiziksel RAM'in düzenini doğrudan görmez.

**Sanal bellek (virtual memory)** ile her süreç kendine ait, büyük ve **bitişik** bir adres alanı varmış gibi görür. Süreç "50.000 numaralı adresi oku" dediğinde bu bir **sanal adres**tir; işletim sistemi ve donanım bunu RAM'deki gerçek bir **fiziksel adrese** çevirir. Slaytın vurgusu: *Süreç fiziksel RAM'in düzenini doğrudan görmez.*

Bu neden harika? (1) Sürecin parçaları RAM'de dağınık durabilir, süreç bunu fark etmez → bitişik yer bulma derdi biter. (2) O an kullanılmayan parçalar diskte durabilir → RAM'den büyük programlar bile çalışabilir. (3) Her sürecin adresleri ayrı ayrı çevrildiği için bir süreç diğerinin belleğine "adres yazarak" ulaşamaz → yalıtım.

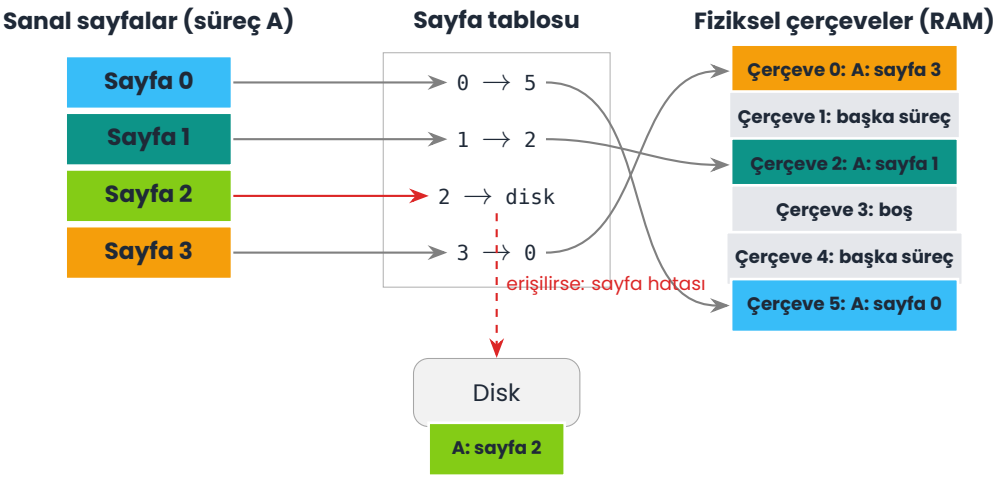
1. bölümdeki soyutlamayı hatırla: dosya, diskin bloklarını soyutluyordu; sanal bellek de RAM'in fiziksel düzenini soyutluyor.

## 5.8 Sayfalama: sanal belleğin nasıl çalıştığı

❖ Kaynak: D1 s. 57

Sanal belleğin en yaygın uygulaması **sayfalama (paging)**dır. Slayt 57'nin dört maddesi:

1. **Sanal bellek sabit boyutlu sayfalara (pages), fiziksel bellek de aynı boyutta çerçevelere (frames) bölünür.**
2. **Bir sürecin sayfalarının fiziksel bellekte yan yana olması gerekmez.**
3. **Sayfa tablosu (page table), bir sanal sayfayı bir fiziksel çerçeveye eşler.**
4. **RAM'de olmayan bir sayfaya erişmek sayfa hatasına (page fault) yol açar; veri diskten getirilir.**




Şekil 5.5: Sayfalama. Süreç A'nın sayfaları 0–1–2–3 sırasıyla bitişik görünür ama RAM'de 5, 2 ve 0 numaralı çerçevelere dağılmıştır; sayfa 2 ise şu an diskte. Sayfa 2'ye erişim sayfa hatası doğurur.



### 5.8.1 Sayfa hatası olunca ne olur?

Sayfa hatası (**page fault**) bir "hata" kelimesi taşısa da bir program hatası değildir; normal bir olaydır. Adımlar:

1. Süreç, sayfa tablosunda "RAM'de değil" diye işaretli bir sayfaya erişmek ister.
2. Donanım bunu fark eder ve kontrolü işletim sistemine verir.
3. İşletim sistemi sayfayı diskte bulur, RAM'de boş bir çerçeve ayırır (boş çerçeve yoksa başka bir sayfayı diske gönderip yer açar).
4. Sayfa diskten çerçeveye okunur; bu yavaştır, süreç bu sırada **bekler** (6. bölümdeki *Waiting* durumu).
5. Sayfa tablosu güncellenir ve süreç kaldığı komuttan devam eder.

Sayfalama ile **dış parçalanma ortadan kalkar**: Her çerçeve aynı boyutta olduğu için herhangi bir boş çerçeve herhangi bir sayfaya uyar; "bitişik büyük delik" arama derdi yoktur. Bedeli ise iç parçalanma (son sayfanın içi kısmen boş kalabilir) ve sayfa tablosunu tutmanın maliyetidir.

**+ EK BİLGİ (KAYNAK DIŞI) — Adres çevirisi sayılarla**

Sayfa boyutu 4096 bayt (4 KB) olsun. Sanal adres 9000'i çevirelim. 9000 / 4096 = 2, kalan 808 → **sayfa 2, sayfa içi konum (offset) 808**. Sayfa tablosu "sayfa 2 → çerçeve 7" diyorsa fiziksel adres = 7 × 4096 + 808 = **29.480**. Yani sayfa numarası değişir, sayfa içi konum aynen korunur. Slayt bu hesaba girmiyor; ileriki işletim sistemleri derslerinde temel hesap türlerinden biridir.

**★ MERAKLISINA (SINAV DIŞI)**

Bilgisayarın yavaşladığında diskin sürekli çalıştığını duyduğun (ya da gördüğün) olmuştur. Bu genellikle RAM'in yetmediği ve işletim sisteminin sürekli sayfa hatası işleyip sayfaları diske gidip geldiği durumdur. Bu aşırı durumun ders kitaplarındaki adı **thrashing**'tir: bilgisayar zamanının çoğunu iş yapmak yerine sayfa taşımaya harcar. RAM eklemek bu yüzden eski bir bilgisayarı gözle görülür biçimde hızlandırabilir.

## 5.9 Dosya yöneticisinin rolü

❖ Kaynak: D1 s. 58

Slayt 58 "MEMORY MANAGEMENT" etiketiyle gelse de konusu **dosya yönetimi**; kalıcı depolamanın yönetimi. Dört kavram:

| Kavram | Anlamı | Örnek |
|---|---|---|
| Dosya (**file**) | İlgili verileri **isimli mantıksal bir birim** hâlinde toplar | `rapor.pdf` |
| Dosya sistemi (**file system**) | Verilerin depolama aygıtında **nasıl düzenlendiğini** tanımlar | NTFS, ext4, FAT32 gibi biçimler |
| Dizin yolu (**directory path**) | Dosyanın **konumunu** belirtir | `C:\Users\Arda\Ders\rapor.pdf` |
| Dosya tanımlayıcısı (**file descriptor**) | **Açık** bir dosyayı temsil eden numara/tutamak | `open` çağrısının döndürdüğü küçük sayı (ör. 3) |

Ayrıca **bir işletim sistemi birden fazla dosya sistemini destekleyebilir**: Windows hem NTFS biçimli diskini hem de FAT32 biçimli USB belleğini okuyabilir. (Dosya sistemi adları slaytta yok, örnek olarak eklendi.)

Yol ile tanımlayıcı arasındaki fark sık sorulur: *yol* dosyanın adresidir ve dosya kapalıyken de vardır; *tanımlayıcı* ise dosya açıldığında işletim sisteminin verdiği geçici bir numaradır. 1. bölümdeki Python örneğinde `open` çağrısının döndürdüğü şey buydu.

## 5.10 Bu bölümün bağlantıları

- Bellek koruması → 1. bölümdeki kullanıcı/çekirdek kipi ve 4. bölümdeki **yalıtım**.
- Takas ve sayfa hatası → 6. bölümdeki **Waiting** durumu ve PCB'deki **bellek yönetimi bilgisi**.
- Sayfalamanın "gereken sayfalar RAM'de, kalanı diskte" fikri → 8. bölümdeki **bellek hiyerarşisi** ve **yerellik**: sık kullanılan şey hızlı katmanda tutulur.
- Yerleştirme algoritmalarındaki "her durumda en iyisi yok" → 7. bölümdeki zamanlama algoritmalarında aynı ders.

**⚠ SINAVDA DİKKAT**

- **Dış parçalanma:** boş alan var ama dağınık → dinamik bölümleme. **İç parçalanma:** ayrılan bloğun içi boş → sabit boyutlu bloklar/sayfalama.
- Yerleştirme: **ilk uyan** = yeten *ilk*; **en iyi uyan** = yeten *en küçük*; **en kötü uyan** = *en büyük*; **sonraki uyan** = *son konumdan* sonra yeten ilk.
- 70K örneği: **100K / 100K / 500K** (ilk / en iyi / en kötü).
- **Sıkıştırma** dış parçalanmayı azaltır ama **zaman alır**. **Takas** belleği boşaltır ama **disk yavaştır**.
- **Sayfa ↔ çerçeve** (aynı boyut); **sayfa tablosu** eşler; **sayfa hatası** = sayfa RAM'de değil, diskten getirilir.

**❖ BU KONUDAN NASIL SORU GELİR?**

1. **Uygulama (en olası):** "Free holes are ...; a process of size ... arrives. Which hole is selected by first fit, best fit, worst fit (and next fit)?" *İpucu:* önce yeten delikleri işaretle, sonra kuralı uygula; art arda istek varsa her adımda listeyi güncelle.
2. **Ayırt etme:** "Explain the difference between internal and external fragmentation. Which one does paging cause?" *İpucu:* boşa giden alan kimin? Sayfalama → iç.
3. **Neden sorusu:** "Why is compaction expensive?" / "Why is swapping expensive?" *İpucu:* veri taşıma / disk yavaşlığı.
4. **Kavramsal:** "What happens on a page fault?" *İpucu:* sayfa RAM'de yok → işletim sistemi diskten getirir → tabloyu günceller → süreç devam eder.
5. **Tanım eşleştirme:** dosya, dosya sistemi, yol, dosya tanımlayıcısı.

**✘ SIK YAPILAN HATA**

- **"En iyi uyan her zaman en iyi algoritmadır."** → Doğrusu: İsteklerin dizisine bağlıdır; en iyi uyan çok sayıda kullanılamaz minik delik bırakabilir. → Neden yanlış: Slayt 52 bile riskini "many tiny holes" olarak yazıyor.
- **"İç parçalanma dinamik bölümlemede çıkar."** → Doğrusu: Dinamik bölümleme sürece tam ihtiyacı kadar yer verdiği için iç parçalanma yok denecek kadar azdır; onun derdi **dış** parçalanmadır.
- **Art arda isteklerde delik listesini güncellemeyi unutmak.** → Doğrusu: Seçilen delikten istek kadar düş, sonraki isteği *güncel* listeye göre yerleştir.
- **"Sayfa hatası programın çöktüğü anlamına gelir."** → Doğrusu: Normal bir olaydır; işletim sistemi sayfayı diskten getirir ve süreç devam eder.
- **"Sayfalar RAM'de yan yana olmalı."** → Doğrusu: Slayt 57: sayfaların bitişik olması gerekmez; bütün mesele bu.

**⚡ HIZLI TEKRAR**

- Bellek yönetimi: **yer ver, takip et, koru, paylaşıma izin ver.**
- **Dinamik bölümleme:** tam boyutta bitişik blok → zamanla **dış parçalanma**.
- **İlk / en iyi / en kötü / sonraki uyan:** ilk yeten / en küçük yeten / en büyük / son yerden ilk yeten.
- **Sıkıştırma** delikleri birleştirir (pahalı); **takas** süreci geçici olarak diske atar (pahalı).
- **Sanal bellek:** süreç büyük ve bitişik adres alanı görür; **adres çevirisi** sanal → fiziksel.
- **Sayfalama:** sayfa ↔ çerçeve, sayfa tablosu, **sayfa hatası**; dış parçalanma yok, az miktarda **iç parçalanma** var.
- **Dosya** (isimli birim), **dosya sistemi** (düzen), **yol** (konum), **tanımlayıcı** (açık dosya).

*Ezberle:* dört yerleştirme kuralı; iç/dış parçalanmanın tanımı. *Türetebil:* her yerleştirme sorusunun cevabı, sayfa sayısı ve iç parçalanma hesabı.
**Bu bölümden sonra şunları yapabiliyor olmalısın:** bir istek dizisini dört algoritmayla yerleştirmek; parçalanma türünü tanımak; sayfa hatası adımlarını sıralamak.

## 5.11 Bölüm cevapları

**Yarı çözümlü (200K, 300K, 150K / 140K, 290K, 150K).** *En iyi uyan:* 150K → yetenler yalnızca 200 (300 artık 10, 150 artık 10) → 200'den yerleşir, delikler 50, 10, 10. **Üçü de yerleşir.** *En kötü uyan:*

140K → en büyük **300** → delikler 200, 160, 150. 290K → en büyük 200 bile yetmez → **karşılanamaz.** (150K ise sonra en büyük delik olan 200'e yerleşir.)

**Kendin çöz.** *İlk uyan:* 140K → 200'e (60, 300, 150); 290K → 60 yetmez, 300'e (60, 10, 150); 150K → 60 ve 10 yetmez, 150'ye (60, 10, 0). Üçü de yerleşir. *Sonraki uyan* (baştan başlayarak) bu örnekte ilk uyanla aynı yolu izler: 140K → 200; 290K → oradan devam, 300; 150K → oradan devam, 150. Üçü de yerleşir. **Cevap:** ilk uyan, sonraki uyan ve en iyi uyan üçünü de karşılar; yalnızca en kötü uyan 290K'yı karşılayamaz.

BÖLÜM 6

# Süreç yönetimi

◆ **BU BÖLÜMDE** Okuma süresi: 50 dk Zorluk: ●●○

- Program ile süreç arasındaki farkı açıklamak
- Süreç Denetim Bloğu'nun (PCB) neleri tuttuğunu ve neden gerekli olduğunu söylemek
- Beş süreç durumunu ve aralarındaki geçişleri, her geçişi tetikleyen olayla birlikte çizmek
- Hazır kuyruğu, bekleme kuyrukları ve zamanlayıcının rolünü ilişkilendirmek
- Bağlam değiştirmeyi adım adım anlatmak ve ek yükünü hesaplamak
- Kesintili ve kesintisiz zamanlamayı ayırmak
- Beş zamanlama ölçütünü tanımlamak ve bir Gantt şemasından hesaplamak

**SINAV ÖNCELİĞİ: YÜKSEK** ❖ Kaynak: D1 s. 59–70

*Neden yüksek?* Durum diyagramı ve geçiş olayları neredeyse her işletim sistemi sınavında çıkar; slayt 69'daki ölçüt tanımları ise 7. bölümdeki bütün hesapların temelidir. Bu bölümü anlamadan zamanlama sorularını çözemezsin.

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Bellek yönetimiyle süreçlere *yer* bulmayı öğrendik. Ama 4. bölümün sonunda gördüğümüz gibi çoklu programlamanın ikinci ihtiyacı **zamanlama**: CPU'yu kime, ne zaman vereceğiz? Bunu yapabilmek için önce işletim sisteminin bir süreci nasıl "tanıdığını", onun o an ne durumda olduğunu nasıl takip ettiğini ve onu yarıda kesip sonra kaldığı yerden nasıl devam ettirdiğini bilmemiz gerekiyor. Bu bölüm 7. bölümdeki algoritmaların altyapısı: hem kavramlar hem de hesap ölçütleri burada.

## 6.1 Program ile süreç aynı şey değil

❖ Kaynak: D1 s. 60

| | Program | Süreç (process) |
|---|---|---|
| Nedir? | Diskte saklanan **durağan (static)** komutlar ve veri | Bir programın **çalışan bir örneği (running instance)** |
| Kendi başına çalışır mı? | **Hayır**, kendi başına yürütülmez | Evet; **durumu, belleği, yazmaçları ve kaynakları** vardır |
| Benzetme | Yemek tarifi kitabı | O tarifle şu an mutfakta pişen yemek |

Tarif kitabı rafta durur, kendi başına yemek yapmaz. Aynı tariften aynı anda iki ayrı tencerede yemek pişebilir; her birinin kendi malzemesi ve "hangi adımdayız?" bilgisi vardır. Bilgisayarda da tek bir

program dosyasından (ör. tarayıcı) aynı anda birden fazla süreç çalışabilir; her biri ayrı bir süreçtir, ayrı belleğe ve ayrı duruma sahiptir.

**✎ DURUP DÜŞÜN**

Not defteri uygulamasını iki kez açtın ve iki ayrı pencere var. Kaç program, kaç süreç var?

## 6.2 Süreç Denetim Bloğu (PCB)

❖ Kaynak: D1 s. 61

İşletim sistemi bir süreci durdurup sonra kaldığı yerden devam ettirebilmek için o süreç hakkında her şeyi bir yere yazmalı. Bu kayıt **Süreç Denetim Bloğu (Process Control Block, PCB)**dur. Her sürecin bir PCB'si vardır.

| PCB — süreç P3 |
|---|
| **Kimlik ve durum:** PID = 3, durum = Hazır |
| **Program sayacı + CPU yazmaçları:** sıradaki komut adresi, ara değerler |
| **Bellek yönetimi bilgisi:** sayfa tablosu nerede, sınırlar |
| **Açık dosyalar ve I/O durumu:** `rapor.txt` açık |
| **Zamanlama bilgisi:** öncelik, şimdiye kadar kullanılan CPU süresi |

CPU elinden alınınca yazmaçlar buraya **kaydedilir**; sırası gelince buradan **geri yüklenir**.

**Şekil 6.1:** Bir PCB'nin içeriği (D1 s. 61; örnek değerler uydurmadır).

Slayt 61'deki beş madde ve ne işe yaradıkları:

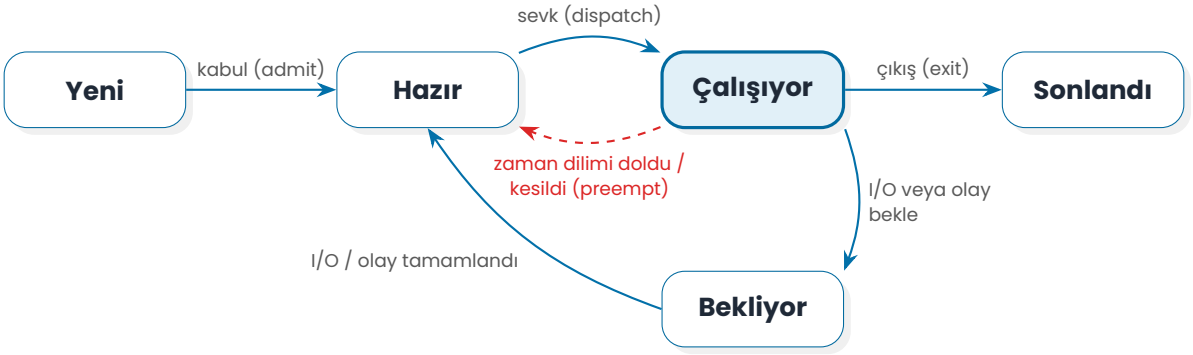
- **Süreç kimliği (process identifier, PID) ve mevcut durum:** Hangi süreç, şu an ne yapıyor?
- **Program sayacı (program counter) ve CPU yazmaçları (registers):** Program sayacı, sürecin *sıradaki* komutunun adresidir. Yazmaçlar, CPU'nun içindeki küçük ve çok hızlı ara bellek hücreleridir; süreç durdurulduğunda o anki ara sonuçlar burada durur. Bunlar kaydedilmezse süreç kaldığı yerden devam edemez.
- **Bellek yönetimi bilgisi:** Sürecin bellekte nerede olduğu (5. bölümdeki sayfa tablosu gibi).
- **Açık dosyalar ve I/O durumu:** Hangi dosyalar açık, hangi aygıt bekleniyor.
- **Zamanlama bilgisi ve öncelik:** Zamanlayıcının karar vermek için kullandığı bilgiler (7. bölümdeki öncelikli zamanlama bunu kullanır).

Slaytın vurgusu: *PCB, işletim sisteminin bir süreci duraklatıp daha sonra doğru biçimde devam ettirmesini sağlar.* Kitap okurken araya giren bir iş için kitabı kapatırken koyduğun ayraç ve "şu karakter şunu yapmıştı" notun gibidir.

## 6.3 Bir sürecin hayatı: beş durum

❖ Kaynak: D1 s. 62–63

Bir süreç doğduğu andan bittiği ana kadar beş durumdan geçer:




**Şekil 6.2:** Süreç durum diyagramı. Düz oklar slayt 62'deki diyagramdan; kırmızı kesikli ok (Çalışıyor → Hazır) slayt 62'nin çiziminde yok ama slayt 63'te geçiş olarak listeleniyor.

| Geçiş | Tetikleyen olay (slayt 63) | Gündelik örnek |
|---|---|---|
| **Yeni → Hazır** (admit) | Süreç oluşturulur ve hazır kuyruğuna kabul edilir | Bir uygulamayı açtın; yüklendi, sırasını bekliyor |
| **Hazır → Çalışıyor** (dispatch) | Zamanlayıcı süreci CPU'ya sevk eder | Sırası geldi, CPU onda |
| **Çalışıyor → Bekliyor** | Süreç I/O'yu veya bir olayı bekler | Program diskten dosya okumak istedi |
| **Bekliyor → Hazır** | Beklenen olay tamamlanır | Disk okumayı bitirdi (kesme geldi) |
| **Çalışıyor → Hazır** | Zaman dilimi dolar veya süreç kesilir (preempt) | Sırasını başkasına vermek zorunda kaldı |
| **Çalışıyor → Sonlandı** (exit) | Süreç işini bitirir | Programı kapattın |

**⚠ SINAVDA DİKKAT — Diyagram ile liste arasındaki fark**

Slayt 62'deki çizimde **Çalışıyor → Hazır** oku yok, ama slayt 63 bu geçişi açıkça listeliyor. Sınavda diyagram çizmen istenirse bu oku **mutlaka ekle**: kesintili (preemptive) zamanlama ve Round Robin bu ok olmadan anlatılamaz. ("Hocaya sorulacaklar" listesinde.)

En sık karıştırılan nokta: **Bekliyor'dan doğrudan Çalışıyor'a geçiş yoktur.** I/O bitince süreç *hazır kuyruğuna* döner ve sırasını bekler; çünkü o anda CPU'da başka bir süreç çalışıyor olabilir. Bir benzetme: Hastanede tahlil sonucunu bekleyen hasta (Bekliyor), sonuç gelince doğrudan doktorun odasına dalmaz; tekrar bekleme salonuna (Hazır) geçer ve çağrılmayı bekler.

İkinci kritik ayrım: **Hazır ile Bekliyor farklıdır.** Hazır süreç *her şeye sahiptir, yalnızca CPU'yu bekler*; CPU verilse hemen çalışır. Bekleyen süreç ise CPU verilse bile çalışamaz, çünkü bir olayı (disk, ağ, klavye) bekliyordur.

**▶ ÖRNEK 6.1 — Ek örnek: bir sürecin durumlarını izlemek**

**Soru.** Bir fotoğraf düzenleme programını açtın. Program (1) yüklendi, (2) CPU'yu aldı ve bir fotoğrafı diskten açmak istedi, (3) fotoğraf okundu, (4) tekrar CPU'yu aldı ve filtre uygulamaya başladı, (5) zaman dilimi doldu, (6) tekrar CPU'yu aldı ve filtreyi bitirdi, (7) sen programı kapattın. Süreç hangi durumlardan geçti?
**Çözüm.** Her olayı slayt 63'teki geçişlerden biriyle eşleştiriyoruz:

1. Yüklendi → **Yeni → Hazır**.
2. CPU'yu aldı → **Hazır → Çalışıyor**; dosya okumak istedi → **Çalışıyor → Bekliyor**.
3. Fotoğraf okundu → **Bekliyor → Hazır** (doğrudan Çalışıyor'a değil!).
4. CPU'yu aldı → **Hazır → Çalışıyor**.
5. Zaman dilimi doldu → **Çalışıyor → Hazır**.
6. CPU'yu aldı → **Hazır → Çalışıyor**.
7. Kapattın → **Çalışıyor → Sonlandı**.

**Durum dizisi:** Yeni → Hazır → Çalışıyor → Bekliyor → Hazır → Çalışıyor → Hazır → Çalışıyor → Sonlandı.

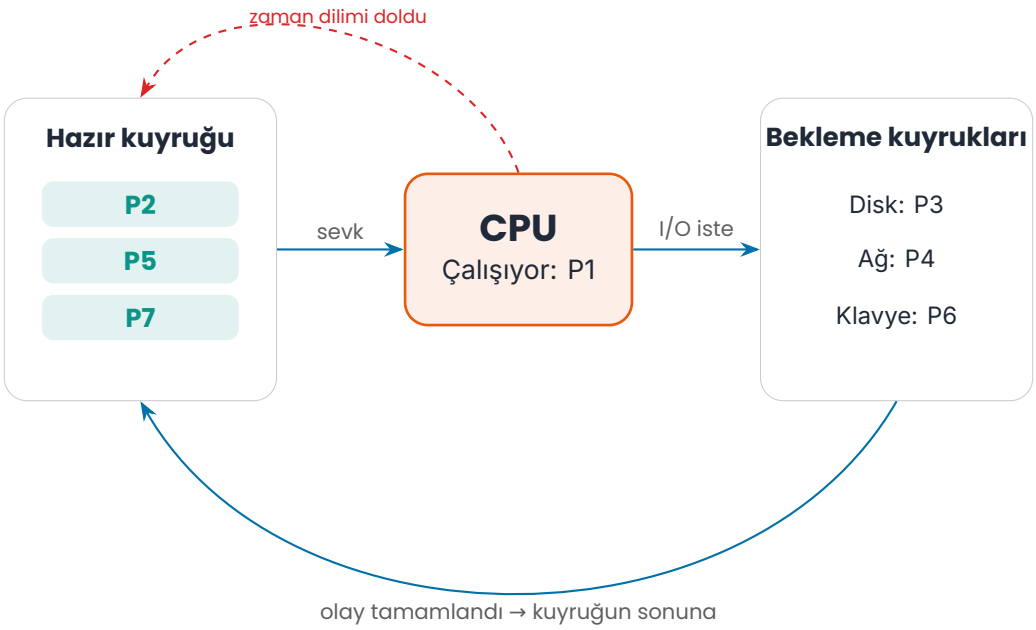
**Bu tip soruda sık hata:** 3. adımda süreci doğrudan Çalışıyor'a geçirmek.

## 6.4 Zamanlayıcı ve kuyruklar

❖ Kaynak: D1 s. 64–65

**CPU zamanlayıcısı (CPU scheduler)**, CPU boşaldığında hazır süreçlerden hangisinin çalışacağına karar veren işletim sistemi parçasıdır. Slayt 64:

- **Hazır kuyruğundaki süreçler arasından seçim yapar.**
- **Seçim politikası (policy) sistemin hedeflerine bağlıdır.** (7. bölümdeki algoritmalar bu politikalardır.)
- **Zamanlayıcı hızlı olmalıdır, çünkü çok sık çalışır.** Saniyede yüzlerce kez karar veriyorsa, her kararda harcanan zaman ek yüktür.
- **Seçim adaleti, yanıt süresini ve throughput'u doğrudan etkiler.**




Şekil 6.3: Hazır ve bekleme kuyrukları (D1 s. 65'teki durum). Oklar, durum diyagramındaki geçişlerin kuyruklar arasındaki hareket olarak görünümüdür.

Slayt 65'teki anlık görüntüde P1 CPU'da çalışıyor; P2, P5, P7 hazır kuyruğunda; P3 diski, P4 ağı, P6 klavyeyi bekliyor. Her aygıt için **ayrı bir bekleme kuyruğu** olduğuna dikkat et: disk okuması biten süreç, klavye bekleyenlerle karışmaz.

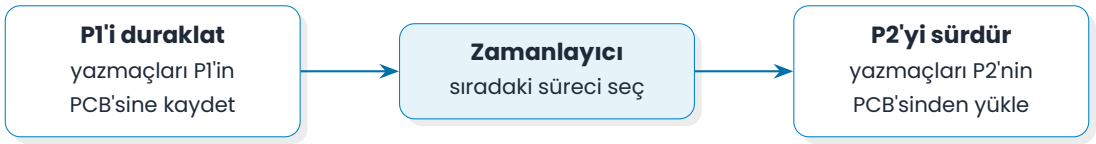
**✎ DURUP DÜŞÜN**

Slayt 65'teki durumda P1 diskten bir dosya okumak isterse hangi kuyruğa girer? CPU'yu bu durumda (FCFS sırasıyla) kim alır?

## 6.5 Bağlam değiştirme

❖ Kaynak: D1 s. 66

CPU P1'den P2'ye geçerken ne olur? **Bağlam değiştirme (context switch)**:




Şekil 6.4: Bağlam değiştirmenin üç adımı (D1 s. 66).

Slaytın vurgusu: *Bağlam değiştirme gerekli bir ek yüktür (overhead) ve doğrudan kullanıcı işi yapmaz.* Geçiş süresince CPU ne P1'in ne de P2'nin işini yapar; yalnızca "devir teslim" yapar. Futbolda oyuncu değişikliği gibi: değişiklik gerekli ama o sırada kimse gol atmıyor.

**▶ ÖRNEK 6.2 — Ek örnek: bağlam değiştirmenin maliyeti**

**Soru.** Her süreç CPU'yu 4 ms kullanıyor, sonra bağlam değiştirme yapılıyor ve her bağlam değiştirme 1 ms sürüyor. CPU zamanının yüzde kaçı ek yüke gidiyor? Zaman dilimi 4 ms yerine 99 ms olsaydı?

**Çözüm.** Her "döngü" = iş süresi + geçiş süresi.

- 4 ms dilim: döngü 4 + 1 = 5 ms; ek yük oranı = 1 / 5 = **%20**. CPU zamanının beşte biri kimsenin işine yaramıyor!
- 99 ms dilim: döngü 99 + 1 = 100 ms; ek yük = 1 / 100 = **%1**.

**Genel formül:** ek yük oranı = s / (q + s); burada q zaman dilimi, s bağlam değiştirme süresi.

**Ders:** Zaman dilimi küçüldükçe ek yük oranı büyür. 7. bölümdeki "kuantum, bağlam değiştirme süresinden anlamlı ölçüde büyük olmalı" kuralının sayısal gerekçesi budur.

## 6.6 Kesintisiz ve kesintili zamanlama

❖ Kaynak: D1 s. 67

| | Kesintisiz (non-preemptive) | Kesintili (preemptive) |
|---|---|---|
| CPU ne zaman el değiştirir? | Süreç CPU'yu **kendisi bırakana** ya da **bitene** kadar çalışır | İşletim sistemi CPU'yu çalışan süreçten **zorla alabilir** |
| Artısı | **Basit** | **Yanıt verebilirliği (responsiveness)** artırabilir |
| Eksisi | Uzun işler kısa işleri geciktirebilir | Geçiş maliyeti ve eşzamanlılık karmaşıklığı ekler |
| 7. bölümdeki örnekler | FCFS, SJF | SRTF, Round Robin |

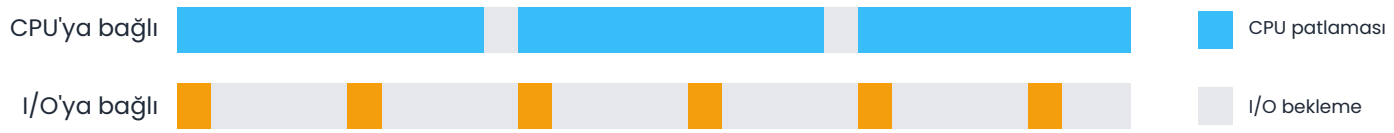
"Kendisi bırakır" ne demek? Süreç bir I/O isteği yaptığında (Çalışıyor → Bekliyor) ya da bittiğinde

CPU'yu bırakır. Kesintili zamanlamada buna ek olarak durum diyagramındaki **Çalışıyor → Hazır** oku devreye girer. Öncelikli zamanlama ise iki biçimde de uygulanabilir (slayt 82).

## 6.7 CPU patlamaları ve I/O patlamaları

❖ Kaynak: D1 s. 68

Bir süreç hayatı boyunca iki davranış arasında gidip gelir: **CPU patlaması (CPU burst)**, yani komutları çalıştırdığı bir dönem, ve **I/O beklemesi (I/O burst)**, yani disk, ağ veya kullanıcıyı beklediği bir dönem. *CPU patlaması → I/O bekleme → CPU patlaması → …* Slayt: *Süreçler genellikle bu iki davranış arasında tekrar tekrar gidip gelir.*




**Şekil 6.5:** Uzun CPU patlamaları ve seyrek I/O ile CPU'ya bağlı bir süreç; kısa CPU patlamaları ve sık I/O ile I/O'ya bağlı bir süreç (şematik).

Bu ayrım 3. bölümdeki "bound" kavramının zamanlamadaki karşılığıdır: **CPU'ya bağlı** süreçler uzun CPU patlamaları yapar, **I/O'ya bağlı** süreçler kısa patlamalar yapıp hemen I/O'ya girer. Zamanlama algoritmaları açısından önemli bir sonuç: I/O'ya bağlı kısa bir süreç, uzun bir CPU'ya bağlı sürecin arkasında beklerse hem kendisi hem de I/O aygıtları boşuna bekler. 7. bölümde bunun adı **konvoy etkisi** olacak.

## 6.8 Zamanlama ölçütleri

❖ Kaynak: D1 s. 69

Bir zamanlama algoritmasının iyi olup olmadığını ölçmek için slayt 69 beş ölçüt veriyor. Bu tablo 7. bölümün bütün hesaplarının temelidir:

| Ölçüt | Tanım (slayt) | Hedef |
|---|---|---|
| CPU kullanımı (**CPU utilization**) | CPU'nun meşgul olduğu zamanın oranı | **En büyük** |
| Üretilen iş (**throughput**) | Birim zamanda tamamlanan süreç sayısı | **En büyük** |
| Tamamlanma süresi (**turnaround time**) | Varıştan bitişe kadar geçen toplam süre | **En küçük** |
| Bekleme süresi (**waiting time**) | Hazır kuyruğunda geçen toplam süre | **En küçük** |
| Yanıt süresi (**response time**) | İlk yanıta kadar geçen süre | **En küçük** |

**∑ TANIM / FORMÜL — Hesap formülleri (tek CPU patlamalı süreçler için)**

Bir sürecin varış zamanı **A** (arrival), CPU patlaması (çalışma süresi) **B** (burst), bitiş zamanı **C** (completion) ve CPU'yu ilk aldığı an **S** (start) olsun.

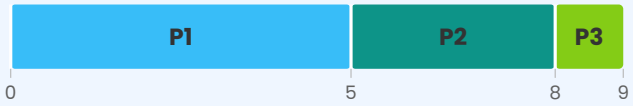
- Tamamlanma süresi: **TAT = C − A**
- Bekleme süresi: **W = TAT − B = C − A − B**
- Yanıt süresi: **R = S − A**
- Ortalama değerler: hepsinin toplamı / süreç sayısı
- Throughput: tamamlanan süreç sayısı / geçen toplam süre

*Nereden geliyor?* Süreç sisteme girdiği andan çıktığı ana kadar (TAT) ya çalışmıştır ya da hazır kuyruğunda beklemiştir (I/O yapmıyorsa). Çalıştığı toplam süre B olduğuna göre geri kalan her şey beklemedir: W = TAT − B. Bu formül **kesintili algoritmalarda da** geçerlidir; süreç kaç parça hâlinde çalışırsa çalışsın.
*Varsayım:* Süreçler I/O yapmıyor. I/O olsaydı I/O'da geçen süre de TAT'tan düşülmeliydi, çünkü bekleme süresi yalnızca *hazır kuyruğunda* geçen süredir.

Neden formüle ihtiyaç var? Çünkü bekleme süresini Gantt şemasından tek tek toplamak, süreç parçalara bölündüğünde (Round Robin, SRTF) çok kolay hata yaptırır. W = TAT − B ise tek satırda biter.

**▶ ÖRNEK 6.3 — Ek örnek: bir Gantt şemasından bütün ölçütler**

**Soru.** Üç süreç FCFS ile çalışıyor. P1: varış 0, patlama 5. P2: varış 1, patlama 3. P3: varış 2, patlama 1. Bütün ölçütleri hesapla.
**Gantt şeması:** P1 0–5, P2 5–8, P3 8–9.




**Çözüm** aşağıdaki tabloda (C = bitiş, S = ilk başlama).

| Süreç | A | B | S | C | TAT = C − A | W = TAT − B | R = S − A |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 5 | 0 | 5 | 5 | 0 | 0 |
| P2 | 1 | 3 | 5 | 8 | 7 | 4 | 4 |
| P3 | 2 | 1 | 8 | 9 | 7 | 6 | 6 |

Yukarıdaki tablodan ortalamalar: TAT = (5 + 7 + 7) / 3 = 19/3 ≈ **6,33**; W = (0 + 4 + 6) / 3 = 10/3 ≈ **3,33**; R = (0 + 4 + 6) / 3 ≈ **3,33**. Throughput = 3 süreç / 9 birim ≈ **0,33 süreç/birim**. CPU 0–9 arası hiç boş kalmadığı için CPU kullanımı **%100**.

✔ Python zamanlama simülatörüyle doğrulandı

**Gözlem:** Kesintisiz algoritmalarda bir süreç CPU'yu ilk aldığında bitene kadar bırakmadığı için **R = W** olur. Kesintili algoritmalarda bu eşitlik bozulur; aynı süreçleri 7. bölümde dört algoritmayla tekrar çalıştıracağız.

**✎ KENDİN ÇÖZ**

Aynı üç süreç (P1: 0/5, P2: 1/3, P3: 2/1) için Gantt şeması **P1 0–5, P3 5–6, P2 6–9** olsaydı (bu, 7. bölümdeki SJF'nin sonucu), ortalama bekleme ve ortalama tamamlanma süresi ne olurdu? FCFS'ye göre hangisi iyileşti?

## 6.9 "En iyi" algoritma diye bir şey yok

❖ Kaynak: D1 s. 70

Slayt 70 bölümü bir mühendislik dersiyle bağlıyor:

- **Toplu işlem sistemleri** throughput ve tamamlanma süresine öncelik verebilir.

- **Etkileşimli sistemler** yanıt süresine ve adalete öncelik verir.
- **Gerçek zamanlı sistemler** son teslim sürelerine öncelik verir.
- **Bir ölçütü iyileştirmek başka bir ölçütü kötüleştirebilir.**

Bu, 1. bölümdeki "ödünleşim" fikrinin ve 4. bölümdeki slayt 45 tablosunun zamanlamadaki yansımasıdır. Sınavda "hangi algoritma en iyidir?" diye sorulursa cevabın ilk cümlesi hep şudur: **"Sistemin hedefine bağlıdır."**

## 6.10 Bu bölümün bağlantıları

- PCB'deki bellek bilgisi → 5. bölümdeki sayfa tablosu; açık dosyalar → 5. bölümdeki dosya tanımlayıcısı.
- Bekliyor → Hazır geçişini tetikleyen şey → 4. bölümdeki **kesme**.
- Çalışıyor → Hazır (preempt) → 4. bölümdeki **çoklu görev** ve 7. bölümdeki **SRTF, Round Robin**.
- Ölçütler ve formüller → 7. bölümdeki bütün sayısal örnekler.
- Bağlam değiştirme ek yükü → 7. bölümdeki **kuantum seçimi**.

**⚠ SINAVDA DİKKAT**

- **Program** durağan, **süreç** çalışan örnek (durum + bellek + yazmaçlar + kaynaklar).
- **PCB:** kimlik/durum, program sayacı + yazmaçlar, bellek bilgisi, açık dosyalar/I/O, zamanlama bilgisi/öncelik. İşlevi: **duraklat ve doğru şekilde devam ettir.**
- **Bekliyor → Çalışıyor diye bir geçiş yok;** Bekliyor → **Hazır**.
- **Hazır** = yalnızca CPU'yu bekliyor; **Bekliyor** = bir olayı bekliyor.
- **Bağlam değiştirme** = ek yük; ek yük oranı = s / (q + s).
- **TAT = C − A, W = TAT − B, R = S − A.**
- Kesintisiz → R = W.

**❖ BU KONUDAN NASIL SORU GELİR?**

1. **Diyagram:** "Draw the process state diagram and label each transition with the event that causes it." *İpucu:* beş durum, altı ok (Çalışıyor → Hazır dahil).
2. **Senaryo izleme:** "A process reads a file, then its time slice expires... List the states." *İpucu:* her olayı geçiş tablosuyla eşle.
3. **Kavramsal:** "Why is the PCB necessary for context switching?" *İpucu:* yazmaçlar ve program sayacı kaydedilmezse devam edilemez.
4. **Hesap:** "Given the Gantt chart, compute average waiting and turnaround times." *İpucu:* tablo kur; TAT = C − A, W = TAT − B.
5. **Doğru/yanlış:** "A waiting process becomes running immediately after its I/O completes." → Yanlış.
6. **Tanım:** "Define throughput / turnaround time / response time."

**✘ SIK YAPILAN HATA**

- **"I/O bitince süreç hemen çalışır."** → Doğrusu: Hazır kuyruğuna döner. → Neden yanlış: CPU o an başka bir süreçte olabilir; karar zamanlayıcının.
- **"Bekleme süresi = sürecin Bekliyor durumunda geçirdiği süre."** → Doğrusu: Slayt 69'a göre bekleme süresi **hazır kuyruğunda** geçen süredir. → Neden yanlış: Bekliyor (I/O) durumundaki süre zamanlayıcının elinde değildir; zamanlama ölçütü hazır kuyruğunu

ölçer. (İsimlerin benzerliği bu tuzağı doğurur.)
- **TAT'ı "bitiş zamanı" sanmak.** → Doğrusu: TAT = bitiş − **varış**. Varış 0 değilse fark büyük olur.
- **"Bağlam değiştirme süreç işinin bir parçasıdır."** → Doğrusu: Kullanıcı işi yapmayan ek yüktür.
- **Yanıt süresini bitişe göre hesaplamak.** → Doğrusu: Yanıt süresi CPU'yu **ilk** aldığı ana kadardır.

⚡ **HIZLI TEKRAR**

- Program ≠ süreç. Süreç = çalışan program + durumu + kaynakları.
- **PCB** bir sürecin "kimlik kartı ve ayracı"dır.
- Durumlar: **Yeni → Hazır → Çalışıyor → (Bekliyor → Hazır) → Sonlandı**; ayrıca Çalışıyor → Hazır (preempt).
- **Zamanlayıcı** hazır kuyruğundan seçer; hızlı olmalı.
- **Bağlam değiştirme:** kaydet → seç → yükle; ek yüktür.
- **Kesintisiz** (bitene/bırakana kadar) ↔ **kesintili** (zorla alınabilir).
- Ölçütler: kullanım ↑, throughput ↑, TAT ↓, W ↓, R ↓. Formüller: **TAT = C − A, W = TAT − B, R = S − A.**
- Tek bir en iyi algoritma yok; hedefe bağlı.

*Ezberle:* beş durum, slayt 63'teki geçiş olayları, beş ölçütün tanımı ve hedef yönü. *Türetebil:* W = TAT − B formülü, ek yük oranı.
**Bu bölümden sonra şunları yapabiliyor olmalısın:** durum diyagramını eksiksiz çizmek; bir senaryodaki durum dizisini yazmak; bir Gantt şemasından bütün ölçütleri hesaplamak.

## 6.11 Bölüm cevapları

**Durup düşün (not defteri).** **Bir** program (diskteki tek dosya), **iki** süreç (her pencere ayrı bir çalışan örnek; ayrı belleği ve durumu var).

**Durup düşün (slayt 65).** P1 **disk bekleme kuyruğuna** girer (Çalışıyor → Bekliyor). CPU boşalır; FCFS sırasıyla hazır kuyruğunun başındaki **P2** CPU'yu alır.

**Kendin çöz (SJF sırası).** Gantt P1 0–5, P3 5–6, P2 6–9. TAT: P1 = 5 − 0 = 5, P3 = 6 − 2 = 4, P2 = 9 − 1 = 8 → ortalama 17/3 ≈ **5,67**. W: P1 = 5 − 5 = 0, P3 = 4 − 1 = 3, P2 = 8 − 3 = 5 → ortalama 8/3 ≈ **2,67**. FCFS'ye göre (3,33 ve 6,33) **hem bekleme hem tamamlanma süresi iyileşti**, çünkü kısa iş (P3) uzun işin (P2) önüne alındı.

BÖLÜM 7

# CPU zamanlama algoritmaları

◆ **BU BÖLÜMDE** Okuma süresi: 75 dk Zorluk: ●●●

- FCFS, SJF, SRTF, öncelikli zamanlama ve Round Robin'in seçim kurallarını söylemek
- Verilen bir süreç kümesi için her algoritmanın Gantt şemasını çizmek
- Ortalama bekleme, tamamlanma ve yanıt sürelerini hatasız hesaplamak
- Konvoy etkisini, açlığı (starvation) ve yaşlandırmayı (aging) açıklamak
- Round Robin'de kuantum seçiminin etkisini ek yük ve yanıt süresi üzerinden tartışmak
- Bir senaryo için uygun algoritmayı gerekçesiyle seçmek

**SINAV ÖNCELİĞİ: YÜKSEK** ❖ Kaynak: D1 s. 71–87 / slayt 73–88

*Neden yüksek?* Haftanın en hesap ağırlıklı ve en çok örnek içeren kısmı; öğrenme çıktısında "CPU zamanlama algoritmalarını uygulamak" açıkça yazıyor ve slayt 88 doğrudan bir sınav sorusu biçiminde. Bu bölüme en çok zamanı ayır.

→ **BU KONUYA NEDEN GEÇİYORUZ?**

6. bölümde zamanlayıcının hazır kuyruğundan bir süreç seçtiğini ve seçimin beş ölçütü etkilediğini gördük; ama seçimin *nasıl* yapıldığını söylemedik. Bu bölümde dört temel kuralı tanıyacağız. Her biri bir öncekinin bir zayıflığını gidermek için var: FCFS basit ama sıraya fazla duyarlı; SJF bekleme süresini en aza indirir ama geleceği bilmeyi gerektirir; öncelik önemli işleri öne alır ama bazılarını aç bırakabilir; Round Robin herkese adil davranır ama kuantumu iyi seçmek gerekir.

↺ **HATIRLATMA — 6. bölümden gerekenler**

- **A** varış, **B** CPU patlaması (çalışma süresi), **S** ilk başlama, **C** bitiş.
- **TAT = C − A**, **W = TAT − B**, **R = S − A**. Ortalama = toplam / süreç sayısı.
- **Kesintisiz:** süreç bitene (veya I/O'ya girene) kadar CPU'yu bırakmaz. **Kesintili:** CPU zorla alınabilir.
- **Gantt şeması:** yatay zaman çizgisi; her kutu o aralıkta CPU'da hangi sürecin çalıştığını gösterir.

## 7.1 Dört temel yaklaşım

❖ Kaynak: D1 s. 72 / slayt 73

| Algoritma | Seçim kuralı | Güçlü yanı | Ana sorunu |
|---|---|---|---|
| **FCFS** | İlk gelen ilk hizmet alır | Basit kuyruk düzeni | Konvoy etkisi (**convoy effect**) |
| **SJF / SRTF** | En kısa iş / en kısa kalan süre | Düşük ortalama bekleme | Patlama süresini tahmin etmek |
| **Öncelik (priority)** | En yüksek öncelikli | Önemli işlere hızlı hizmet | Açlık (**starvation**) |
| **Round Robin** | Sırayla, zaman kuantumu kadar | İyi etkileşim ve adalet | Kuantum seçimi |

Bu tabloyu bölümün sonunda tekrar dolduracağız; şimdilik her satırın "neden" sorusunu cevaplamak için yola çıkıyoruz.

### 7.1.1 Her zamanlama sorusunda işe yarayan çözüm şablonu

Sınavdaki zamanlama sorularının neredeyse hepsi aynı kalıptadır: bir süreç tablosu (varış, patlama, belki öncelik) ve bir algoritma verilir; Gantt şeması ve ortalamalar istenir. Şu altı adımı uygularsan hata yapma ihtimalin çok düşer:

1. **Soruyu tanı:** Varış zamanları hep 0 mı? Algoritma kesintili mi? Öncelikte küçük sayı mı yüksek? RR'de kuantum kaç?
2. **Karar anlarını belirle:** Kesintisiz algoritmada karar yalnızca bir süreç *bittiğinde* verilir. Kesintili algoritmada ayrıca *her yeni varışta* (SRTF, kesintili öncelik) veya *her kuantum dolduğunda* (RR) karar verilir.
3. **Her karar anında hazır olanları listele:** "t anında kuyrukta kimler var, kalan süreleri ne?" Bunu yazmadan seçim yapma.
4. **Gantt şemasını çiz** ve her kutunun bitiş zamanını altına yaz.
5. **Tablo kur:** A, B, S, C, TAT, W, R sütunları.
6. **Kontrol et:** (a) Son bitiş zamanı = toplam patlama süresi + CPU'nun boş kaldığı süre. (b) Her süreç için $W \geq 0$. (c) Kesintisizde R = W olmalı.

## 7.2 FCFS: ilk gelen ilk hizmet alır

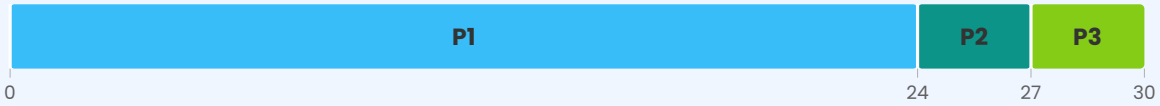
❖ Kaynak: D1 s. 73 / slayt 74

**First-Come, First-Served (FCFS)** en sezgisel kuraldır; kantindeki kuyruğun aynısı:

- **CPU, hazır kuyruğuna ilk giren sürece verilir.**
- **Kesintisizdir:** süreç bitene ya da I/O beklemeye geçene kadar CPU onda kalır.
- **Bir kuyrukla kolayca uygulanır:** gelen sona eklenir, baştaki çalışır.
- **Uzun bir süreç arkasındaki kısa süreçleri geciktirebilir.**

**▶ ÖRNEK 7.1 — Slayt örneği [D1 s. 74 / slayt 75]: sıra P1, P2, P3**

**Verilenler.** Gantt şemasından okunan patlama süreleri: **P1 = 24, P2 = 3, P3 = 3**. Üçü de 0 anında, P1, P2, P3 sırasıyla geliyor. (Slayt patlama sürelerini ayrıca yazmıyor; Gantt'taki aralıklardan çıkarıyoruz.)
**Karar anları.** t = 0: kuyruk P1, P2, P3 → P1 çalışır (0–24). t = 24: P2 (24–27). t = 27: P3 (27–30).

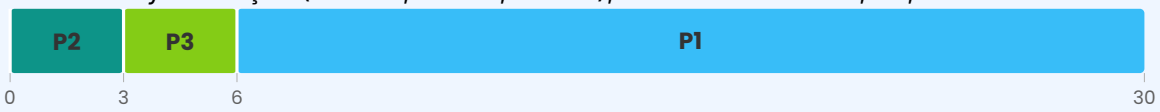



| Süreç | A | B | C | TAT = C − A | W = TAT − B |
|---|---|---|---|---|---|
| P1 | 0 | 24 | 24 | 24 | 0 |
| P2 | 0 | 3 | 27 | 27 | 24 |
| P3 | 0 | 3 | 30 | 30 | 27 |

**Sonuç.** Ortalama bekleme = (0 + 24 + 27) / 3 = **17**. Ortalama tamamlanma = (24 + 27 + 30) / 3 = **27**. Slaytla aynı.
**Gözlem:** Toplam 30 birimlik işin 24'ü P1'e ait; iki kısa iş (3'er birim) P1'i bekleyerek 24 ve 27 birim kaybetti.

**▶ ÖRNEK 7.2 — Slayt örneği [D1 s. 75 / slayt 76]: sıra P2, P3, P1**

**Verilenler.** Aynı süreçler (P1 = 24, P2 = 3, P3 = 3), ama bu kez sıra P2, P3, P1.




**Çözüm.** Beklemeler: P2 = 0, P3 = 3, P1 = 6. Ortalama bekleme = (0 + 3 + 6) / 3 = **3**. Tamamlanma süreleri (slaytta yok): P2 = 3, P3 = 6, P1 = 30 → ortalama (3 + 6 + 30) / 3 = **13**.
**Sonuç.** Yalnızca sıra değişti, ortalama bekleme **17'den 3'e** düştü. Slaytın yorumu: *Kısa işler önce geldiğinde sonuç ciddi biçimde iyileşti.* FCFS'nin kendisi değişmedi; değişen tek şey varış sırası. FCFS'nin en büyük zayıflığı bu: **performansı tamamen şansa (varış sırasına) bağlı.**

✔ Python zamanlama simülatörüyle doğrulandı

## 7.2.1 Konvoy etkisi

❖ Kaynak: D1 s. 76 / slayt 77

**∑ TANIM / FORMÜL — Konvoy etkisi (convoy effect)**

Tek bir uzun, CPU'ya bağlı sürecin, arkasındaki birçok kısa sürecin beklemesine yol açması. *"One long CPU-bound process makes many short processes wait behind it."*

Tek şeritli bir yolda en önde giden traktörü düşün; arkasında spor arabalar sıralanmış, hepsi traktörün hızıyla ilerliyor. Slayt üç sonuç sayıyor:

1. **Ortalama bekleme ve yanıt süresi artar.** (17'ye karşı 3 örneğini gördük.)
2. **I/O aygıtları ile CPU sırayla boşta kalabilir.** Uzun süreç CPU'dayken, kısa ve I/O'ya bağlı süreçler hazır kuyruğunda bekler; I/O aygıtları boştadır. Uzun süreç I/O'ya geçtiğinde kısa süreçler CPU'yu hızla kullanıp hepsi I/O kuyruğuna yığılır; bu kez CPU boşta kalır. Kaynaklar sırayla ziyan edilir.
3. **FCFS etkileşimli sistemlerde kötü bir kullanıcı deneyimi yaratabilir.** Bir tuşa bastın ama uzun bir hesap bitene kadar ekran tepki vermiyor.

Slaytın özeti: *Algoritma basit ama varış sırasına aşırı duyarlı.*

## 7.3 SJF: en kısa iş önce

❖ Kaynak: D1 s. 77 / slayt 78

Konvoy etkisinin çaresi açık: kısa işleri öne alalım. **Shortest Job First (SJF)**:

- **Bir sonraki CPU patlaması en kısa olan hazır süreci seçer.**
- **Eşitlik durumunda FCFS kullanılabilir** (önce gelen önce).
- **Kesintisiz SJF, çalışan patlamayı yarıda kesmez.** Daha kısa bir iş gelse bile mevcut iş biter.
- **Bir sonraki CPU patlamasının uzunluğu bilinmeli ya da tahmin edilmelidir.**

Slaytın vurgusu: *SJF, verilen bir iş kümesi için ortalama bekleme süresini en aza indirebilir.*

### 7.3.1 Neden en kısa iş önce bekleme süresini en aza indirir?

Bunu kendin türetebilirsin. Hepsi 0 anında gelen n iş, $b_1$, $b_2$, ..., $b_n$ sırasıyla çalışsın. Birinci iş 0 bekler, ikinci iş $b_1$ bekler, üçüncü iş $b_1 + b_2$ bekler, ... Toplam bekleme:

$W_{toplam} = 0 + b_1 + (b_1 + b_2) + \ldots = (n-1)\cdot b_1 + (n-2)\cdot b_2 + \ldots + 1\cdot b_{n-1} + 0\cdot b_n$

Yani **önde çalışan işin süresi, arkasındaki herkesin beklemesine eklenir**; birinci iş n − 1 kez, ikinci iş n − 2 kez sayılır. Toplamı küçültmek için en büyük katsayının yanına en küçük sayıyı koymalısın: **en kısa iş en öne.** Bir başka deyişle, yan yana iki işin yerini değiştirdiğinde, kısa olan öne geçerse toplam bekleme her zaman azalır. Bu yüzden SJF'den daha düşük ortalama bekleme veren başka bir (kesintisiz) sıra yoktur.

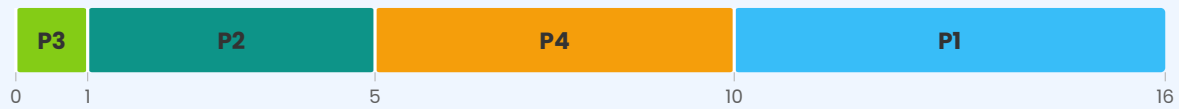
FCFS örneğiyle kontrol: sıra 24, 3, 3 → $W_{toplam}$ = 2·24 + 1·3 = 51 → ortalama 17. Sıra 3, 3, 24 → 2·3 + 1·3 = 9 → ortalama 3. Tutuyor.

**+ EK BİLGİ (KAYNAK DIŞI) — Patlama süresi nasıl tahmin edilir?**

SJF'nin pratik problemi: işletim sistemi bir sürecin bir sonraki CPU patlamasının ne kadar süreceğini önceden bilemez. Ders kitaplarındaki yaygın çözüm, geçmiş patlamaların **üstel ortalamasını** almaktır: tahmin_yeni = α · (son gerçek patlama) + (1 − α) · tahmin_eski, burada 0 ≤ α ≤ 1. Örneğin α = 0,5, eski tahmin 10, son gerçek patlama 6 ise yeni tahmin = 0,5·6 + 0,5·10 = **8**. Slayt yalnızca "bilinmeli ya da tahmin edilmeli" diyor; formül kaynak dışıdır.

**▶ ÖRNEK 7.3 — Slayt örneği [D1 s. 78 / slayt 79]: eşzamanlı varış**

**Verilenler.** Dört süreç de 0 anında hazır. Gantt'tan okunan patlamalar: **P1 = 6, P2 = 4, P3 = 1, P4 = 5.**
**Yöntem.** Hepsi aynı anda hazır ve kesintisiz → patlamaları küçükten büyüğe sırala: P3 (1), P2 (4), P4 (5), P1 (6).




| Süreç | B | S | C | TAT | W |
|---|---|---|---|---|---|
| P3 | 1 | 0 | 1 | 1 | 0 |
| P2 | 4 | 1 | 5 | 5 | 1 |
| P4 | 5 | 5 | 10 | 10 | 5 |
| P1 | 6 | 10 | 16 | 16 | 10 |

**Sonuç.** Ortalama bekleme = (0 + 1 + 5 + 10) / 4 = **4**; ortalama tamamlanma = (1 + 5 + 10 + 16) / 4 = **8**. Slaytla aynı.
**Kontrol:** Varışlar 0 olduğu için her sürecin W'si = başlama zamanı ve TAT'ı = bitiş zamanı.

**▶ ÖRNEK 7.4 — Slayt örneği [D1 s. 79 / slayt 80]: farklı varış zamanları**

**Verilenler.** Varışlar: **P1 = 0, P2 = 2, P3 = 4, P4 = 5**. Gantt'tan okunan patlamalar: **P1 = 7, P2 = 4, P3 = 1, P4 = 4**.
**Karar anları (kesintisiz → yalnızca bir süreç bitince karar verilir):**

- **t = 0:** Hazırda yalnızca P1 var → P1 çalışır. Kesintisiz olduğu için P2, P3, P4 gelse bile **P1 bitene kadar (t = 7) devam eder.**
- **t = 7:** Hazır: P2 (4), P3 (1), P4 (4). En kısa → **P3** (7–8).
- **t = 8:** Hazır: P2 (4), P4 (4). **Eşitlik!** FCFS ile çöz: P2 önce gelmişti (2 < 5) → **P2** (8–12).
- **t = 12:** Hazır: P4 → **P4** (12–16).




| Süreç | A | B | S | C | TAT = C − A | W = TAT − B |
|---|---|---|---|---|---|---|
| P1 | 0 | 7 | 0 | 7 | 7 | 0 |
| P2 | 2 | 4 | 8 | 12 | 10 | 6 |
| P3 | 4 | 1 | 7 | 8 | 4 | 3 |
| P4 | 5 | 4 | 12 | 16 | 11 | 7 |

**Sonuç.** Ortalama bekleme = (0 + 6 + 3 + 7) / 4 = 16/4 = **4** (slaytla aynı). Ortalama tamamlanma (slaytta yok) = (7 + 10 + 4 + 11) / 4 = 32/4 = **8**.
**Bu tip soruda en sık hata:** P2, P3, P4'ün geldiği anlarda P1'i durdurmak. SJF **kesintisizdir**; çalışan patlama kesilmez.

## 7.4 SRTF: en kısa kalan süre önce

❖ Kaynak: D1 s. 80 / slayt 81

**Shortest Remaining Time First (SRTF)**, SJF'nin **kesintili** hâlidir. Kural: *Yeni gelen sürecin kalan süresi, çalışan sürecin kalan süresinden kısaysa çalışan süreç kesilir.* Karar yalnızca bir süreç bittiğinde değil, **her yeni varışta** da verilir.

**▶ ÖRNEK 7.5 — Slayt örneği [D1 s. 80 / slayt 81]: aynı süreçler, SRTF**

**Verilenler.** Önceki örnekle aynı: P1 (0, 7), P2 (2, 4), P3 (4, 1), P4 (5, 4).
**Karar anları** (kalan süreler parantez içinde):

- **t = 0:** Yalnız P1 (7) → P1 çalışır.
- **t = 2:** P2 geldi (4). P1'in kalanı 7 − 2 = **5**. 4 < 5 → **P1 kesilir, P2 çalışır.**
- **t = 4:** P3 geldi (1). P2'nin kalanı 4 − 2 = **2**. 1 < 2 → **P2 kesilir, P3 çalışır.**
- **t = 5:** P3 bitti; aynı anda P4 geldi (4). Hazır: P1 (5), P2 (2), P4 (4). En kısa → **P2** (5–7).
- **t = 7:** P2 bitti. Hazır: P1 (5), P4 (4) → **P4** (7–11).
- **t = 11:** P4 bitti → **P1** (11–16).




| Süreç | A | B | S (ilk) | C | TAT = C − A | W = TAT − B | R = S − A |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 7 | 0 | 16 | 16 | 9 | 0 |
| P2 | 2 | 4 | 2 | 7 | 5 | 1 | 0 |
| P3 | 4 | 1 | 4 | 5 | 1 | 0 | 0 |
| P4 | 5 | 4 | 7 | 11 | 6 | 2 | 2 |

**Sonuç.** Ortalama bekleme = (9 + 1 + 0 + 2) / 4 = 12/4 = **3**; ortalama tamamlanma = (16 + 5 + 1 + 6) / 4 = 28/4 = **7**. Slaytla aynı. Ortalama yanıt süresi (slaytta yok) = (0 + 0 + 0 + 2) / 4 = **0,5**.
**Karşılaştırma:** Kesintisiz SJF'de ortalama bekleme 4 idi, SRTF'de 3. Kesme sayesinde kısa işler uzun işin bitmesini beklemedi. Bedeli: P1 iki kez parçalandı (bağlam değiştirme ek yükü) ve en son bitti.
**P1'in beklemesini Gantt'tan kontrol:** P1, 2–11 arasında hazır kuyruğunda bekledi → 11 − 2 = 9. Formülle de 16 − 0 − 7 = 9. Tutuyor.

**⚠ SINAVDA DİKKAT — Slayt 81'deki şekil**

Slayt 81'deki Gantt şemasında "7" ve "11" etiketleri çubuk sınırlarının tam altında değil, biraz sola kaymış görünüyor. Hesapla doğrulandığı üzere doğru sınırlar **P2: 5–7, P4: 7–11, P1: 11–16**. Kendi şemanı çizerken etiketleri sınırların altına koy.

✔ SJF ve SRTF örnekleri Python simülatörüyle doğrulandı

## 7.5 Öncelikli zamanlama

❖ Kaynak: D1 s. 81 / slayt 82

Bazen işlerin önemi farklıdır: bir hastane sisteminde acil alarm, rapor yazdırmaktan önce gelmeli. **Öncelikli zamanlama (priority scheduling):**

- **Her sürece bir öncelik değeri verilir; en yüksek öncelikli süreç seçilir.**
- **Kesintili ya da kesintisiz olabilir.** Kesintilide daha yüksek öncelikli bir süreç gelince çalışan süreç kesilir.
- **Öncelik kullanıcıya, iş türüne, son teslim süresine veya sistem politikasına dayanabilir.**
- **Bazı sistemlerde küçük sayı yüksek öncelik demektir; bu kural açıkça belirtilmelidir.**

Son madde sınav için çok önemli: soruda "1 en yüksek öncelik" ya da "küçük sayı yüksek öncelik" yazmıyorsa varsayımını cevabının başına yaz.

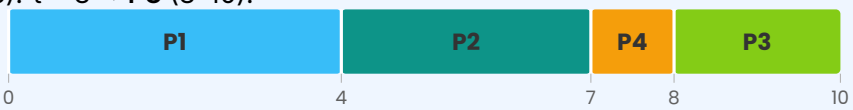
SJF'yi de bir öncelik algoritması gibi düşünebilirsin: öncelik = patlama süresi, küçük olan önce.

**▶ ÖRNEK 7.6 — Ek örnek: kesintisiz ve kesintili öncelik**

**Verilenler** (küçük sayı = yüksek öncelik):

| | P1 | P2 | P3 | P4 |
|---|---|---|---|---|
| Varış | 0 | 1 | 2 | 3 |
| Patlama | 4 | 3 | 2 | 1 |
| Öncelik | 3 | 1 | 4 | 2 |

*Kesintisiz:* t = 0 yalnız P1 → P1 (0–4). t = 4: P2 (1), P3 (4), P4 (2) → **P2** (4–7). t = 7: P3 (4), P4 (2) → **P4** (7–8). t = 8 → **P3** (8–10).




W: P1 = 0, P2 = 7 − 1 − 3 = 3, P3 = 10 − 2 − 2 = 6, P4 = 8 − 3 − 1 = 4 → ortalama 13/4 = **3,25**.
*Kesintili:* t = 0 P1. t = 1: P2 geldi, önceliği 1 < 3 → **P1 kesilir**, P2 (1–4). t = 2: P3 (4) gelir, P2'den düşük → devam. t = 3: P4 (2) gelir, P2'den (1) düşük → devam. t = 4: P2 biter; hazır P1 (3, kalan 3), P3 (4), P4 (2) → **P4** (4–5). t = 5: P1 (5–8). t = 8: P3 (8–10).




W: P1 = 8 − 0 − 4 = 4, P2 = 0, P3 = 6, P4 = 5 − 3 − 1 = 1 → ortalama 11/4 = **2,75**.
**Gözlem:** Her iki durumda da en düşük öncelikli P3 en son çalıştı. Sistemde sürekli yüksek öncelikli işler gelseydi P3 hiç çalışamayabilirdi; bu da bizi açlığa getiriyor.

✔ Python simülatörüyle doğrulandı

### 7.5.1 Açlık ve yaşlandırma

❖ Kaynak: D1 s. 82 / slayt 83

| | Açlık (starvation) | Yaşlandırma (aging) |
|---|---|---|
| Nedir? | Düşük öncelikli bir süreç **süresiz olarak ertelenebilir** | Bekleyen bir sürecin önceliği **zamanla kademeli olarak yükseltilir** |
| Not | Sistem yükü arttıkça risk büyür | Süresiz ertelemeyi sınırlar |

Açlık, bir restoranda rezervasyonu olmayan müşterinin, rezervasyonlu müşteriler sürekli geldiği için hiç masa bulamamasıdır. Yaşlandırma ise "her 15 dakikada bekleyen müşterinin sırası bir basamak öne alınır" kuralıdır; sonunda en uzun bekleyen müşteri herkesi geçer.

*Sayılarla bir fikir (ek):* En düşük öncelik 20 olsun ve bekleyen süreçlerin öncelik sayısı her 2 dakikada 1 azaltılsın (küçük = yüksek). Öncelik 20'deki bir süreç en geç 40 dakika sonra en yüksek önceliğe (0) ulaşır ve mutlaka çalışır. Yani yaşlandırma beklemeye bir **üst sınır** koyar.

## 7.6 Round Robin: sırayla, dilim dilim

❖ Kaynak: D1 s. 83 / slayt 84

Zaman paylaşımının (3. bölüm) zamanlama algoritması **Round Robin (RR)**:

- **Her süreç en fazla bir zaman kuantumu (time quantum) kadar çalışır.**
- **Bitmeyen süreç hazır kuyruğunun sonuna eklenir.**
- **n süreç ve q kuantumla, bir süreç kabaca (n − 1)·q süre sonra tekrar CPU alır.**
- **Etkileşimli sistemler için adil ve yanıt veren bir yaklaşımdır.**

(n − 1)·q nereden geliyor? Kuyrukta sen dahil n süreç var. CPU'dan çıktığında sana sıra tekrar gelene kadar senden önceki diğer n − 1 sürecin her biri en fazla q kadar çalışır. Örneğin 5 süreç ve q = 20 ms ise en kötü durumda **4 × 20 = 80 ms** sonra tekrar CPU'yu alırsın. Bu, RR'nin **yanıt süresine bir üst sınır** koyması demektir; FCFS'de böyle bir garanti yoktur (24 birimlik P1 örneğini hatırla).

**⚠ SINAVDA DİKKAT — Kuyruk sırası kuralı**

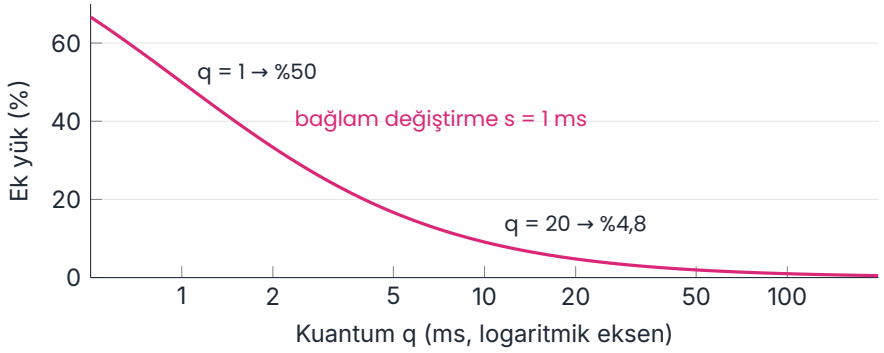
Bir sürecin kuantumu dolduğu anda yeni bir süreç de gelmişse, kuyrukta hangisi önde? Yaygın kural (bu kitaptaki bütün RR çözümlerinde kullanılan): **önce yeni gelen kuyruğa girer, sonra kuantumu dolan süreç sona eklenir.** Slayt bu ayrıntıyı belirtmiyor; sınavda farklı bir kural varsayarsan cevabının başına yaz.

### 7.6.1 Kuantumu nasıl seçmeli?

❖ Kaynak: D1 s. 84 / slayt 85

| | Kuantum çok büyük | Kuantum çok küçük |
|---|---|---|
| Ne olur? | Davranış **FCFS'ye yaklaşır** | **Bağlam değiştirme sayısı artar** |
| Sonucu | Yanıt süresi kötüleşebilir | Ek yük throughput'u düşürebilir |

Neden büyük kuantum FCFS'ye dönüşür? Kuantum her süreçten uzunsa hiçbir süreç kesilmez; her biri sırası gelince sonuna kadar çalışır, yani FCFS. Neden küçük kuantum kötü? 6. bölümdeki hesabı hatırla: ek yük oranı = s / (q + s).



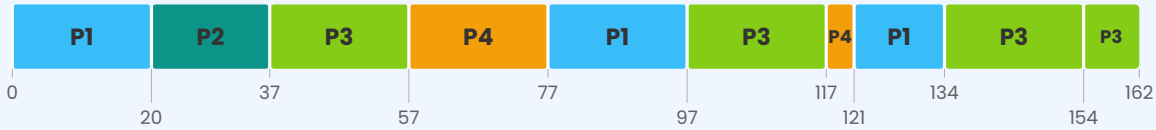

**Şekil 7.1:** Bağlam değiştirme 1 ms sürüyorsa, CPU zamanının ne kadarının ek yüke gittiği: s / (q + s). Kuantum küçüldükçe ek yük hızla artar (ek hesap, slaytta yok).

Slaytın kuralı: *Kuantum, bağlam değiştirme süresinden anlamlı ölçüde büyük olmalıdır.* Ama çok da büyük olmamalı ki yanıt süresi FCFS gibi kötüleşmesin. İyi kuantum bu iki uç arasındaki ödünleşimdir.

**▶ ÖRNEK 7.7 — Slayt örneği [D1 s. 85 / slayt 86]: Round Robin, q = 20**

**Verilenler.** Patlamalar: **P1 = 53, P2 = 17, P3 = 68, P4 = 24**; hepsi 0 anında, P1, P2, P3, P4 sırasıyla geliyor. q = 20.
**İzleme tablosu** (her satır bir kuantum; "kalan" çalıştıktan sonraki süredir):

| Aralık | Çalışan | Ne kadar? | Kalan | Sonra kuyruk |
|---|---|---|---|---|
| 0–20 | P1 | 20 | 53 → 33 | P2, P3, P4, P1 |
| 20–37 | P2 | 17 (bitti) | 0 | P3, P4, P1 |
| 37–57 | P3 | 20 | 68 → 48 | P4, P1, P3 |
| 57–77 | P4 | 20 | 24 → 4 | P1, P3, P4 |
| 77–97 | P1 | 20 | 33 → 13 | P3, P4, P1 |
| 97–117 | P3 | 20 | 48 → 28 | P4, P1, P3 |
| 117–121 | P4 | 4 (bitti) | 0 | P1, P3 |
| 121–134 | P1 | 13 (bitti) | 0 | P3 |
| 134–154 | P3 | 20 | 28 → 8 | P3 |
| 154–162 | P3 | 8 (bitti) | 0 | — |




Ortalamaları hesaplayalım (slayt bunları vermiyor, yalnızca “yanıt süresi iyi; çok sayıda bağlam değiştirme olur” diyor):

| Süreç | B | S (ilk) | C | TAT = C − 0 | W = TAT − B | R = S |
|---|---|---|---|---|---|---|
| P1 | 53 | 0 | 134 | 134 | 81 | 0 |
| P2 | 17 | 20 | 37 | 37 | 20 | 20 |
| P3 | 68 | 37 | 162 | 162 | 94 | 37 |
| P4 | 24 | 57 | 121 | 121 | 97 | 57 |

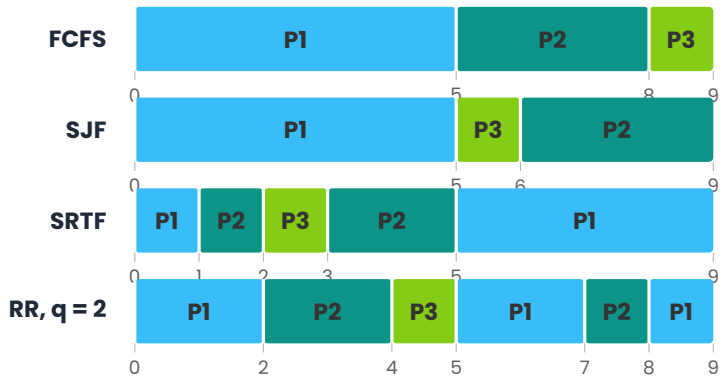
Ortalama bekleme = (81 + 20 + 94 + 97) / 4 = 292/4 = **73**. Ortalama tamamlanma = (134 + 37 + 162 + 121) / 4 = 454/4 = **113,5**. Ortalama yanıt = (0 + 20 + 37 + 57) / 4 = 114/4 = **28,5**.

✔ Python simülatörüyle doğrulandı

**Yorumlar.** (1) Yanıt süresi iyi: her süreç en geç 57. birimde ilk kez CPU’yu aldı. Aynı süreçler FCFS ile çalışsaydı P4 ilk kez 53 + 17 + 68 = 138’de çalışacaktı. (2) Bekleme süresi ise yüksek; RR ortalama beklemeyi değil, **adaleti ve yanıt süresini** iyileştirir. (3) Farklı süreçler arasında 8 bağlam değiştirme oldu. 134–154 ve 154–162 dilimleri arasında süreç değişmez: P3 kuyrukta yalnız kaldığı için kuantum dolunca yine kendisi seçilir. Slayt bunu iki ayrı kutu olarak çizmiş; tek bir P3 kutusu (134–162) olarak çizmek de doğrudur.

## 7.7 Aynı süreçler, dört algoritma

Algoritmaları yan yana koyunca güçlü ve zayıf yanları netleşir. 6. bölümdeki küme: **P1 (varış 0, patlama 5), P2 (1, 3), P3 (2, 1)**. RR için q = 2.




Şekil 7.2: Aynı üç süreç dört algoritmayla. RR'de t = 2'de P3 geldiği an P1'in kuantumu da doluyor; kurala göre önce P3 kuyruğa girer, sonra P1: kuyruk P2, P3, P1.

| Algoritma | Ort. bekleme (W) | Ort. tamamlanma (TAT) | Ort. yanıt (R) |
|---|---|---|---|
| FCFS | 3,33 | 6,33 | 3,33 |
| SJF | 2,67 | 5,67 | 2,67 |
| SRTF | **1,67** | **4,67** | **0** |
| RR (q = 2) | 3,33 | 6,33 | 1 |

✔ Dört algoritma Python simülatörüyle çalıştırıldı

Okuması: **SRTF** bekleme ve tamamlanmada en iyisi (kısa işleri hiç bekletmiyor); **RR** ortalama beklemede FCFS'den iyi değil ama **yanıt süresinde** FCFS ve SJF'den çok daha iyi; **SJF** kesintisizler içinde en düşük beklemeyi veriyor. 6. bölümün dersi burada sayılarla görünüyor: tek bir "en iyi" yok, ölçüte göre değişiyor.

## 7.8 Hangi algoritma ne zaman?

❖ Kaynak: D1 s. 86 / slayt 87

| Algoritma | Uygun ortam | Avantajı | Dikkat edilecek |
|---|---|---|---|
| **FCFS** | Basit toplu iş kuyruğu | Düşük uygulama maliyeti | Konvoy etkisi |
| **SJF / SRTF** | Patlama tahmini mümkünse | Düşük ortalama bekleme | Uzun işlerin gecikmesi |
| **Öncelik** | İşlerin önemi farklıysa | Kritik işlere hızlı hizmet | Açlık |
| **Round Robin** | Etkileşimli sistem | Adalet ve hızlı ilk yanıt | Kuantum ve ek yük |

Dikkat: SJF/SRTF satırında "dikkat edilecek" sütunu slayt 73'teki "burst prediction"dan farklı olarak **"uzun işlerin gecikmesi"** diyor. İkisi de doğru: SJF'nin hem tahmin problemi hem de uzun işleri sürekli geride bırakma (bir tür açlık) riski vardır.

★ **MERAKLISINA (SINAV DIŞI)**

Gerçek işletim sistemleri bu dört algoritmanın saf hâllerini değil, karışımlarını kullanır. Örneğin Windows çok seviyeli, kesintili, öncelik tabanlı bir zamanlayıcı kullanır; Linux uzun yıllar "Completely Fair Scheduler (CFS)" adlı, her sürece CPU'dan "adil pay" vermeye çalışan bir

zamanlayıcı kullandı ve 2023'te onun yerini EEVDF adlı yeni bir algoritma aldı. Hepsinin içinde bu bölümdeki fikirleri bulursun: öncelik, adalet, kesme ve zaman dilimi.

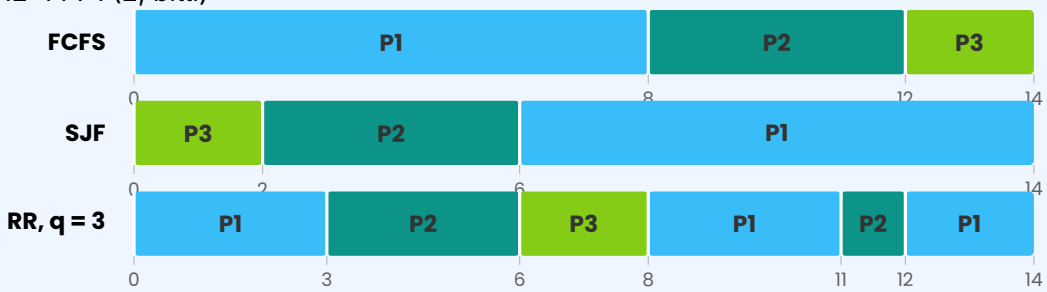
## 7.9 Slayt alıştırması: algoritmayı seç

❖ Kaynak: D1 s. 87 / slayt 88

**▶ ÖRNEK 7.8 — Slayt örneği [D1 s. 87 / slayt 88]: P1 = 8, P2 = 4, P3 = 2**

**Soru.** P1 = 8, P2 = 4, P3 = 2; hepsi t = 0'da geliyor. FCFS ve SJF çizelgelerini çiz ve ortalama bekleme sürelerini karşılaştır. *Uzantı:* q = 3 için Round Robin Gantt şeması nasıl olur?
**FCFS** (slayttaki sıra P1 → P2 → P3): P1 0–8, P2 8–12, P3 12–14. Beklemeler: 0, 8, 12 → ortalama 20/3 ≈ **6,67**. Tamamlanma: 8, 12, 14 → ortalama 34/3 ≈ **11,33**.
**SJF** (slayttaki sıra P3 → P2 → P1): P3 0–2, P2 2–6, P1 6–14. Beklemeler: P3 = 0, P2 = 2, P1 = 6 → ortalama 8/3 ≈ **2,67**. Tamamlanma: 2, 6, 14 → ortalama 22/3 ≈ **7,33**.
**Round Robin, q = 3:** kuyruk P1, P2, P3.

- 0–3 P1 (kalan 5) → kuyruk P2, P3, P1
- 3–6 P2 (kalan 1) → kuyruk P3, P1, P2
- 6–8 P3 (2, bitti) → kuyruk P1, P2
- 8–11 P1 (kalan 2) → kuyruk P2, P1
- 11–12 P2 (1, bitti) → kuyruk P1
- 12–14 P1 (2, bitti)




RR beklemeleri (W = C − B): P1 = 14 − 8 = 6, P2 = 12 − 4 = 8, P3 = 8 − 2 = 6 → ortalama 20/3 ≈ **6,67**. Yanıt süreleri: 0, 3, 6 → ortalama **3**.
**Karşılaştırma.** SJF ortalama beklemede açık ara en iyisi (2,67). RR'nin ortalama beklemesi bu örnekte FCFS ile aynı (6,67) ama ilk yanıt süresi daha iyi (FCFS'de ortalama yanıt 6,67, RR'de 3). Sınav cevabını şöyle bitir: "Toplu işlemde ortalama bekleme önemliyse SJF, etkileşimli bir sistemde ilk yanıt önemliyse RR tercih edilir."

✓ Python simülatörüyle doğrulandı

**▶ ÖRNEK 7.9 — Yarı çözümlü: SRTF**

Süreçler: P1 (varış 0, patlama 8), P2 (1, 4), P3 (2, 9), P4 (3, 5).

- t = 0: yalnız P1 → P1 çalışır.
- t = 1: P2 geldi (4). P1'in kalanı 7. 4 < 7 → P1 kesilir, P2 çalışır.
- t = 2: P3 geldi (9). P2'nin kalanı 3 → P2 devam.
- t = 3: P4 geldi (5). P2'nin kalanı 2 → P2 devam.
- t = 5: P2 bitti. Hazır: P1 (7), P3 (9), P4 (5) → ..... çalışır.
- t = ..... : o da biter. Hazır: ..... → ..... çalışır.

- Sonra kalan süreç çalışır ve t = 26'da her şey biter.

Gantt şemasını tamamla ve ortalama beklemeyi bul. *Cevap bölüm sonunda.*

**✎ KENDİN ÇÖZ**

Yarı çözümlü örnekteki aynı dört süreç (P1: 0/8, P2: 1/4, P3: 2/9, P4: 3/5) için **kesintisiz SJF** Gantt şemasını çiz ve ortalama bekleme süresini hesapla. SRTF ile karşılaştır: fark nereden geliyor?

## 7.10 Bu bölümün bağlantıları

- Konvoy etkisi → 3. ve 6. bölümdeki CPU'ya bağlı / I/O'ya bağlı süreçler.
- Round Robin → 3. bölümdeki zaman paylaşımı ve 4. bölümdeki çoklu görev ve preemption.
- Kuantum seçimi → 6. bölümdeki bağlam değiştirme ek yükü.
- Öncelik ve deadline → 2. bölümdeki gerçek zamanlı sistemler.
- "En iyi algoritma yok" → 5. bölümdeki yerleştirme algoritmalarında aynı sonuç ve 1. bölümdeki ödünleşim.

**⚠ SINAVDA DİKKAT**

- **FCFS:** kesintisiz, varış sırasına duyarlı, konvoy etkisi.
- **SJF:** kesintisiz, en kısa patlama önce; eşitlikte FCFS; ortalama beklemeyi en aza indirir (eşzamanlı varışta kanıtlanabilir); patlama tahmini gerekir.
- **SRTF:** kesintili SJF; **her varışta** kalan süreleri karşılaştır.
- **Öncelik:** küçük sayı = yüksek mi? **Kuralı yaz.** Sorun: **açlık**; çözüm: **yaşlandırma**.
- **RR:** en fazla q kadar çalış, bitmezse kuyruğun sonuna; en fazla (n − 1)·q bekleme; q büyük → FCFS, q küçük → ek yük.
- Kesintisizde **R = W**; kesintilide genellikle R < W.
- Her soruda: karar anları → hazır listesi → Gantt → tablo → kontrol.

Kolay karıştırılanlar:

| | SJF | SRTF |
|---|---|---|
| Kesinti | Yok | Var |
| Karar anı | Yalnızca bir süreç bitince | Bitince **ve her yeni varışta** |
| Neye bakar? | Toplam (sonraki) patlama | **Kalan** süre |
| Eşzamanlı varışta | İkisi aynı sonucu verir | İkisi aynı sonucu verir |

**❖ BU KONUDAN NASIL SORU GELİR?**

1. **Hesap (en olası):** "Given the following processes (arrival, burst), draw the Gantt chart for FCFS / SJF / SRTF / RR (q = ...) and compute the average waiting and turnaround times." *İpucu:* şablonun altı adımı; RR'de kuyruk sırası kuralını yaz.
2. **Karşılaştırma:** "Which algorithm gives the lowest average waiting time for this set? Which gives the best response time?" *İpucu:* çoğu zaman SRTF/SJF bekleme, RR yanıt.
3. **Kavramsal:** "What is the convoy effect? Which algorithm suffers from it?" → FCFS.

4. **Kavramsal:** "What is starvation and how does aging solve it?"
5. **Analiz:** "What happens to Round Robin if the quantum is very large / very small?" *İpucu:* FCFS / ek yük; s / (q + s) hesabını yaz.
6. **Senaryo:** "Which scheduling algorithm would you choose for an interactive system / a batch system / a system with critical jobs?" *İpucu:* slayt 87 tablosu.
7. **Doğru/yanlış:** "SJF is preemptive." → Yanlış (SRTF kesintili sürümüdür).

**✘ SIK YAPILAN HATA**

- **Kesintisiz SJF'de yeni gelen kısa işle çalışan işi kesmek.** → Doğrusu: Kesintisizde çalışan iş bitene kadar sürer. → Neden yanlış: Bu SRTF olur.
- **SRTF'de karşılaştırmayı toplam patlamayla yapmak.** → Doğrusu: Çalışan sürecin **kalan** süresiyle karşılaştır.
- **RR'de bitmeyen süreci kuyruğun başına koymak.** → Doğrusu: Kuyruğun **sonuna** eklenir.
- **Varış zamanını unutmak:** TAT'ı bitiş zamanı olarak yazmak. → Doğrusu: TAT = C − A.
- **Kesilen sürecin beklemesini yalnızca ilk beklemesiyle hesaplamak.** → Doğrusu: W = TAT − B; bütün bekleme parçalarını otomatik toplar.
- **Öncelikte "büyük sayı yüksek öncelik" varsayıp söylememek.** → Doğrusu: Varsayımını açıkça yaz; slayt 82 bunu özellikle uyarıyor.
- **"RR her zaman en düşük ortalama beklemeyi verir."** → Doğrusu: RR adalet ve yanıt süresi için iyidir; ortalama beklemesi çoğu zaman SJF'den kötüdür (slayt 86'da 73!).

**⚡ HIZLI TEKRAR**

- **FCFS** → basit, konvoy etkisi (17'ye karşı 3).
- **SJF** → en kısa önce, kesintisiz, ortalama beklemenin en küçüğü; tahmin gerekir.
- **SRTF** → kesintili SJF, her varışta kalan süreleri karşılaştır (slayt 81: W = 3, TAT = 7).
- **Öncelik** → kural açık olmalı; **açlık ↔ yaşlandırma**.
- **RR** → q kadar çalış, kuyruğun sonuna; (n − 1)·q; q büyük → FCFS, küçük → ek yük (slayt 86: W = 73, R = 28,5).
- Formüller: TAT = C − A, W = TAT − B, R = S − A.
- Seçim: toplu iş → FCFS/SJF; tahmin varsa → SJF/SRTF; önem farkı → öncelik; etkileşim → RR.

*Ezberle:* her algoritmanın kuralı ve ana sorunu (slayt 73). *Türetebil:* bütün Gantt şemaları ve ortalamalar, SJF'nin neden en iyi olduğu, ek yük oranı.
**Bu bölümden sonra şunları yapabiliyor olmalısın:** varışlı/varışsız her süreç kümesi için beş algoritmayla Gantt çizip ortalamaları hesaplamak; sonuçları yorumlayıp bir senaryo için algoritma önermek.

## 7.11 Bölüm cevapları

**Yarı çözümlü (SRTF).** t = 5: en kısa kalan **P4** (5) çalışır (5–10). t = **10**: P4 biter; hazır **P1 (7), P3 (9)** → **P1** çalışır (10–17). t = 17: P3 (17–26). Gantt: P1 0–1, P2 1–5, P4 5–10, P1 10–17, P3 17–26. Beklemeler (W = C − A − B): P1 = 17 − 0 − 8 = 9, P2 = 5 − 1 − 4 = 0, P3 = 26 − 2 − 9 = 15, P4 = 10 − 3 − 5 = 2 → ortalama 26/4 = **6,5**.

**Kendin çöz (kesintisiz SJF).** t = 0: yalnız P1 → P1 0–8 (kesilmez). t = 8: hazır P2 (4), P3 (9), P4 (5) → P2 8–12. t = 12: P3 (9), P4 (5) → P4 12–17. t = 17: P3 17–26. Beklemeler: P1 = 0, P2 = 12 − 1 − 4 =

7, P4 = 17 − 3 − 5 = 9, P3 = 26 − 2 − 9 = 15 → ortalama 31/4 = **7,75**. **Karşılaştırma:** SRTF (6,5) daha iyi. Fark, SJF'de P1'in 8 birim boyunca kesilmeden çalışmasından geliyor: 1. birimde gelen kısa P2 kesintisiz algoritmada 7 birim bekledi; SRTF'de ise hiç beklemedi.

✔ Bölümdeki bütün sayısal sonuçlar dogrulama/zamanla.py ile çalıştırıldı

BÖLÜM 8

# Bellek hiyerarşisi

◆ **BU BÖLÜMDE** Okuma süresi: 30 dk Zorluk: ●●○

- CPU ile ana bellek arasındaki hız farkının neden bir problem olduğunu açıklamak
- Bellek hiyerarşisinin katmanlarını hız, kapasite ve maliyet açısından sıralamak
- Önbellek isabeti (cache hit) ve ıskasını (cache miss) tanımlamak
- Zamansal ve uzamsal yerelliği örneklerle ayırt etmek ve önbelleğin neden işe yaradığını yerellikle açıklamak

**SINAV ÖNCELİĞİ: ORTA** ❖ Kaynak: D1 s. 88–94 / slayt 89–95

*Neden orta?* Hesap gerektirmeyen ama kavram sorusuna çok uygun bir konu: "hiyerarşiyi sırala", "zamansal/uzamsal yerellik örneği ver". Slayt sayısı az, puanı kolay.

→ **BU KONUYA NEDEN GEÇİYORUZ?**

5. bölümde sanal belleğin "gereken sayfaları RAM'de, gerisini diskte" tuttuğunu gördük. Yani bilgisayar zaten hızlı ama küçük bir yer (RAM) ile yavaş ama büyük bir yer (disk) arasında akıllıca veri taşıyordu. Aynı fikir CPU ile RAM arasında da var, hem de daha da büyük ölçekte. Bu bölüm, haftanın sorularından sonuncusunu cevaplıyor: *Bellek hiyerarşisi performansı nasıl etkiler?*

## 8.1 CPU ile bellek arasındaki hız uçurumu

❖ Kaynak: D1 s. 89 / slayt 90

Slayt 90'ın dört maddesi bir sebep–sonuç zinciri oluşturuyor:

1. **CPU komutları çok hızlı çalıştırır; ana belleğe erişim daha uzun sürer.**
2. **İşlemci veri beklerken yürütme boru hattı (execution pipeline) duraksayabilir (stall).** Boru hattı, işlemcinin birkaç komutu bir fabrika bandı gibi aynı anda farklı aşamalarda işlemesidir. Bir komutun verisi gelmezse bant durur.
3. **Önbellekler (caches), sık kullanılan veriyi işlemciye yakın tutar.**
4. **Amaç, her katmanın hız, kapasite ve maliyet avantajlarını birleştirmektir.**

Bir benzetme: Ödev yazarken masandaki defter (yazmaç) elinin altında, sırt çantandaki kitaplar (önbellek) birkaç saniyede, odandaki kitaplık (RAM) birkaç dakikada, kütüphanedeki kitaplar (disk) bir saatte, başka şehirdeki arşiv (ağ depolama) günler içinde ulaşılabilir. Hepsini masaya koyamazsın (masa küçük), hepsini kütüphanede bırakırsan da her kelime için oraya gidip gelirsin. Akıllı çözüm: sık kullandığını yakında tut.

## 8.2 Bellek hiyerarşisi piramidi

❖ Kaynak: D1 s. 90–91 / slayt 91–92



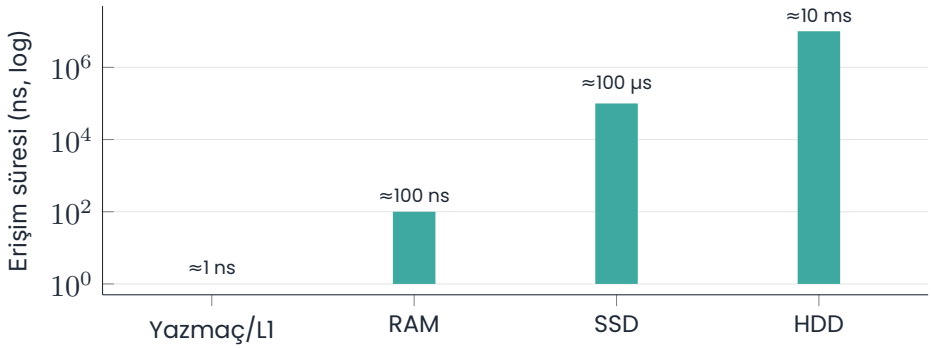

**Şekil 8.1:** Bellek hiyerarşisi (D1 s. 90 / slayt 91). Yukarı çıktıkça hız ve bayt başına maliyet artar, kapasite azalır.

| Katman | CPU'ya yakınlık | Tipik özellik | Kullanım |
|---|---|---|---|
| **Yazmaç (register)** | İşlemci çekirdeğinin içinde | Çok küçük, çok hızlı | O anki işlemin işlenenleri (operand) |
| **Önbellek (cache)** | Çekirdeğe çok yakın | **Donanım tarafından yönetilir** | Sık kullanılan bloklar |
| **RAM** | Bellek veriyolu (memory bus) üzerinde | **Uçucu (volatile)** ana bellek | Çalışan süreçler |
| **SSD / HDD** | I/O yolu üzerinden | **Kalıcı (persistent)** ve büyük | Dosyalar ve uygulamalar |

Tablodaki iki kelimeye dikkat: **Uçucu** demek elektrik kesilince içeriğin silinmesi demektir (RAM). **Kalıcı** demek elektrik olmadan da verinin korunmasıdır (SSD/HDD). Bilgisayarı kapatınca açık ama kaydedilmemiş belgenin kaybolmasının nedeni budur. Ayrıca önbelleğin **donanım tarafından yönetildiğini** not et: programcı ya da işletim sistemi "şunu önbelleğe koy" demez, işlemci bunu kendisi yapar. RAM ile disk arasındaki taşımayı ise (5. bölümdeki sayfalama ve takas) **işletim sistemi** yönetir.

**+ EK BİLGİ (KAYNAK DIŞI) — Katmanlar ne kadar farklı? (yaklaşık mertebeler)**

Slayt sayı vermiyor. Ders kitaplarında ve üretici belgelerinde sık geçen **kaba mertebeler** şöyledir (donanıma göre çok değişir, ezberleme; oranları hisset): yazmaç ve L1 önbellek ≈ 1 ns, RAM ≈ 100 ns, SSD ≈ 100 µs (= 100.000 ns), HDD ≈ 10 ms (= 10.000.000 ns). Yani RAM önbellekten yaklaşık **100 kat**, HDD ise RAM'den yaklaşık **100.000 kat** yavaş. Takasın ve sayfa hatasının neden bu kadar pahalı olduğunun (5. bölüm) sayısal gerekçesi budur.




**Şekil 8.2:** Yaklaşık erişim süreleri, logaritmik eksende (ek bilgi, kaynak dışı mertebeler). Her basamak bir öncekinin yüzlerce, binlerce katı.

## 8.3 Önbellek nasıl yardım eder?

❖ Kaynak: D1 s. 92 / slayt 93

- **Son kullanılan veri ve komutların kopyalarını tutar.**
- **Önbellek isabetinde (cache hit) veri daha hızlı alınır.** Aranan veri önbellekte bulunmuştur.
- **Önbellek ıskasında (cache miss) veri daha alt bir katmandan getirilir.** Aranan veri önbellekte yoktur; RAM'e gidilir (ve getirilen veri, sonra tekrar lazım olur diye önbelleğe de konur).
- **Daha büyük önbellek her zaman daha iyi değildir; gecikme, enerji ve maliyet etkilenir.** Büyük bir önbellekte aramak daha uzun sürer, daha çok enerji harcar ve çip üzerinde pahalı yer kaplar. Bu da bir ödünleşim.

**+ EK BİLGİ (KAYNAK DIŞI) — Ortalama erişim süresi**

Önbelleğin işe yarayıp yaramadığı **isabet oranına (hit ratio, h)** bağlıdır. Basitleştirilmiş bir modelde ortalama erişim süresi:
**T_ort = h · t_önbellek + (1 − h) · t_RAM**
Örnek: h = 0,95, t_önbellek = 1 ns, t_RAM = 100 ns → T_ort = 0,95·1 + 0,05·100 = 0,95 + 5 = **5,95 ns**. Önbellek olmasaydı her erişim 100 ns sürecekti; yani önbellek ortalamada yaklaşık **17 kat** hızlandırma sağlıyor. (Bazı kaynaklar ıskada önce önbelleğe bakma süresini de ekler: 0,95·1 + 0,05·(1 + 100) = 6 ns; sonuç neredeyse aynıdır.) Formül slaytta yok.

## 8.4 Yerellik ilkesi: önbellek neden işe yarıyor?

❖ Kaynak: D1 s. 93 / slayt 94

Önbellek küçük; bütün programı tutamaz. O hâlde neden bu kadar sık isabet ediyor? Çünkü programlar belleğe rastgele erişmez; erişimlerinde iki tür düzen vardır. Buna **yerellik ilkesi (principle of locality)** denir.

| | Zamansal yerellik (temporal locality) | Uzamsal yerellik (spatial locality) |
|---|---|---|
| Tanım | **Yakın zamanda kullanılan** veri yakında **yeniden** kullanılabilir | **Erişilen adresin yakınındaki** veri yakında kullanılabilir |
| Slayttaki örnek | Döngü sayacı (loop counter) | Dizi elemanları (array elements) |

| Önbellek bundan nasıl yararlanır? | Kullanılan veriyi bir süre önbellekte tutar | Bir veri istendiğinde **komşularıyla birlikte bir blok** hâlinde getirir |
|---|---|---|

Bir dizinin elemanlarını toplayan basit bir döngüye bakalım (sözde kod):

```
Sözde kod
toplam ← 0
i ← 0'dan 999'a kadar her i için:
    toplam ← toplam + dizi[i]
```

- `i` ve `toplam` döngünün **her turunda** yeniden kullanılıyor → **zamansal yerellik**.
- `dizi[0]`, `dizi[1]`, `dizi[2]`, … bellekte **yan yana** duruyor ve sırayla okunuyor → **uzamsal yerellik**. `dizi[0]` için RAM'e gidildiğinde önbelleğe onunla birlikte komşu birkaç eleman da getirilir; sonraki birkaç erişim isabet olur.
- Döngünün komutları da her turda aynı → komutlar için de zamansal yerellik.




**Şekil 8.3:** Uzamsal yerellik (şematik; blok boyutu örnek olarak 4 eleman seçildi). Sırayla okunan bir dizide her blok için yalnızca ilk erişim ıskadır.

**▶ ÖRNEK 8.1 — Ek örnek: hangi yerellik?**

**Soru.** Aşağıdakiler hangi tür yerelliğe örnektir? (a) Bir oyunda karakterin can değerinin her karede okunması. (b) Bir fotoğrafın piksellerinin satır satır işlenmesi. (c) Bir fonksiyonun bir döngü içinde tekrar tekrar çağrılması. (d) Bir metin dosyasının baştan sona okunması.
**Çözüm.** Ölçüt: *aynı* şey mi tekrar kullanılıyor (zamansal), yoksa *yanındaki* şey mi kullanılıyor (uzamsal)?

- (a) Aynı değişken tekrar tekrar → **zamansal**.
- (b) Bellekte art arda duran pikseller sırayla → **uzamsal**.
- (c) Fonksiyonun aynı komutları tekrar çalıştırılıyor → **zamansal**.
- (d) Dosyanın art arda baytları sırayla → **uzamsal**.

**Sık hata:** "Dizi" görünce otomatik olarak uzamsal demek. Bir dizinin *aynı* elemanını tekrar tekrar okursan bu zamansaldır.

**✎ KENDİN ÇÖZ**

Bir öğrenci 1000 × 1000'lik bir tabloyu (bellekte satır satır yan yana saklanıyor) önce sütun sütun, sonra satır satır topluyor. İkisi aynı sonucu veriyor ama satır satır olan çok daha hızlı çalışıyor. Yerellik ilkesiyle açıkla.

**★ MERAKLISINA (SINAV DIŞI)**

Programcılar bu ilkeyi bilerek kod yazınca aynı algoritma birkaç kat hızlanabilir; "önbellek dostu (cache-friendly) kod" denen şey budur. Yerellik fikri yalnızca işlemcide değil her yerde var: tarayıcının önbelleği, sık ziyaret ettiğin sitelerin dosyalarını diskte tutar; telefonun son

açılan uygulamaları RAM'de bekletir. Hepsi aynı mantık: yakında yine lazım olacak şeyi yakında tut.

## 8.5 Haftanın özeti ve gelecek hafta

❖ Kaynak: D1 s. 94–95 / slayt 95

Slayt 95 haftayı beş cümlede topluyor; her birinin hangi bölüme karşılık geldiğini eşleştirelim:

1. İşletim sistemi, uygulamalara güvenli soyutlamalar sunan bir kaynak yöneticisidir → **1. bölüm**.
2. Bellek yönetimi yerleştirme, koruma, takas ve sanal adreslemeyle ilgilenir → **5. bölüm**.
3. Süreç yönetimi durumları, kuyrukları ve bağlam değiştirmeyi koordine eder → **6. bölüm**.
4. Zamanlama algoritmaları farklı ölçütleri iyileştirir; seçim iş yüküne bağlıdır → **7. bölüm**.
5. Bellek hiyerarşisi hız, kapasite ve maliyeti dengeler → **8. bölüm**.

Son slayt (fiziksel s. 95) gelecek haftanın konusunu duyuruyor: **veri depolama (data storage)**. Bu bölümdeki hiyerarşinin alt katmanları (SSD/HDD) ve 5. bölümdeki dosya sistemi orada ayrıntılanacak gibi görünüyor.

## 8.6 Bu bölümün bağlantıları

- Önbellek ↔ RAM ilişkisi, 5. bölümdeki RAM ↔ disk ilişkisinin (sayfalama) daha hızlı katmandaki benzeridir: ikisinde de "sık kullanılanı hızlı katmanda tut, gerisini alttan getir" fikri var. Önbellek ıskası ile sayfa hatası aynı düşüncenin iki ölçeği.
- Yazmaçlar → 6. bölümde bağlam değiştirmede PCB'ye kaydedilen "CPU yazmaçları".
- Hız, kapasite, maliyet dengesi → 1. bölümdeki ödünleşim.

**⚠ SINAVDA DİKKAT**

- Hiyerarşi (hızlıdan yavaşa): **yazmaç → L1/L2/L3 önbellek → RAM → SSD/HDD → arşiv/ağ depolama.** Yukarı çıktıkça hız ve bayt başına maliyet ↑, kapasite ↓.
- **Önbellek donanım tarafından yönetilir**; **RAM uçucudur**; **SSD/HDD kalıcıdır.**
- **İsabet** → hızlı; **ıska** → alt katmandan getir.
- **Zamansal** = aynı veri tekrar (döngü sayacı); **uzamsal** = yanındaki veri (dizi elemanları).
- Büyük önbellek ≠ her zaman iyi (gecikme, enerji, maliyet).

**❖ BU KONUDAN NASIL SORU GELİR?**

1. **Sıralama:** "Order the following from fastest to slowest: RAM, register, HDD, L2 cache."
2. **Tanım + örnek:** "Explain temporal and spatial locality with one example each." *İpucu:* döngü sayacı / dizi elemanları (slayttaki örnekleri kullan).
3. **Kavramsal:** "Why is a larger cache not always better?" *İpucu:* gecikme, enerji, maliyet.
4. **Neden:** "Why do caches work even though they are small?" *İpucu:* yerellik ilkesi.
5. **Eşleştirme:** volatile / persistent / hardware-managed → RAM / SSD / cache.

**✘ SIK YAPILAN HATA**

- **"Önbelleği işletim sistemi yönetir."** → Doğrusu: Slayt 92: önbellek **donanım** tarafından yönetilir.
- **"RAM kalıcı bellektir."** → Doğrusu: RAM uçucudur; elektrik kesilince içeriği gider.
- **Zamansal ile uzamsalı karıştırmak.** → Doğrusu: zaman = *aynı* şey tekrar; uzam = *yanındaki* şey.
- **"Önbellek ne kadar büyükse o kadar iyi."** → Doğrusu: Büyüdükçe erişim gecikmesi, enerji ve maliyet artar.

**⚡ HIZLI TEKRAR**

- **Hız uçurumu:** CPU hızlı, RAM görece yavaş → boru hattı duraksar → önbellek gerekir.
- **Piramit:** yazmaç → önbellek → RAM → SSD/HDD → arşiv; hız/maliyet ↔ kapasite.
- **Önbellek:** son kullanılanın kopyası; **isabet** hızlı, **ıska** alttan getirme.
- **Yerellik: zamansal** (aynı veri tekrar) + **uzamsal** (komşu veri) → önbelleğin işe yaramasının sebebi.

*Ezberle:* piramidin sırası, slayt 92 tablosunun "tipik özellik" sütunu, iki yerellik türü ve örnekleri. *Türetebil:* ortalama erişim süresi (ek).
**Bu bölümden sonra şunları yapabiliyor olmalısın:** katmanları sıralamak ve özelliklerini söylemek; bir erişim desenindeki yerellik türünü belirlemek; önbelleğin neden işe yaradığını açıklamak.

## 8.7 Bölüm cevapları

**Kendin çöz (satır satır / sütun sütun).** Tablo bellekte satır satır yan yana saklandığı için **satır satır** toplarken art arda okunan elemanlar bellekte komşudur: önbelleğe getirilen her blokta sıradaki birkaç eleman da hazırdır → **uzamsal yerellik** sayesinde çoğu erişim isabet olur. **Sütun sütun** toplarken art arda okunan iki eleman bellekte bir satır uzunluğu (1000 eleman) uzaktadır; getirilen bloktaki komşular hemen kullanılmaz ve büyük olasılıkla önbellekten atılır → çok sayıda ıska. Hesap aynı, bellek erişim deseni farklı.

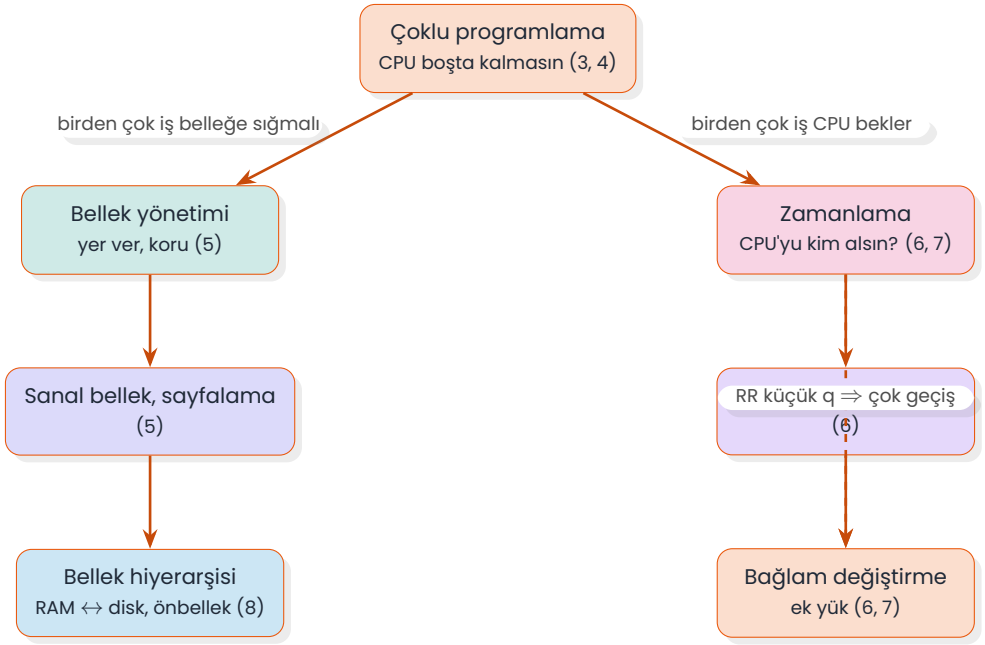
BÖLÜM 9

# Konular arası bağlantılar

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Sınavda puan kaybettiren şey çoğu zaman bilgisizlik değil, **birbirine benzeyen iki kavramı karıştırmaktır.** Bu bölüm, haftanın kavramlarını yan yana koyar. Önce kavramların birbirine nasıl bağlandığını, sonra karıştırılan çiftlerin karşılaştırma tablolarını göreceksin.

## 9.1 Kavramlar birbirine nasıl bağlanıyor?




**Şekil 9.1:** Çoklu programlama iki sorunu doğurur: belleği paylaştırmak ve CPU'yu paylaştırmak. Haftanın geri kalanı bu iki soruya verilen cevaplardır.

Bir cümleyle: **çoklu programlama** (1) birden çok işi bellekte tutar, bu yüzden bellek **yönetilmek** zorundadır; (2) birden çok iş CPU'yu bekler, bu yüzden **zamanlama** gerekir. Zamanlama süreçlerin **durumlarına** (PCB) dayanır; süreç değiştirmenin bedeli **bağlam değiştirme ek yüküdür.** Belleğin bir kısmı diske taşındığında (sanal bellek, takas) hız farkı yüzünden **hiyerarşi** konuya girer.

## 9.2 Karşılaştırma tabloları

### 9.2.1 Program, süreç, iş parçacığı, işlemci çekirdeği

| | Ne? | Nerede yaşar? | Örnek |
|---|---|---|---|
| **Program** | Diskte duran talimat dosyası (pasif) | Depolama | `chrome.exe` dosyası |
| **Süreç (process)** | Çalışan program + durumu + kaynakları (aktif); PCB'si vardır | RAM + CPU | Açık tarayıcı penceresi |
| **İş parçacığı (thread)** | Süreç içindeki ayrı yürütme akışı | Sürecin içinde | Sekme içindeki bir iş |
| **Çekirdek (core)** | CPU'nun fiziksel yürütme birimi | Donanım | 8 çekirdekli işlemci |

### 9.2.2 Üç ``aynı anda çok iş'' kavramı

| | Çoklu programlama | Zaman paylaşımı | Çoklu görev |
|---|---|---|---|
| Amaç | CPU kullanımını ve throughput'u artırmak | Çok kullanıcıya hızlı yanıt | Kullanıcının birkaç işi aynı anda yürütmesi |
| Nasıl? | I/O beklerken başka iş çalıştır | Kısa zaman dilimleriyle sırayla çalıştır | Kesme (preemption) + zamanlayıcı |
| Ölçüt | Kullanım, throughput | Yanıt süresi | Yanıt süresi (bedeli ek yük) |

Üçü birbirini dışlamaz: modern sistemler hepsini birlikte kullanır. Sınavda sorulan fark *amaç* farkıdır.

### 9.2.3 İç parçalanma ile dış parçalanma

| | İç parçalanma | Dış parçalanma |
|---|---|---|
| Boşluk nerede? | **Ayrılan bloğun içinde** | **Bloklar arasında** |
| Neden? | Sabit boyutlu birim, istek daha küçük (sayfalama son sayfa) | Dinamik bölümleme; blokların bitip yeniden dağılması |
| Toplam boş yeterli mi? | Bu bellek zaten ayrılmış, kullanılamıyor | Toplam yeterli, ama **bitişik değil** |
| Çözüm | Daha küçük birim | Sıkıştırma (pahalı), sayfalama |

### 9.2.4 Yerleştirme algoritmaları

| | İlk uyan | En iyi uyan | En kötü uyan | Sonraki uyan |
|---|---|---|---|---|
| Kural | Baştan ilk yeten | **En küçük** yeten | **En büyük** blok | Son yerden devam, ilk yeten |
| Not | Hızlı | Küçük artıklar bırakır | Büyük artık kalır | Konum hatırlanır |

70K örneği (slayt 53): ilk uyan → 100K, en iyi uyan → 100K, en kötü uyan → 500K.

### 9.2.5 Sıkıştırma, takas, sayfalama

| | Ne yapar? | Bedeli |
|---|---|---|
| Sıkıştırma | Delikleri tek blokta toplar | Süreçler taşınır, zaman alır |
| Takas | Süreci geçici olarak diske atar | Disk yavaş, çok pahalı |
| Sayfalama | Belleği sabit boyutlu sayfa/çerçevelere böler | Sayfa tablosu, sayfa hatası; dış parçalanma yok |

### 9.2.6 Kesintili ve kesintisiz zamanlama, dört algoritma

| Algoritma | Kesintili? | Kural | Ana sorun |
|---|---|---|---|
| FCFS | Hayır | Gelen sırayla | Konvoy etkisi |
| SJF | Hayır | En kısa burst | Süreyi önceden bilmek; uzun işler beklemede |
| SRTF | **Evet** | En kısa **kalan** süre | Çok geçiş, tahmin gerekir |
| Öncelik | İkisi de olabilir | En yüksek öncelik | **Açlık** (çözüm: yaşlandırma) |
| Round Robin | **Evet** | q kadar çalış, sona geç | q'ya duyarlı; çok küçük q → ek yük |

### 9.2.7 Beş zamanlama ölçütü

| Ölçüt | Tanım | İstenen | Formül |
|---|---|---|---|
| CPU kullanımı | CPU'nun meşgul kalma oranı | ↑ | meşgul / toplam |
| Throughput | Birim zamanda biten iş | ↑ | iş sayısı / süre |
| Turnaround (TAT) | Varıştan bitişe | ↓ | C − A |
| Bekleme (W) | Hazır kuyruğunda geçen toplam | ↓ | TAT − B |
| Yanıt (R) | Varıştan **ilk** CPU'ya | ↓ | S − A |

### 9.2.8 Önbellek, RAM ve disk

| | Yönetimi | Uçucu mu? | Hız / kapasite |
|---|---|---|---|
| Yazmaç | Çekirdek içinde | Evet | En hızlı, en küçük |
| Önbellek | **Donanım** | Evet | Çok hızlı, küçük |
| RAM | İşletim sistemi | **Evet** | Orta, GB |
| SSD/HDD | İşletim sistemi + dosya sistemi | **Hayır (kalıcı)** | Yavaş, büyük |

## 9.3 Bölümler arası ``uyum kontrolü''

✎ **DURUP DÜŞÜN**

**Durup düşün.** (1) Round Robin'de kuantumu çok küçük seçersek hangi bölümün hangi kavramı sorun olur? (2) Sayfalama neden dış parçalanmayı ortadan kaldırır ama iç parçalanmayı tamamen kaldırmaz? (3) Bir RTOS'ta "en iyi" zamanlama algoritması neden ortalama bekleme süresini en küçüklemek değildir?

**Cevaplar.** (1) 6. ve 7. bölüm: **bağlam değiştirme ek yükü** artar; q büyürse RR, FCFS'ye yaklaşır. (2) Çünkü bütün birimler aynı boyuttadır, boş çerçeve herhangi bir sayfaya uyar (dış parçalanma yok); fakat son sayfa genelde tam dolmaz (iç parçalanma). (3) RTOS'ta ölçüt **deadline'ı karşılamaktır** (2. bölüm); ortalama süre iyi olsa da bir işin son teslim süresi kaçarsa sistem hatalıdır.

BÖLÜM 10

# Sınav öncesi hızlı tekrar

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Sınav sabahı tek oturuşta okunacak yoğunlukta bir özet. Anlatım yoktur; anlamadığın madde için ilgili bölüme dön.

## 10.1 Temel tanımlar

**∑ TANIM / FORMÜL**

- **İşletim sistemi:** bilgisayar **kaynaklarını yöneten** ve donanımı **soyutlayan** yazılım katmanı (1).
- **Uygulama / sistem programı / sürücü:** kullanıcı için / sistem için / belirli donanım için (1).
- **Kullanıcı kipi / çekirdek kipi:** sınırlı yetki / tam yetki; **sistem çağrısı** çekirdeğe kontrollü giriştir (1).
- **RTOS:** doğru sonuç **doğru zamanda**; "gerçek zamanlı ≠ çok hızlı", mesele **deadline** (2).
- **Çoklu programlama:** I/O beklerken CPU boşta kalmasın diye bellekte çok iş (3). **Zaman paylaşımı:** kısa dilimlerle hızlı yanıt (3).
- **Core / thread / process:** fiziksel birim / süreç içi akış / çalışan program (3).
- **Beş görev:** arayüz, program çalıştırma, aygıt yönetimi (kesme), dosyalar, koruma ve güvenlik (4). Güvenlik: *authentication, authorization, isolation, auditing*.
- **İç / dış parçalanma:** blok **içinde** / bloklar **arasında** boşluk (5).
- **Sanal bellek:** süreç büyük ve bitişik adres alanı görür; **sayfa ↔ çerçeve**, **sayfa tablosu**, **sayfa hatası** (5).
- **PCB:** sürecin kimlik ve "ayracı": durum, program sayacı, yazmaçlar vb. (6).
- **Durumlar:** Yeni → Hazır → Çalışıyor → (Bekliyor → Hazır) → Sonlandı; Çalışıyor → Hazır (kesme) (6).
- **Bağlam değiştirme:** eski sürecin durumunu kaydet, yenisininkini yükle; **saf ek yük** (6).
- **Önbellek** donanım yönetir; **yerellik:** zamansal + uzamsal (8).

## 10.2 Formüller

**∑ TANIM / FORMÜL**

- **TAT = C − A** (bitiş − varış) · **W = TAT − B** (turnaround − burst) · **R = S − A** (ilk CPU − varış)
- **Round Robin:** n süreç, kuantum q ⇒ bir süreç en fazla **(n − 1)·q** sonra yeniden CPU alır.
- **CPU kullanımı (tek program):** CPU süresi / (CPU + I/O) → 6 / (6 + 10) = **%37,5**; n program için ≈ n × bu oran (en çok %100).
- **RR kuyruk kuralı (bu notta):** aynı anda gelen yeni süreç, kesilen sürecin **önüne** girer.
- *Ek (kaynak dışı):* ortalama erişim süresi = h·t_önbellek + (1 − h)·t_RAM.

## 10.3 Sayısal sonuçlar (slayt örnekleri)

| Örnek | Sonuç |
|---|---|
| FCFS, burstler 24, 3, 3 (aynı anda) | Ortalama W = **17**, TAT = **27**; ters sıra (3, 3, 24): W = **3** |
| SJF, varışsız (P1=6, P2=4, P3=1, P4=5) | Ortalama W = **4**, TAT = **8** |
| SRTF, varışlı küme (slayt 81) | Ortalama W = **3**, TAT = **7** |
| RR q = 20, (53, 17, 68, 24) | Ortalama W = **73**, TAT = **113,5**, R = **28,5** |
| Alıştırma 8, 4, 2 | FCFS W = **6,67**; SJF W = **2,67**; RR (q = 3) W = **6,67** |
| Bellek 70K isteği | İlk uyan → **100K**, en iyi uyan → **100K**, en kötü uyan → **500K** |

## 10.4 Kritik ayrımlar

- Program (pasif, diskte) ≠ süreç (aktif, RAM'de).
- İç parçalanma (blok **içi**) ≠ dış parçalanma (bloklar **arası**).
- Kesintili (CPU zorla alınır: SRTF, RR) ≠ kesintisiz (FCFS, SJF).
- Bekleme süresi ≠ turnaround ≠ yanıt süresi.
- Çoklu programlama (**amaç: kullanım**) ≠ zaman paylaşımı (**amaç: yanıt**).
- Uçucu (RAM) ≠ kalıcı (SSD/HDD). Önbelleği **donanım** yönetir.
- Çekirdek (işlemci çekirdeği = *core*) ≠ çekirdek (işletim sistemi = *kernel*).

## 10.5 En sık yapılan hatalar

1. **SJF'yi "kesintili" sanmak.** SJF kesintisizdir; kesintili hâli SRTF'tir.
2. **Bekleme süresini "bitiş − varış" hesaplamak.** Bu turnaround'dur; W = TAT − B.
3. **Round Robin'de yeni gelen süreci kesilenin arkasına koymak** (bu notta önüne girer; hocanın kuralı farklıysa onu uygula).
4. **Yanıt süresini bitişe göre hesaplamak.** İlk CPU'ya göre.
5. **"Gerçek zamanlı = hızlı" demek.**
6. **En kötü uyan algoritmasını "en yavaş" sanmak.** Kural bloğun *büyüklüğüne* bakar, hızına değil.
7. Önbelleği işletim sisteminin yönettiğini söylemek.

## 10.6 Sınav öncesi kontrol listesi

- ☐ Beş durumu ve geçişleri (kesme dahil) çizebiliyorum.
- ☐ Dört algoritmanın Gantt'ını ve TAT/W/R'sini hesaplayabiliyorum.
- ☐ Dört yerleştirme kuralını bir istek dizisine uygulayabiliyorum.
- ☐ İç/dış parçalanmayı örnekle ayırabiliyorum.
- ☐ Sistem çağrısı adımlarını sırayla yazabiliyorum.
- ☐ Çoklu programlama / zaman paylaşımı / çoklu görev farkını anlatabiliyorum.
- ☐ Bellek hiyerarşisini hızdan yavaşa sıralayabiliyorum.
- ☐ Deneme sınavını en az bir kez çözdüm.

BÖLÜM 11

# Deneme sınavı

**→ BU KONUYA NEDEN GEÇİYORUZ?**

**26 soru:** 14 çoktan seçmeli (Bölüm A, her sorunun tek doğru cevabı vardır) ve 12 açık uçlu/hesap sorusu (Bölüm B). Önerilen süre **100 dakika**. Cevap anahtarına bakmadan, kâğıt ve hesap makinesiyle çöz; bittikten sonra bir sonraki bölümdeki anahtarla karşılaştır. "Ek" işaretli sorular slaytta olmayan, notta türetilmiş örneklerdir.

## Bölüm A: Çoktan seçmeli

**Soru 1** (Kolay) — Slayttaki sınıflandırmaya göre aşağıdakilerden hangisi bir **sistem programıdır**? (Aygıt sürücüleri slaytta ayrı bir sınıftır.)

(A) Müzik çalar uygulaması (B) C derleyicisi (`gcc`) (C) Yazıcı sürücüsü (D) Web tarayıcısı

❖ Kaynak: D1 s. 6

**Soru 2** (Kolay) — **Gerçek zamanlı işletim sistemi (RTOS)** için aşağıdakilerden hangisi doğrudur?

(A) Bütün işletim sistemleri arasında en yüksek işlemci hızına sahiptir. (B) Ekranda gerçek saati gösterir. (C) Doğru sonucu, **son teslim süresini (deadline)** güvenilir biçimde karşılayarak üretir. (D) Yalnızca masaüstü bilgisayarlarda kullanılır.

❖ Kaynak: D1 s. 17

**Soru 3** (Orta) — Kullanıcı kipinde çalışan bir program diske doğrudan erişemez. Dosya okumak için ne yapar?

(A) **Sistem çağrısı** yaparak çekirdeğe istek gönderir. (B) Kendi sürücüsünü yükleyip diske erişir. (C) Kendini çekirdek kipine geçirir. (D) Diski önbelleğe kopyalar.

❖ Kaynak: D1 s. 11–12

**Soru 4** (Orta) — **Çoklu programlamanın** (multiprogramming) temel amacı hangisidir?

(A) Çok kullanıcıya çok kısa yanıt süresi sağlamak (B) Bellek parçalanmasını önlemek (C) Kullanıcı kimliğini doğrulamak (D) Bir iş I/O beklerken CPU'yu boşta bırakmayıp **CPU kullanımını ve throughput'u** artırmak

❖ Kaynak: D1 s. 25

**Soru 5** (Orta) — Dinamik bölümlemeli bir bellekte toplam boş alan 300 KB, fakat en büyük **tek** boş blok 120 KB'dir. 200 KB'lık bir istek bu yüzden karşılanamıyor. Bu durumun adı nedir?

(A) İç parçalanma (B) Dış parçalanma (C) Sayfa hatası (D) Bağlam değiştirme

❖ Kaynak: D1 s. 51

**Soru 6** (Orta) — Boş bloklar sırayla 150K, 400K, 120K, 300K'dır. 100K'lık bir istek **en iyi uyan (best fit)** ile hangi bloğa yerleşir?

(A) 150K (B) 400K (C) 120K (D) 300K

❖ Kaynak: D1 s. 52

**Soru 7** (Orta) — **Sayfalama** için hangisi doğrudur?

(A) Süreç sabit boyutlu sayfalara bölünür; sayfalar fiziksel bellekte **bitişik olmayan** çerçevelere yerleşebilir. (B) Sürecin bütün sayfaları fiziksel bellekte bitişik olmak zorundadır. (C) Dinamik bölümlemeden daha çok dış parçalanma doğurur. (D) Sayfa tablosuna ihtiyaç duyulmaz.

❖ Kaynak: D1 s. 56–57

**Soru 8** (Kolay) — Bir süreç beklediği **I/O işlemi tamamlandığında** hangi duruma geçer?

(A) Çalışıyor (B) Bekliyor (C) Sonlandı (D) Hazır

❖ Kaynak: D1 s. 62–63

**Soru 9** (Orta) — Aşağıdakilerden hangisi **kesintili (preemptive)** bir zamanlama algoritmasıdır?

(A) FCFS (B) SRTF (C) SJF (klasik, kesintisiz) (D) Kesintisiz öncelik

❖ Kaynak: D1 s. 67; s. 80 / slayt 81

**Soru 10** (Orta) — **Bağlam değiştirme** (context switch) için hangisi doğrudur?

(A) Kullanıcı programı için yararlı iş yapılan süredir. (B) Yalnızca I/O olduğunda gerçekleşir. (C) Bu sürede hiçbir kullanıcı programı ilerlemez; çok sık yapılırsa verim düşer. (D) Süreç denetim bloğuna (PCB) ihtiyaç duymaz.

❖ Kaynak: D1 s. 66

**Soru 11** (Orta, ek) — Hepsi 0. anda gelen dört sürecin CPU süreleri A = 6, B = 3, C = 1, D = 4 birimdir. **SJF** ile ortalama bekleme süresi kaçtır?

(A) 3,25 (B) 4,5 (C) 5 (D) 6,25

❖ Kaynak: Konu: D1 s. 77 / slayt 78; sayısal örnek ek

**Soru 12** (Orta) — Bir **Round Robin** sisteminde 6 süreç ve kuantum q = 5 ms'dir. Hazır kuyruğundaki bir süreç, en kötü durumda CPU'yu tekrar almak için yaklaşık ne kadar bekler?

(A) 5 ms (B) 30 ms (C) 6 ms (D) 25 ms

❖ Kaynak: D1 s. 83 / slayt 84

**Soru 13** (Orta) — **Önbellek** neden işe yarar?

(A) Disk her zaman RAM'den hızlıdır. (B) Programlar yakın zamanda kullandıkları veriyi ve **komşu verileri** yeniden kullanma eğilimindedir (**yerellik**). (C) Önbellek RAM'den büyüktür. (D) İşletim sistemi önbelleğe hangi verinin gireceğini seçer.

❖ Kaynak: D1 s. 92–93 / slayt 93–94

**Soru 14** (Orta) — Bir döngüde `toplam` değişkeni **her turda** kullanılıyor ve `d[0], d[1], d[2], …` dizi elemanları **sırayla** okunuyor. Bunlar sırasıyla hangi yerellik türüdür?

(A) Uzamsal, zamansal (B) İkisi de zamansal (C) Zamansal, uzamsal (D) İkisi de uzamsal

❖ Kaynak: D1 s. 93 / slayt 94

## Bölüm B: Açık uçlu ve hesap soruları

**Soru 15** (Orta) — Bir uygulamanın dosya okuma isteğinin **kullanıcı kipinden çekirdeğe** gidip geri dönmesini beş adımda yaz. Her adımda hangi kipte olunduğunu belirt. Ayrıca işletim sisteminin "kaynak yöneticisi" ve "soyutlama katmanı" rollerini birer örnekle açıkla.

❖ Kaynak: D1 s. 7–8, 11–12

**Soru 16** (Orta, ek) — Bir programın her turunda 4 ms CPU, ardından 12 ms I/O bekleme vardır. Programlar birbirini engellemiyor varsayımıyla, sistemde **1, 2, 3 ve 4** program varken CPU kullanımını bul. Kaç programdan sonra CPU tam dolar?

❖ Kaynak: D1 s. 46; sayısal örnek ek

**Soru 17** (Orta, ek) — Boş bloklar bellek sırasıyla **150K, 420K, 120K, 300K**'dır. İstekler sırayla **100K, 250K, 130K, 90K**'dır. (a) **İlk uyan**, **en iyi uyan** ve **en kötü uyan** algoritmalarıyla her isteğin hangi bloğa gittiğini ve son boş blok listesini yaz. (b) Ardından **280K**'lık bir istek gelirse hangi algoritmalar karşılar? Karşılayamayan varsa nedenini açıkla.

❖ Kaynak: D1 s. 52–53; sayısal örnek ek

**Soru 18** (Orta, ek) — Sayfa boyutu 4 KB'dır. 17 KB'lık bir süreç için kaç sayfa gerekir ve **iç parçalanma** kaç KB'dir? 16 KB'lık bir süreçte bu değerler ne olur?

❖ Kaynak: D1 s. 51, 57; sayısal değerler ek

**Soru 19** (Orta) — Bir sürecin ömrü şöyledir: kabul edilir → CPU'ya verilir → disk okuma ister → okuma biter → yeniden CPU'ya verilir → kuantumu dolar → yeniden CPU'ya verilir → çıkış yapar. Her olaydan sonraki **durumu** yaz ve her geçişin adını belirt.

❖ Kaynak: D1 s. 62–63

**Soru 20** (Zor) — Süreçler (varış / burst): P1 (0 / 5), P2 (1 / 3), P3 (2 / 8), P4 (3 / 2). **FCFS, SJF, SRTF** ve **RR (q = 3)** için Gantt şemasını çiz, her algoritmada ortalama bekleme süresini bul. RR'de aynı anda gelen yeni süreç, kesilenin **önüne** girer. En düşük ortalama beklemeyi hangisi verir, neden?

❖ Kaynak: D1 s. 73–87 / slayt 74–88; süreç kümesi ek

**Soru 21** (Zor, ek) — P1 = 8, P2 = 4 birim; ikisi de 0. anda gelir. **RR, q = 4**, her **süreç değişiminde** 1 birim bağlam değiştirme süresi harcanır (ilk çalıştırmada yok). Gantt'ı çiz; ortalama W, ortalama R (R = ilk CPU'ya girişin zamanı − varış) ve CPU'nun **yararlı** kullanım oranını bul.

❖ Kaynak: D1 s. 66, 83 / slayt 84; sayısal örnek ek

**Soru 22** (Orta) — Bir hastane için ilaç infüzyon pompası kontrol yazılımı geliştiriliyor. Hangi tür işletim sistemi uygun olur? Kritik ölçüt nedir? Neden masaüstü işletim sistemi uygun değildir?

❖ Kaynak: D1 s. 14–17

**Soru 23** (Orta, ek) — Önbellek isabet oranı %95, önbellek erişimi 2 ns, RAM erişimi 100 ns'dir (ıskada yalnızca RAM erişim süresi harcanır). **Ortalama erişim süresini** bul ve yalnızca RAM kullanımına göre kaç kat hızlanma olduğunu yaz.

❖ Kaynak: Konu: D1 s. 92 / slayt 93; formül ek

**Soru 24** (Zor) — "Bellekte aynı anda ne kadar çok süreç olursa CPU o kadar verimli kullanılır" iddiasını değerlendir. En az üç gerekçeyle sınırlarını açıkla.

❖ Kaynak: D1 s. 25, 46, 49–51, 66

**Soru 25** (Orta) — Bir öğrenci şöyle yazmış: *"SJF, çalışan sürecin yerini daha kısa bir süreç gelince alır ve ortalama bekleme süresini en küçük yapar."* Cümledeki **hatayı** bul, düzelt ve doğru ifadeyi hangi algoritmanın verdiğini söyle.

❖ Kaynak: D1 s. 77–80 / slayt 78–81

**Soru 26** (Kolay) — Aşağıdaki durumları **authentication, authorization, isolation, auditing** kavramlarıyla eşleştir: (a) Kullanıcı parolasıyla giriş yapar. (b) Bir kullanıcı başkasının dosyasını okuyamaz. (c) Bir uygulamanın çökmesi diğer uygulamaları etkilemez. (d) Hangi dosyanın kim tarafından, ne zaman silindiği kaydedilir.

❖ Kaynak: D1 s. 39

BÖLÜM 12

# Deneme sınavı cevap anahtarı

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Her çoktan seçmeli sorunun **doğru şıkkı** ve **her yanlış şıkkın hangi kavram karışıklığını temsil ettiği** yazılıdır. Açık uçlu sorularda adım adım çözüm ile hocanın **kısmi puan verebileceği ara adımlar** işaretlidir. Hesapların hepsi Python benzetimiyle kontrol edilmiştir (`dogrulama/` klasöründeki betikler).

**Cevap özeti (Bölüm A):** 1-B · 2-C · 3-A · 4-D · 5-B · 6-C · 7-A · 8-D · 9-B · 10-C · 11-A · 12-D · 13-B · 14-C

## Bölüm A cevapları

**Soru 1 — B.** Slayt, derleyiciyi sistem programı örneği olarak verir (sistemi destekler, işleri kolaylaştırır). (A) ve (D) kullanıcının işini yapan **uygulama**dır. (C) belirli bir donanımla işletim sistemi arasında tercümanlık yapan **sürücü**dür. Karışıklık: "araç gibi görünen her şey sistem programı" sanmak.

**Soru 2 — C.** RTOS'ta mesele hız değil, **zamanında doğru sonuç**tur. (A) "gerçek zamanlı = çok hızlı" yanılgısıdır. (B) "gerçek zamanlı"nın saat anlamıyla karıştırılmasıdır. (D) RTOS masaüstünde de olabilir; kullanım alanı kısıtı slaytta yoktur, tıbbi, otomotiv ve robot örnekleri verilir.

**Soru 3 — A.** Kullanıcı kipi ayrıcalıksızdır; çekirdeğe **sistem çağrısı** (trap) ile kontrollü girilir. (B) Sürücüler çekirdek tarafındadır, kullanıcı programı yükleyemez. (C) Program kendi kendine kip değiştiremez; bu güvenliği ortadan kaldırırdı. (D) Önbellek diskin kopyası değildir ve işin yolu değildir.

**Soru 4 — D.** Çoklu programlama, I/O beklerken başka işi çalıştırarak CPU'yu doldurmaktır. (A) **zaman paylaşımının** amacıdır. (B) parçalanma bellek yönetiminin konusudur. (C) güvenlik görevidir (kimlik doğrulama).

**Soru 5 — B.** Toplam yeterli ama **bitişik değil → dış parçalanma**. (A) iç parçalanmada boşluk ayrılan bloğun içindedir. (C) sayfa hatası, sayfanın RAM'de olmamasıdır. (D) bağlam değiştirme süreçler arası geçiştir.

**Soru 6 — C.** İsteği karşılayan bloklar 150K, 400K, 120K, 300K'ın hepsidir (100K sığar); **en küçüğü** 120K'dır. (A) 150K ikinci en küçüktür. (B) 400K en kötü uyan seçimidir. (D) 300K'ın hiçbir kuralla ilgisi yok. İlk uyan seçseydi 150K olurdu.

**Soru 7 — A.** Sayfalamanın amacı bitişiklik zorunluluğunu kaldırmaktır. (B) bu zorunluluk dinamik bölümlemeye aittir. (C) sayfalama dış parçalanmayı ortadan kaldırır, yalnızca iç parçalanma olabilir. (D) adres çevirisi için sayfa tablosu şarttır.

**Soru 8 — D.** I/O bitince süreç doğrudan çalışmaz; **Hazır** kuyruğuna döner, zamanlayıcı onu seçer. (A) çok yapılan hatadır: bekleyen süreç CPU'yu otomatik almaz. (B) zaten bekliyordu. (C) süreç bitmiş değildir.

**Soru 9 — B.** SRTF, daha kısa kalan süreli süreç gelince çalışanı **keser**. (A) FCFS ve (C) klasik SJF çalışan süreci bitene kadar bırakmaz. (D) "kesintisiz öncelik" tanımı gereği kesintisizdir.

**Soru 10 — C.** Bağlam değiştirme saf **ek yüktür**; kullanıcı programı ilerlemez. (A) yararlı iş sanmak ana yanılgıdır. (B) I/O dışında da olur (kuantum bitişi, kesme). (D) PCB tam olarak durumun saklandığı yerdir.

**Soru 11 — A.** SJF sırası: C (1), B (3), D (4), A (6). Başlangıçlar 0, 1, 4, 8 → W = 0, 1, 4, 8 → toplam 13, ortalama **3,25**. (D) 6,25 **FCFS**'nin sonucudur (W = 0, 6, 9, 10). (B), (C) rastgele hesap hatalarıdır.

**Soru 12 — D.** Kendi diliminden sonra kuyrukta diğer 5 süreç, her biri en çok q kadar çalışır: (n − 1)·q = 5 × 5 = **25 ms**. (B) n·q, kendini de sayma hatasıdır. (A) yalnızca bir dilimdir. (C) sayı karışıklığı.

**Soru 13 — B.** Önbellek **yerelliğe** dayanır. (A) ters: disk RAM'den yavaştır. (C) önbellek RAM'den küçüktür. (D) önbelleği **donanım** yönetir.

**Soru 14 — C.** `toplam`'ın tekrar tekrar kullanılması **zamansal**; `d[i]` elemanlarının yan yana okunması **uzamsal** yerelliktir. (A) sıra ters. (B), (D) bir türü iki kez saymaktır.

## Bölüm B cevapları

### 12.0.1 Soru 15

**Çözüm.** (1) Uygulama dosya okuma isteğini hazırlar ve sistem çağrısını yapar (**kullanıcı kipi**). (2) İşlemci **çekirdek kipine** geçer (trap). (3) Çekirdek isteği ve yetkiyi **kontrol eder** (**çekirdek kipi**). (4) Çekirdek aygıt/sürücü aracılığıyla diski okur ve veriyi hazırlar (**çekirdek kipi**). (5) Sonuç uygulamaya döner, işlemci **kullanıcı kipine** geri geçer.

Roller: **kaynak yöneticisi** → CPU, bellek ve disk gibi kaynakları süreçler arasında adil ve güvenli paylaştırır (ör. birden çok programın aynı diski kullanması). **Soyutlama** → donanım ayrıntılarını gizler (ör. program "dosya aç" der, diskin hangi sektörde olduğunu bilmez).

*Kısmi puan:* kipleri doğru belirtme, "kontrol" adımını yazma, iki rolün tanımı ve birer örnek.

### 12.0.2 Soru 16

**Çözüm.** Bir programın CPU oranı 4 / (4 + 12) = 4/16 = %25. İdeal durumda programlar birbirini engellemez, kullanım doğrusal artar: 1 → **%25**, 2 → **%50**, 3 → **%75**, 4 → **%100**. Dördüncü programdan sonra CPU tam dolar; 5. program kullanımı artırmaz (üst sınır %100).

*Kısmi puan:* %25'in doğru bulunması; "en fazla %100" sınırı.

### 12.0.3 Soru 17

**(a) İlk uyan.** 100K → 150K blok (kalan 50K). 250K → 420K blok (kalan 170K). 130K → 50K yetmez, 170K yeter (kalan 40K). 90K → 40K yetmez, 120K yeter (kalan 30K). Son liste: **50K, 40K, 30K, 300K**.

**En iyi uyan.** 100K → yetenler arasında en küçüğü 120K (kalan 20K). 250K → 420K ve 300K'dan küçüğü 300K (kalan 50K). 130K → 150K (kalan 20K). 90K → yalnızca 420K yeter (kalan 330K). Son liste: **20K, 330K, 20K, 50K**.

**En kötü uyan.** 100K → en büyük 420K (kalan 320K). 250K → en büyük 320K (kalan 70K). 130K → en büyük 300K (kalan 170K). 90K → en büyük 170K (kalan 80K). Son liste: **150K, 70K, 120K, 80K**.

**(b) 280K isteği.** İlk uyan: 300K bloğa yerleşir (kalan 20K) → **karşılar**. En iyi uyan: 330K bloğa yerleşir (kalan 50K) → **karşılar**. En kötü uyan: boş toplam 150 + 70 + 120 + 80 = 420K ≥ 280K, ama en büyük blok 150K → **karşılayamaz**; bu **dış parçalanmadır**. Çare: **sıkıştırma** (veya sayfalama).

*Kısmi puan:* her isteğin doğru bloğu, kalan boyutlar, "toplam yeterli, bitişik değil" gerekçesi.

### 12.0.4 Soru 18

**Çözüm.** 17 / 4 = 4,25 → **5 sayfa**. Ayrılan 5 × 4 = 20 KB, kullanılan 17 KB → iç parçalanma 20 − 17 = **3 KB** (yalnızca **son sayfada**). 16 KB için 16 / 4 = **4 sayfa**, iç parçalanma **0**.

### 12.0.5 Soru 19

| Olay | Yeni durum | Geçişin adı |
|---|---|---|
| Kabul edilir | **Hazır** | admit (kabul) |
| CPU'ya verilir | **Çalışıyor** | dispatch (sevk) |
| Disk okuma ister | **Bekliyor** | I/O veya olay beklemesi |
| Okuma biter | **Hazır** | I/O veya olay tamamlanması |
| CPU'ya verilir | **Çalışıyor** | dispatch |
| Kuantumu dolar | **Hazır** | kesme (preemption) |
| CPU'ya verilir | **Çalışıyor** | dispatch |
| Çıkış yapar | **Sonlandı** | exit |

Başlangıç durumu **Yeni**'dir. *Kısmi puan:* her doğru durum; Bekliyor → Çalışıyor'a doğrudan geçiş yazmamak.

### 12.0.6 Soru 20

**FCFS:** P1 0–5, P2 5–8, P3 8–16, P4 16–18. W: P1 = 0, P2 = 5 − 1 = 4, P3 = 8 − 2 = 6, P4 = 16 − 3 = 13 → toplam 23, ortalama **5,75**.

**SJF (kesintisiz):** P1 0–5. t = 5'te hazır olanlar P2 (3), P3 (8), P4 (2) → P4 5–7, P2 7–10, P3 10–18. W: P1 = 0, P2 = 7 − 1 = 6, P3 = 10 − 2 = 8, P4 = 5 − 3 = 2 → toplam 16, ortalama **4**.

**SRTF:** t = 1'de P2 gelir (3 < P1'in kalan 4'ü) → P2 çalışır. t = 3'te P4 (2) gelir, ama P2'nin kalanı 1, P2 sürer. P2 t = 4'te biter. Hazır: P1 (4), P3 (8), P4 (2) → P4 4–6, P1 6–10, P3 10–18. Gantt: P1 0–1, P2 1–4, P4 4–6, P1 6–10, P3 10–18. W: P1 = 10 − 0 − 5 = 5, P2 = 4 − 1 − 3 = 0, P3 = 18 − 2 − 8 = 8, P4 = 6 − 3 − 2 = 1 → toplam 14, ortalama **3,5**.

**RR q = 3:** t = 0 P1 (0–3, kalan 2). Kuyruk: P2 (1'de geldi), P3 (2'de), P4 (3'te geldi, P1'den önce) → kuyruk P2, P3, P4, P1. P2 3–6 (biter), P3 6–9 (kalan 5), P4 9–11 (biter), P1 11–13 (biter), P3 13–16 (kalan 2), P3 16–18. W: P1 = 13 − 5 = 8, P2 = 5 − 3 = 2, P3 = 16 − 8 = 8, P4 = 8 − 2 = 6 → toplam 24, ortalama **6**.

| Algoritma | Ort. W | Ort. TAT | Ort. R |
|---|---|---|---|
| FCFS | 5,75 | 10,25 | 5,75 |
| SJF | 4 | 8,5 | 4 |
| SRTF | **3,5** | 8 | **2,25** |
| RR (q = 3) | 6 | 10,5 | 3 |

**Yorum.** En düşük ortalama beklemeyi **SRTF** verir: her an kalan süresi en kısa işi çalıştırarak kısa işleri bekletmez, uzun işin biraz geç bitmesi ortalamayı az etkiler. Bedeli: daha çok bağlam değiştirme ve kalan süreyi bilme/tahmin zorunluluğu.

*Kısmi puan:* her algoritmanın doğru Gantt'ı; W = TAT − B uygulaması; RR'de sıra kuralı.

### 12.0.7 Soru 21

**Çözüm.** P1 0–4 (kalan 4), bağlam değiştirme 4–5, P2 5–9 (biter), bağlam değiştirme 9–10, P1 10–14 (biter). **TAT:** P1 = 14, P2 = 9. **W = TAT − B:** P1 = 14 − 8 = 6, P2 = 9 − 4 = 5 → ortalama **5,5**. **R = S − A:** P1 = 0, P2 = 5 → ortalama **2,5**. Yararlı kullanım: 12 birim iş / 14 birim toplam = **%85,7** (kayıp: 2 / 14 ≈ %14,3, saf ek yük).

*Not:* Bağlam değiştirme sürelerini bekleme saydık (W = TAT − B ile tutarlı). Hocanın kabulü farklıysa onu uygula.

### 12.0.8 Soru 22

**Çözüm. Gerçek zamanlı işletim sistemi (RTOS)** uygun. Kritik ölçüt **deadline'ın güvenilir biçimde karşılanması**: ilaç dozunun gecikmesi hayati risk. Masaüstü işletim sistemi ortalama performans ve kullanım kolaylığı için tasarlanmıştır; arka plan işlemleri, güncellemeler yüzünden bir işin ne zaman çalışacağının **garantisi yoktur**. Bu bir "başarısız olursa ne olur?" sorusunun cevabıdır.

*Kısmi puan:* RTOS adı, "deadline" kavramı, gerekçe.

### 12.0.9 Soru 23

**Çözüm.** Ortalama = 0,95 × 2 + 0,05 × 100 = 1,9 + 5,0 = **6,9 ns**. Yalnızca RAM ile 100 ns; hızlanma 100 / 6,9 ≈ **14,5 kat**. Isabet oranı %95 bile olsa ortalama süreyi %5'lik ıska oranı belirler.

### 12.0.10 Soru 24

**Çözüm (iddia sınırlı doğrudur).** (1) CPU kullanımı %100'e ulaşınca ek süreç kullanımı artırmaz (Soru 16). (2) **Bellek sınırlıdır:** her süreç yer ister; fazla süreç dış parçalanma, sıkıştırma ve **takas** maliyetini artırır. (3) **Bağlam değiştirme** ek yükü ve hazır kuyruğu uzar; yanıt süresi kötüleşir. (4) Süreçler **CPU'ya bağlı** ise I/O beklemedikleri için boşluk yoktur ve ek süreç yalnızca kuyruğu uzatır. Sonuç: çoklu programlama CPU kullanımını artırır ama ödünleşim vardır.

*Kısmi puan:* iddianın I/O bekleme gerekçesiyle kısmen doğru olduğunu söyleme; her geçerli sınır.

### 12.0.11 Soru 25

**Çözüm.** İki hata var. (1) **SJF kesintisizdir**; çalışan sürecin yerini yeni gelen almaz. Bunu yapan **SRTF**'tir. (2) "Ortalama bekleme süresini en küçük yapar" iddiası SJF için (burst'ler önceden bilindiğinde, kesintisiz algoritmalar arasında) doğrudur. Doğru ifade: *"SJF, hazır süreçler arasından en kısa burst'lü olanı seçer ve bitene kadar çalıştırır; SRTF ise çalışanı daha kısa kalan süreli gelen süreçle değiştirir."* Soru 20'de SJF 4, SRTF 3,5 çıkmıştır.

### 12.0.12 Soru 26

(a) **Authentication** (kimlik doğrulama). (b) **Authorization** (yetkilendirme, izinler). (c) **Isolation** (yalıtım). (d) **Auditing** (denetim, kayıt tutma).

## Cevapların bağımsız kontrolü

Python benzetimleri (FCFS, SJF, SRTF, RR, bellek yerleştirme, kullanım, erişim süresi) çalıştırıldı ve elle izlemeyle karşılaştırıldı. Cevap anahtarını görmeyen **bağımsız bir ajan** 26 sorunun hepsini çözdü: 14 çoktan seçmelinin tamamı ve açık uçlu sorulardaki bütün sayısal sonuçlar anahtarla **aynı çıktı**. Ajanın işaretlediği üç belirsizlik (Soru 1'de sürücü/sistem programı, Soru 21'de R tanımı, Soru 23'te ıska maliyeti) soru metinlerine açıklama eklenerek giderildi.

BÖLÜM 13

# Terim sözlüğü

(İngilizce → Türkçe) Metinde İngilizce terimler Türkçe karşılıklarının yanında parantez içinde geçer. Aşağıda hepsi alfabetik sıradadır.

| İngilizce | Türkçe |
|---|---|
| abstraction | soyutlama |
| address translation | adres çevirisi |
| admit/dispatch | kabul/sevk |
| aging | yaşlandırma |
| application program | uygulama programı |
| auditing | denetim |
| authentication | kimlik doğrulama |
| authorization | yetkilendirme |
| batch system | toplu işlem sistemi |
| bottleneck | darboğaz |
| cache | önbellek |
| cache hit/miss | önbellek isabeti/ıskası |
| CLI | komut satırı arayüzü |
| compaction | sıkıştırma |
| context switch | bağlam değiştirme |
| convoy effect | konvoy etkisi |
| core | işlemci çekirdeği (kernel = işletim sistemi çekirdeği ile karıştırma) |
| CPU burst / I/O burst | CPU patlaması / I/O patlaması |
| CPU utilization | CPU kullanımı |
| CPU-bound / I/O-bound | CPU'ya bağlı / I/O'ya bağlı |
| deadline | son teslim süresi (deadline) |
| device driver | aygıt sürücüsü |
| directory | dizin |
| distribution | dağıtım |
| dynamic partitioning | dinamik bölümleme |
| embedded | gömülü |
| external/internal fragmentation | dış/iç parçalanma |
| FCFS | ilk gelen ilk hizmet alır |
| file descriptor | dosya tanımlayıcısı |
| file system | dosya sistemi |
| first/best/worst/next fit | ilk uyan / en iyi uyan / en kötü uyan / sonraki uyan |
| frame | çerçeve |

| İngilizce | Türkçe |
|---|---|
| Gantt chart | Gantt şeması |
| GUI | grafik kullanıcı arayüzü |
| hole | boşluk |
| I/O | giriş/çıkış |
| interrupt | kesme |
| isolation | yalıtım |
| job | iş |
| kernel | çekirdek |
| kernel mode | çekirdek kipi |
| latency | gecikme |
| locality | yerellik |
| memory hierarchy | bellek hiyerarşisi |
| multi-user | çok kullanıcılı |
| multiprogramming | çoklu programlama |
| multitasking | çoklu görev |
| new/ready/running/waiting/terminated | yeni/hazır/çalışıyor/bekliyor/sonlandı |
| operating system (OS) | işletim sistemi |
| overhead | ek yük |
| package manager | paket yöneticisi |
| page | sayfa |
| page fault | sayfa hatası |
| page table | sayfa tablosu |
| permission | izin |
| persistent | kalıcı |
| pipeline | boru hattı |
| preemption | kesme (CPU'yu elinden alma) |
| preemptive / non-preemptive | kesintili / kesintisiz |
| priority | öncelik |
| privilege | ayrıcalık |
| process | süreç |
| Process Control Block (PCB) | süreç denetim bloğu |
| program | program |
| program counter | program sayacı |
| ready queue | hazır kuyruğu |
| real-time OS (RTOS) | gerçek zamanlı işletim sistemi |
| register | yazmaç |
| resource manager | kaynak yöneticisi |
| response time | yanıt süresi |
| Round Robin | döngüsel sıralı (Round Robin) |
| scheduler | zamanlayıcı |
| single-tasking | tekli görev |
| SJF | en kısa iş önce |
| SRTF | en kısa kalan süre önce |
| stall | duraksama |
| starvation | açlık |
| swapping | takas |
| system call | sistem çağrısı |
| system program | sistem programı |

| İngilizce | Türkçe |
|---|---|
| temporal/spatial | zamansal/uzamsal |
| thread | iş parçacığı |
| throughput | üretilen iş (throughput) |
| time sharing | zaman paylaşımı |
| time slice / quantum | zaman dilimi / kuantum |
| trap | tuzak (yazılım kesmesi) |
| turnaround time | tamamlanma süresi (turnaround) |
| uniprogramming | tekli programlama |
| user mode | kullanıcı kipi |
| virtual memory | sanal bellek |
| volatile | uçucu |
| waiting queue | bekleme kuyruğu |
| waiting time | bekleme süresi |

BÖLÜM 14

# Hocaya sorulacak noktalar

**→ BU KONUYA NEDEN GEÇİYORUZ?**

Slaytları satır satır incelerken aşağıdaki çelişki, eksiklik ve belirsizlikleri buldum. Bunlar hocanın hatası olmak zorunda değil (slayt tasarımı, gizli slayt, sadeleştirme olabilir); ama sınavda **hangi yorumun esas alınacağı** belli olmadığı için kısa bir soru olarak sorulabilir. Her satırda notta hangi yorumu seçtiğimi de yazdım.

| # | Konum | Gördüğüm | Notta kullandığım yorum | Sorulacak soru |
|---|---|---|---|---|
| 1 | D1 s. 32 ve 37–39 | Slayt 32'de görevler "3 Resource efficiency, 4 Device management, 5 Files and protection" diye sıralı; ayrıntı slaytları ise 3 = aygıtlar, 4 = dosyalar, 5 = koruma. "Resource efficiency" için ayrı slayt yok. | Beş görev: arayüz, program çalıştırma, aygıtlar, dosyalar, koruma | "Beş görevin tam listesi hangisi?" |
| 2 | D1 s. 62 | Durum diyagramında **Çalışıyor → Hazır** oku (kesme/kuantum) eksik; s. 63'teki geçiş listesi bu geçişi içeriyor. | Okun var olduğunu kabul ettim | "Diyagramda bu ok sınavda çizilecek mi?" |
| 3 | D1 s. 80 / slayt 81 | SRTF Gantt'ında 7 ve 11 etiketleri çubuk sınırlarının altında değil, kaymış görünüyor. | Doğru sınırlar 7 ve 11 (hesapla doğrulandı) | "Etiketler tasarım kayması mı?" |
| 4 | D1 s. 78–80 / slayt 79–81 | Burst süreleri açıkça yazılmamış; Gantt'tan çıkarmak gerekiyor. | Burst'ler Gantt'tan çıkarıldı (P1 = 6, P2 = 4, P3 = 1, P4 = 5 vb.) | "Örneklerin süreleri bunlar mı?" |
| 5 | D1 s. 58 | "The role of the file manager" slaytı "MEMORY MANAGEMENT" etiketi altında ama konusu dosya yönetimi. | 5. bölümde dosya yöneticisi alt bölümü olarak işledim | "Dosya yöneticisi bellek yönetiminin parçası mı sayılıyor?" |
| 6 | D1 s. 50 | Diyagramda tek boş blok var; yan kutu "toplam boş bellek yeterli ama bitişik değil" diyor ama diyagram bu durumu henüz göstermiyor. | Sonraki durum kastedildi | "Hangi durum kastediliyor?" |
| 7 | D1 s. 71–72 | Slayt numarası 72 yok (s. 72 PDF sayfası = slayt 73). | Etiketlerde iki numarayı birlikte yazdım | "Gizli bir slayt var mı, sınav kapsamında mı?" |

| # | Konum | Gördüğüm | Notta kullandığım yorum | Sorulacak soru |
|---|---|---|---|---|
| 8 | D1 s. 85 / slayt 86 | Round Robin örneğinde son iki P3 dilimi ayrı çizilmiş (134–154, 154–162); kuyrukta tek süreç kalınca birleşik de gösterilebilir. | İki gösterim de doğru; ölçütler değişmiyor | "Gantt'ta ardışık aynı süreç dilimleri birleştirilmeli mi?" |
| 9 | RR (genel) | Aynı anda varış ve kuantum bitişi çakışınca kuyruk sırası slaytta açıkça yazmıyor. | **Yeni gelen, kesilenin önüne** girer | "Bu çakışmada hangi kural esas?" |
| 10 | Bellek hiyerarşisi | Slayt katmanlar için sayı vermiyor (ns, µs). | Sayıları **kaynak dışı mertebe** olarak işaretledim | "Sayısal değer ezberlenecek mi?" |
| 11 | Sınav formatı | Format ve ağırlık belli değil. | Karışık format varsaydım | "Çoktan seçmeli/klasik oranı nedir; hesap makinesi serbest mi?" |

**↺ HATIRLATMA**

Hocadan cevap aldığında ilgili bölümün "Sık yapılan hatalar" kutusunun yanına küçük bir not düşmek yeterlidir; önemli olan sınavda **hocanın** kuralını uygulamandır.

BÖLÜM 15

# Kaynak kapsamı kontrol tablosu

**→ BU KONUYA NEDEN GEÇİYORUZ?**

D1'in (95 sayfa) her ana başlığı ve bu notta işlendiği yer. "Tam" = slayttaki bütün bilgi ve örnekler işlendi. Sayfa/slayt numarası kuralı: fiziksel sayfa 72'den sonra slayt numarası = sayfa + 1 (Başlangıç bölümüne bak).

| D1 sayfaları | Kaynak başlığı | Nottaki yer | Durum |
|---|---|---|---|
| 1–4 | Kapak, haftanın soruları, öğrenme çıktıları, LinkedIn kariyer görevi | Başlangıç; bölüm açılışları | Kısmi: kariyer görevi sınav konusu değil, işlenmedi (neden: bilgisayar mühendisliği kariyer planı ödevi; çalışma notu kapsamı dışı) |
| 5–12 | 1. İşletim sistemi temelleri | Bölüm 1 | Tam |
| 13–21 | 2. İşletim sistemi aileleri | Bölüm 2 | Tam (s. 21 sınıf etkinliği çözüldü) |
| 22–30 | 3. Tarihçe ve kaynaklar | Bölüm 3 | Tam |
| 31–47 | 4. Temel görevler ve kipler | Bölüm 4 | Tam |
| 48–58 | 5. Bellek yönetimi | Bölüm 5 | Tam (70K alıştırması çözüldü) |
| 59–70 | 6. Süreç yönetimi | Bölüm 6 | Tam |
| 71–87 (slayt 73–88) | 7. CPU zamanlama | Bölüm 7 | Tam (bütün Gantt örnekleri ve alıştırma çözüldü) |
| 88–95 (slayt 89–95) | 8. Bellek hiyerarşisi, haftanın özeti, gelecek hafta | Bölüm 8 | Tam |
| (ek) | Konular arası bağlantılar | Bölüm 9 | Nota özgü |
| (ek) | Hızlı tekrar, deneme sınavı, cevap anahtarı | Bölüm 10–12 | Nota özgü |

**Sayfa numarası belirsizlikleri:** (1) Slayt 72 yok; (2) bölüm kapak slaytlarında numara basılı değil, bunlar fiziksel sayfa ile anıldı. Kaynakta kod olmadığından kod doğrulaması yoktur; **hesaplar Python ile doğrulandı** (zamanlama, bellek yerleştirme, kullanım, erişim süresi).